

PRINT-FLOW-360 · RESEARCH LIBRARY

Engineering Ops

7 research documents · each with plain-language key takeaways

Compiled 2026-06-26

Table of Contents

1.	Backup & Restore Runbook — Print-Flow-360	
2.	Docker Containerization Plan — Print-Flow-360	
3.	Docker Runbook — Print-Flow-360	
4.	Legal, Compliance & Data Privacy	
5.	Payments, Billing & Financial Correctness	
6.	Production Readiness & Reliability Reference	
7.	Multi-Tenant Isolation & Access-Control Security Review — 2026-06-15	

Backup & Restore Runbook — Print-Flow-360

Source: [readme/research/engineering-ops/BACKUP_AND_RESTORE_RUNBOOK_2026-06-15.md](#)

IN SHORT — KEY TAKEAWAYS

This document is a step-by-step guide for how to back up all the data in Print-Flow-360 and how to bring it back if something goes wrong. It matters because right now the platform has NO backups set up at all — if the database crashed today, every order, customer, and uploaded file would be permanently lost.

The main parts explained simply:

- **Current state (critical warning)** — A check done on 15 June 2026 found that zero backup tools are installed. There are no scheduled backups, no database recovery tools, and no protection on uploaded files. This is the most urgent finding in the whole document.
- **What gets backed up** — Four things need protection: the main database (all tenant orders, customers, products), the admin database, uploaded files and print-ready PDFs stored in Amazon S3 (cloud file storage), and the app's secret keys. If the print-ready PDF for a paid order is lost, that customer never gets what they bought.
- **Database backup with pgBackRest** — A tool called pgBackRest saves a copy of the database every day and also records every change made in between. This means you can roll the database back to any point in the last few days — even to one minute before a mistake happened. Target: at most 5 minutes of data lost.
- **Database backup with logical dumps** — A simpler backup method (called pg_dump or spatie/laravel-backup) that saves a plain copy of the database once a day. Less powerful than pgBackRest but portable and easy to move offsite. Acts as a safety net.
- **File storage backup (Amazon S3)** — Uploaded images and print PDFs live in a cloud bucket. Turning on “versioning” means every file change is kept, so deleted or overwritten files can be recovered. Replication copies everything to a second region automatically.
- **Backup heartbeat (dead-man's switch)** — A small monitoring service that expects a “ping” every time a backup runs. If the ping does not arrive, it sends an alert. Without this, a backup can silently stop working for weeks and nobody notices.
- **Restore: full database** — Step-by-step commands to bring the whole database back from the latest good backup. Takes 1–4 hours.
- **Restore: point-in-time (PITR)** — Roll the database back to an exact date and time, for example “just before a bad migration ran at 14:07.” Only possible once pgBackRest and WAL archiving are set up.
- **Restore: one tenant's data** — If a single store owner accidentally deletes their records, you can recover just their rows from a backup without touching any other tenant's data.
- **Post-restore reconciliation** — After any restore, the database and the file storage may be out of sync (the database says a file exists but the file is gone). A reconciliation check compares the two and flags missing files before any customer sees a broken download.
- **Restore drills** — A backup that has never been tested is not a real backup. The runbook says to do a practice restore every week (automated) and a full game-day drill every month. Results must be recorded.
- **Disaster scenarios** — Ready-made decision trees for five situations: database corruption, one tenant's data accidentally deleted, a paid order's print file going missing, Redis (the job queue) losing jobs, and the whole hosting region going down.

- **Incident roles** — During an outage, four roles are assigned: Incident Commander (makes all decisions), Restore Operator (runs the commands), Comms Lead (writes status updates for customers), and Scribe (timestamps everything for the review after).

What to do with this: The most urgent action is to install backup tooling before anything else — start with pgBackRest and S3 versioning, then wire the daily schedule and heartbeat monitors. Until at least one verified backup exists and has been successfully restored in a drill, the platform has no recovery capability at all.

Audience: On-call SRE / ops engineer. This is an operational runbook — copy-pasteable commands, decision trees, drills. **Last reviewed:** 2026-06-15 **Scope:** PostgreSQL (central `live_db` + `admin_db`), object storage (S3 `printfLOW360` — uploaded files & print-ready PDFs), Redis (queues), secrets/env.



Current state — read this first

A codebase scan on 2026-06-15 found **no backup tooling installed and no recovery capability**:

Capability	State today	Risk
<code>spatie/laravel-backup</code>	Not installed	Critical
WAL archiving / PITR (pgBackRest / WAL-G)	Not configured	Critical
Scheduled backups	None (<code>routes/console.php</code> has no backup task)	Critical
S3 bucket versioning / replication / Object Lock	Not verified / likely off	High
Redis for queues	Not configured — <code>QUEUE_CONNECTION=sync</code>	High
Backup heartbeat / dead-man's-switch	None	High
Read replica / standby	None	High

Every section below marked **⚠ Prerequisite: not yet set up** is the **TARGET procedure**. It will not run until the prerequisite is installed/configured. Do the prerequisites in this order before relying on any restore:

1. Provision a **dedicated PostgreSQL backup repository** (separate host or S3 bucket) + install **pgBackRest** on the DB host.
2. Enable **WAL archiving** in `postgresql.conf` (`archive_command` , `archive_timeout=300`).
3. Enable **S3 versioning + cross-region replication + Object Lock** on `printfLOW360` .
4. Install `spatie/laravel-backup` as the portable logical-dump safety net.
5. Wire **backup heartbeats** to a dead-man's-switch ([Healthchecks.io](#) / Better Stack).
6. Stand up a **scratch restore environment** and schedule the **restore drill**.

1. What we back up & where

1.1 Inventory

#	Asset	What it holds	Location	Backup method (target)
1	PostgreSQL <code>live_db</code>	Central app data + tenant metadata (<code>tenants</code> table) + all tenant business rows. 122/126 models are shared-DB with <code>tenant_id</code> (<code>BelongsToTenant</code>), so one PITR covers all tenants at the same timestamp .	PG host <code>:5432</code> , db <code>live_db</code>	pgBackRest base + WAL (PITR) and nightly <code>pg_dump</code> (portable)
2	PostgreSQL <code>admin_db</code>	Super-admin / SaaS-landlord operations	PG host <code>:5432</code> , db <code>admin_db</code>	Same as <code>live_db</code> (add to pgBackRest stanza + nightly dump)
3	Object storage — uploaded files	Customer uploads, product images, branding (<code>{tenant}/branding/brand.css</code>). DB stores relative paths only ; bytes live here.	S3 bucket <code>printflow360</code>	Versioning + cross-region replication + Object Lock
4	Object storage — print-ready PDFs	Paid-order production artwork: <code>designs/{design_id}/files/print-ready-{ts}.pdf</code> , linked via <code>designer_documents.print_ready_file</code> and snapshotted onto orders. Loss after payment = customer never gets what they bought .	S3 bucket <code>printflow360</code> (per-tenant prefix; pdf-service may use per-tenant buckets)	Versioning + replication + Object Lock (longer retention)
5	Redis (queues)	In-flight BullMQ jobs (pdf-service) + Laravel queue once moved off <code>sync</code>	Redis (queue-dedicated instance)	AOF persistence; treated as recoverable-by-replay, not as a backup of record
6	Secrets / env	<code>.env</code> (all 5 apps), <code>APP_KEY</code> , <code>PDF_SERVICE_INTERNAL_SECRET</code> , AWS/Stripe/gateway keys	Host filesystem + secrets manager	Secrets manager + encrypted nightly snapshot; IaC in git

Note on tenancy model: `config/tenancy.php` ships the stancI `PostgreSQLDatabaseManager` (database-per-tenant capability), but `DatabaseTenancyBootstrapper` is **commented out** and 122/126 models carry `tenant_id` — i.e. the running model is **shared DB, row-scoped by `tenant_id`**. The runbook assumes shared-DB. If per-tenant databases are ever enabled, every restore step that names `live_db` must be repeated per tenant database, and §3.3 (single-tenant restore) changes from a row filter to a per-database restore.

1.2 Retention & 3-2-1(-1-0)

Target: **3** copies, on **2** media/locations, **1** offsite, **1** immutable (Object Lock), **0** errors (heartbeat-verified).

Asset	Daily	Weekly	Monthly	Offsite / immutable
PG base backup (pgBackRest)	full daily	—	—	repo in 2nd region; Object Lock
PG WAL (PITR)	continuous (<code>archive_timeout=300s</code>)	—	—	same repo, immutable
PG logical dump (spatie/pg_dump)	7 days	4 weeks	12 months	separate S3 bucket, different region, Object Lock
S3 file objects	versioned (every write)	—	—	cross-region replica + Object Lock
Secrets/env snapshot	nightly	—	12 months	secrets manager + encrypted offsite

- **RPO targets:** DB $\approx$ **5 min** (WAL); object files $\approx$ **near-zero** (versioning + replication).
- **RTO targets:** core DB + storefront $\approx$ **1–4 hrs** (backup-restore tier; no warm standby yet).
- **Encryption:** all backups encrypted at rest (pgBackRest `--repo-cipher-type=aes-256-cbc` ; S3 SSE).

2. Backup procedures

2.1 PostgreSQL — continuous archiving + PITR (pgBackRest)

 **Prerequisite: not yet set up.** Requires pgBackRest installed, a repo (S3 or 2nd host), and WAL archiving enabled in `postgresql.conf`.

Step A — enable WAL archiving in `postgresql.conf` (then `systemctl restart postgresql`):

```
# postgresql.conf
wal_level = replica
archive_mode = on
archive_command = 'pgbackrest --stanza=printflow archive-push %p'
archive_timeout = 300           # forces a WAL segment at least every 5 min -> ~5 min RPO
max_wal_senders = 3
```

Step B — pgBackRest config (`/etc/pgbackrest/pgbackrest.conf`):

```
[global]
repo1-path=/var/lib/pgbackrest
repo1-type=s3
repo1-s3-bucket=printfLOW360-pgbackrest
repo1-s3-region=us-east-1
repo1-s3-endpoint=s3.us-east-1.amazonaws.com
repo1-retention-full=4          # keep 4 full backups (~ weekly fulls -> ~1 month)
repo1-cipher-type=aes-256-cbc
repo1-cipher-pass=<from secrets manager>
start-fast=y

[printfLOW]
pg1-path=/var/lib/postgresql/16/main
pg1-port=5432
```

Step C — create stanza & first full backup:

```
sudo -u postgres pgbackrest --stanza=printfLOW stanza-create
sudo -u postgres pgbackrest --stanza=printfLOW --type=full backup
sudo -u postgres pgbackrest --stanza=printfLOW check      # verifies archive_command works
```

Step D — schedule (cron on the DB host). Heartbeat ping wraps each run (see §2.4):

```
# Weekly full (Sun 02:00), daily incremental (02:00 other days)
0 2 * * 0 postgres pgbackrest --stanza=printfLOW --type=full backup && curl -fsS https://hc-
ping.com/<PG_FULL_UUID> || curl -fsS https://hc-ping.com/<PG_FULL_UUID>/fail
0 2 * * 1-6 postgres pgbackrest --stanza=printfLOW --type=incr backup && curl -fsS https://hc-
ping.com/<PG_INCR_UUID> || curl -fsS https://hc-ping.com/<PG_INCR_UUID>/fail
```

2.2 PostgreSQL — portable logical dumps (safety net)

WAL/PITR is the primary recovery path; logical dumps are the **portable, schema-version-independent** net and the basis for single-tenant extraction. Two ways to run them:

Option 1 — **spatie/laravel-backup** (preferred; ships DB + app files to offsite S3):

 **Prerequisite: not yet set up.** `composer require spatie/laravel-backup`, publish config, point `backup.destination.disks` at a **separate** offsite S3 bucket (different region, Object Lock), set `backup.backup.source.databases => ['pgsql', 'admin_pgsql']`.

```
php artisan backup:run          # dump DB(s) + bundle to timestamped zip -> offsite S3
php artisan backup:run --only-db # DB only (faster, what the nightly cron uses)
php artisan backup:clean        # apply retention (daily 7 / weekly 4 / monthly 12)
php artisan backup:list         # show what exists + health
```

Schedule in `routes/console.php` (this is the missing piece today):


```
use Illuminate\Support\Facades\Schedule;

Schedule::command('backup:clean')->dailyAt('01:30');
Schedule::command('backup:run --only-db')->dailyAt('02:00')
    ->onSuccess(fn () => Http::get('https://hc-ping.com/<SPATIE_DB_UUID>'))
    ->onFailure(fn () => Http::get('https://hc-ping.com/<SPATIE_DB_UUID>/fail'));
Schedule::command('backup:run')->weeklyOn(0, '03:00'); // full DB+files weekly
```

⚠ Prerequisite gotcha: `Schedule::` only fires if the system cron runs `php artisan schedule:run` every minute, **and** the scheduler itself is heartbeat-monitored (§2.4). With `QUEUE_CONNECTION=sync`, `backup:run` runs inline — acceptable for nightly batch, but move queues to Redis before relying on async backup jobs.

Option 2 — raw `pg_dump` (no extra packages; use when spatie isn't installed yet):

```
# Custom format (-Fc) = compressed, parallel-restorable, selective. Run as a role with read on all schemas.
TS=$(date +%Y%m%dT%H%M%S)
PGPASSWORD=$DB_PASS pg_dump -h "$DB_HOST" -p 5432 -U "$DB_USER" -Fc -Z6 -f "/backups/live_db-$TS.dump" live_db
PGPASSWORD=$ADMIN_DB_PASS pg_dump -h "$ADMIN_DB_HOST" -p 5432 -U "$ADMIN_DB_USER" -Fc -Z6 -f
"/backups/admin_db-$TS.dump" admin_db
# Encrypt + push offsite (different region bucket with Object Lock)
gpg --encrypt --recipient ops@printflow ... "/backups/live_db-$TS.dump"
aws s3 cp "/backups/live_db-$TS.dump.gpg" s3://printflow360-db-backups/daily/ --region eu-west-1
```

2.3 Object storage (S3) — versioning, replication, Object Lock

⚠ Prerequisite: not yet set up. Enable on bucket `printflow360`. Object versioning makes a point-in-time bucket view reconstructable and turns deletes into recoverable delete-markers — this is what keeps DB-referenced print files restorable.

```
# Versioning (delete becomes a recoverable delete-marker, not a real delete)
aws s3api put-bucket-versioning --bucket printflow360 \
    --versioning-configuration Status=Enabled

# Cross-region replication to a 2nd-region bucket (near-zero object RPO)
aws s3api put-bucket-replication --bucket printflow360 \
    --replication-configuration file://replication.json

# Object Lock (immutability) — must be enabled at/after bucket creation with versioning on
aws s3api put-object-lock-configuration --bucket printflow360 \
    --object-lock-configuration '{"ObjectLockEnabled":"Enabled","Rule":{"DefaultRetention":
{"Mode":"GOVERNANCE","Days":35}}}'

# Server-side encryption
aws s3api put-bucket-encryption --bucket printflow360 \
    --server-side-encryption-configuration '{"Rules":[{"ApplyServerSideEncryptionByDefault":
{"SSEAlgorithm":"aws:kms"}}}]'
```

Soft-delete print files rather than hard-deleting: if the app/pdf-service ever deletes objects, route them through a lifecycle-expire quarantine (window **longer than DB backup retention**) so a restored older DB

row still finds its object. Per-tenant buckets used by pdf-service must each get the same versioning + replication config — enumerate them.

2.4 Backup heartbeat (dead-man's-switch)

⚠ Prerequisite: not yet set up. A silent backup failure is the worst failure. Each scheduled job pings a heartbeat on success; if the ping doesn't arrive in the expected window, the monitor pages you.

- Create one check per job in **Healthchecks.io** (or Better Stack): `pg-full`, `pg-incr`, `spatie-db-nightly`, `s3-replication-lag`, `laravel-scheduler`.
- Success ping at end of job; failure ping (`/fail` URL) in the error branch (see crons in §2.1/§2.2).
- Add a **scheduler heartbeat** so a dead `schedule:run` cron is caught:

```
* * * * * cd /var/www/printflow && php artisan schedule:run >> /dev/null 2>&1 && curl -fsS https://hc-ping.com/<SCHEDULER_UUID>
```

- Recommended belt-and-braces: install `spatie/laravel-health` with the `BackupsHealthCheck` so `/up` -style readiness also reports “last backup age” and goes 503 if the newest backup is stale.

3. Restore procedures

Golden rule: restore **DB and object storage to a consistent point**, then run the **reconciliation job** (§3.5). A DB restored to time T can reference files added/deleted after T → orphaned (safe) or **missing** (customer-facing breakage). Never declare “recovered” before reconciliation passes.

3.1 Full database restore (latest good backup, pgBackRest)

```
# 1. Stop the app (maintenance mode) so nothing writes during restore.
php artisan down --secret="restore-2026" --render="errors:503"

# 2. Stop Postgres and restore the latest backup into the data dir.
sudo systemctl stop postgresql
sudo -u postgres pgbackrest --stanza=printflow --delta restore

# 3. Start Postgres; it replays WAL to the end of the backup automatically.
sudo systemctl start postgresql

# 4. Sanity: connect and confirm key tables exist + row counts look sane.
psql -h localhost -U "$DB_USER" -d live_db -c "SELECT count(*) FROM tenants;"
psql -h localhost -U "$DB_USER" -d live_db -c "SELECT count(*) FROM orders;"

# 5. Reconcile files (§3.5), then bring app up.
php artisan up
```

From a logical dump instead (portable / cross-version):

```
createdb -h localhost -U "$DB_USER" live_db_restored
pg_restore -h localhost -U "$DB_USER" -d live_db_restored --jobs=4 --clean --if-exists /backups/live_db-<TS>.dump
```

3.2 Point-in-time recovery (PITR) to a timestamp

Use when you must recover to **just before** a bad event (e.g. a destructive migration at 14:07).

```
php artisan down --secret="restore-2026"
sudo systemctl stop postgresql

# Restore base + tell PG to replay WAL only up to the target time.
sudo -u postgres pgbackrest --stanza=printflow \
  --type=time --target="2026-06-15 14:05:00+00" \
  --target-action=promote --delta restore

sudo systemctl start postgresql
# PG replays WAL to 14:05:00 then promotes. Verify the bad change is gone:
psql -h localhost -U "$DB_USER" -d live_db -c "SELECT max(created_at) FROM orders;"
```

- **Target options:** `--type=time` (timestamp), `--type=xid` (transaction id), `--type=lsn`, or `--type=immediate` (end of base backup).
- After promotion the timeline diverges — take a **fresh full backup immediately** so future PITR works from the new timeline.
- **RPO note:** you can only recover to a moment covered by archived WAL; with `archive_timeout=300` the worst-case gap is ~5 min.

3.3 Single-tenant restore (one tenant asks to undo a deletion)

Cheaper than a full PITR. Two paths depending on blast radius:

Path A — restore specific tenant rows from a logical dump (shared-DB model). Restore the whole dump to a **scratch** DB, then copy back only the affected tenant's rows:

```
# 1. Restore yesterday's dump into a throwaway DB (does NOT touch prod).
createdb -h localhost -U "$DB_USER" live_db_scratch
pg_restore -h localhost -U "$DB_USER" -d live_db_scratch --jobs=4 /backups/live_db-<TS>.dump

# 2. Inspect the tenant's data in the scratch DB.
psql -d live_db_scratch -c "SELECT count(*) FROM orders WHERE tenant_id = '<TENANT_UUID>';"

# 3. Export just that tenant's affected table(s) and re-insert into prod.
# (Do this per table, respecting FK order; wrap in a transaction; dry-run first.)
pg_dump -h localhost -U "$DB_USER" -d live_db_scratch \
  --data-only --table=orders \
  | ... filter to tenant_id = '<TENANT_UUID>' ... \
  | psql -d live_db # into prod, inside BEGIN; ... COMMIT;
```

⚠ Re-inserting rows risks PK/unique collisions and FK violations. Always: scratch-restore → diff → script the minimal `INSERT ... ON CONFLICT / UPDATE` → run inside a transaction → reconcile files (§3.5). If unsure, prefer full PITR to just-before-deletion (§3.2).

Path B — if per-tenant databases are ever enabled: restore that tenant's database from its own pgBackRest stanza / dump; no row filtering needed.

Recover the tenant's deleted files from S3 versioning (delete-markers):

```
# List versions/delete-markers under the tenant prefix and remove the delete-marker to "undelete".
aws s3api list-object-versions --bucket printflow360 --prefix "<TENANT_UUID>/" \
  --query "DeleteMarkers[].{Key:Key,VersionId:VersionId}"
aws s3api delete-object --bucket printflow360 --key "<KEY>" --version-id "<DELETE_MARKER_VERSION_ID>"
```

3.4 Restoring object-storage files

```
# Full bucket restore from cross-region replica (DR region) back to primary:
aws s3 sync s3://printflow360-replica/ s3://printflow360/ --source-region eu-west-1 --region us-east-1

# Restore a single object to a prior version (e.g. a corrupted print-ready PDF):
aws s3api list-object-versions --bucket printflow360 \
  --prefix "designs/<DESIGN_ID>/files/" --query "Versions[.
{Key:Key,VersionId:VersionId,LastModified:LastModified}]"
aws s3api copy-object --bucket printflow360 \
  --copy-source "printflow360/designs/<DESIGN_ID>/files/print-ready-<ts>.pdf?versionId=<GOOD_VERSION_ID>" \
  --key "designs/<DESIGN_ID>/files/print-ready-<ts>.pdf"
```

3.5 Post-restore reconciliation (mandatory — DB ↔ storage consistency)

 **Prerequisite: build this as an artisan command** (e.g. `php artisan storage:reconcile`). It is the safeguard against the project's "silent-lie" bug class — a paid order whose print file 404s on download.

What it must do, **per tenant base path**:

1. Enumerate every DB-stored relative path: `designer_documents.print_ready_file`, order `design_print_ready_file` snapshots, and every `HasImageFields` column across models.
2. Diff each against actual bucket contents (`aws s3 ls` / `Storage::exists`).
3. **Missing files** (DB row → no object): attempt restore of the object's prior version from S3 versioning; if unrecoverable, set the order/design file status to a recoverable state and surface a plain-language recovery action — **never** leave a broken download. Notify staff for paid orders.
4. **Orphans** (object → no DB row): list for later cleanup (do not auto-delete).
5. Confirm order/quote/invoice **snapshot** file references still resolve.
6. Emit a report (counts of OK / missing / restored / orphaned) to logs + alert channel.

Manual spot-check meanwhile:

```
psql -d live_db -t -c "SELECT print_ready_file->>'image' FROM designer_documents WHERE print_ready_file IS NOT NULL;" \
| while read p; do aws s3 ls "s3://printflow360/$p" >/dev/null 2>&1 || echo "MISSING: $p"; done
```

4. Restore DRILL / verification

“A backup you’ve never restored isn’t a backup.” Validate *recoverability*, not just that backup jobs ran green.

4.1 Cadence

- **Weekly:** automated drill of the critical tier (DB PITR into scratch + smoke checks).
- **Monthly:** full game-day — restore DB **and** object storage into an isolated scratch env, boot the app, walk a real order. Drill more often than the shortest retention window.

4.2 Scratch-env drill procedure

Use the existing test DB pattern as the sandbox: `.env.testing` → `printflow360_test` on pgsql. Never point a drill at prod.

```
# 1. Restore latest base + WAL to a scratch instance / DB.
sudo -u postgres pgbackrest --stanza=printflow --type=time \
  --target="$(date -u -d '10 min ago' +%Y-%m-%d %H:%M:%S+00')" \
  --target-action=promote --delta restore # against a scratch PGDATA, not prod

# 2. Restore a slice of object storage into a scratch bucket.
aws s3 sync s3://printflow360-replica/designs/ s3://printflow360-drill/designs/ --region eu-west-1

# 3. Boot the app against scratch DB + scratch bucket.
APP_ENV=drill DB_DATABASE=live_db_drill FILESYSTEM_DISK=s3 AWS_BUCKET=printflow360-drill \
  php artisan optimize:clear && php artisan up && curl -fsS http://localhost:8000/up
```

4.3 What to verify (record every value, trend over time)

Check	How	Pass criteria
Restore duration (RTO)	wall-clock of steps 1–3	≤ 4 hr target; flag regressions as data grows
Recovered point (RPO)	<code>SELECT max(created_at) FROM orders;</code> vs intended	gap ≤ 5 min
Key row counts	<code>tenants</code> , <code>orders</code> , <code>customers</code> , <code>designer_documents</code> , <code>invoices</code>	within expected delta of prod
Constraint/FK integrity	<code>psql -d live_db_drill -c "SET CONSTRAINTS ALL IMMEDIATE;"</code> + sample joins	no violations
A known order's print file opens	pick a paid order UUID; resolve its <code>design_print_ready_file</code> ; download from scratch bucket; open the PDF	valid, non-zero, opens in a reader
App boots	<code>curl /up</code> 200; load a storefront page; log in to admin	no 5xx
DB ↔ storage diff	run reconciliation (§3.5) against scratch	0 missing for the restored slice
Manual-step count	count steps needing a human	trend down (each = future automation target)

4.4 Recording results

 **Prerequisite: not yet set up.** Append each drill to a tracked log + push the pass/fail to the heartbeat monitor so a silent drill failure is impossible.

Record per drill: date, operator, backup-set used, restore duration, recovered point, every check pass/fail, anomalies, follow-up actions. Keep the log in version control (e.g. `readme/drills/` or an ops sheet). A drill that didn't report its result counts as **failed**.

4.5 Teardown

```
dropdb live_db_drill; aws s3 rb s3://printflow360-drill --force
```

5. Disaster scenarios playbook

Decision entry point: identify the scenario, declare severity, page the Incident Commander, then follow the branch. Default to **manually-initiated, push-button** recovery (automated steps, human decision) — no auto-failover on false alarms.

5.1 Prod DB corrupted / data loss

- RTO 1–4 hr · RPO ≤ 5 min

1. `php artisan down` — stop writes immediately.
2. Determine corruption time. If a specific bad event → **PITR to just-before** (§3.2). If general corruption → **full restore latest** (§3.1).
3. Take a fresh full backup post-promotion.
4. Run reconciliation (§3.5).
5. `php artisan up`; verify §4.3 checks; post status update.

5.2 Accidental tenant data deletion (one tenant)

- **RTO 1–2 hr · RPO ≤ 24 hr (last dump) or ≤ 5 min (PITR)**

1. Identify `tenant_id` + tables affected + deletion time.
 2. Scratch-restore yesterday's dump (§3.3 Path A); do **not** touch prod yet.
 3. Diff scratch vs prod for that tenant; script minimal re-insert/update inside a transaction.
 4. Undelete the tenant's files from S3 versioning (§3.3).
 5. Reconcile (§3.5); confirm with the tenant.
- If the deletion also corrupted shared data or the scope is large → escalate to full PITR (§3.2).

5.3 Lost/corrupt print file after payment

- **RTO < 1 hr · RPO near-zero (versioning)** — customer-facing; high priority.

1. Resolve the order's `design_print_ready_file` (or live `designer_documents.print_ready_file`).
2. Restore the object's prior good version from S3 versioning (§3.4).
3. If no prior version exists → **regenerate** via pdf-service for that design (idempotent, keyed on design/order), then re-link.
4. Run reconciliation for that order; confirm the customer download opens; notify the print shop.
5. Never let the customer see a silently-missing file — set a recoverable status + recovery action meanwhile.

5.4 Redis loss (in-flight queue jobs)

- **RTO < 30 min · RPO = unpersisted jobs (minimize with AOF)**

1. Redis is the **durability boundary**, not a backup of record. With AOF (`appendfsync everysec`) + `noeviction` , a clean restart recovers persisted jobs; BullMQ stalled-job recovery re-runs in-flight jobs (safe **only if jobs are idempotent**).
2. If Redis data is truly lost: identify paid orders whose print-file job was in-flight (cross-check `orders` / `designer_documents` file status against expected). Re-enqueue generation for any order missing its print-ready file (idempotent → no double work).
3. Verify queue depth/age return to normal; alert if oldest waiting job > 5 min.

 **Prerequisite:** Redis must be configured with AOF + `noeviction` on a **dedicated, HA (replica + Sentinel)** instance separate from cache. Today queues run `sync` — there is nothing to recover, but a crashed web request silently loses the job. Moving critical file work to Redis + Horizon is a precondition for this scenario to even be recoverable.

5.5 Region outage

- **RTO 1–4 hr · RPO ≤ 5 min (DB WAL) / near-zero (S3 replica)**

1. Declare DR; Incident Commander activates failover.
2. Provision DB host in the DR region; restore from the cross-region pgBackRest repo + replay WAL (§3.1/§3.2).
3. Point app at the **cross-region S3 replica** bucket (`AWS_BUCKET`).
4. Restore secrets/env from secrets manager; redeploy app from IaC/git.
5. Repoint DNS / health checks to DR; run §4.3 verification; run reconciliation (§3.5).
6. Communicate ETA via status page.

This is the **Backup & Restore** DR tier. Sub-30-min RTO would require Pilot Light / Warm Standby (always-on replicated DB + IaC-deployable app) — not warranted at current scale; revisit when an hours-long RTO becomes unacceptable.

6. Roles & comms during an incident

6.1 Roles (assign by expertise; name a backup for each)

Role	Responsibility
Incident Commander (IC)	Declares the incident + severity, decides activation/failover, owns the decision tree, ends the incident. Single decision-maker — removes “who calls it” ambiguity.
Restore Operator(s)	Executes §3 restore steps; reports progress/blockers to IC; never improvises destructive steps without IC sign-off.
Comms Lead	Owns status page + customer/staff updates; shields operators from inbound questions.
Scribe	Timestamps every action/decision for the post-incident review and actual-vs-target RTO/RPO.

6.2 Call tree / contacts

 **Prerequisite: fill in real contacts and keep current.** Store offline (not only in the system that may be down).

- Internal: IC, Restore Operator, Comms Lead (phone + escalation order).
- Vendors: DB host / managed PG support, S3 provider support, DNS provider, payment gateways (Stripe/Razorpay), pdf-service host.
- Monitoring: [Healthchecks.io](https://healthchecks.io) / Better Stack, Sentry (once installed) — link to the alert that fired.

6.3 Flow

1. Alert fires (heartbeat miss, health 503, or human report) → on-call acknowledges.
2. On-call pages **IC**; IC declares severity and assigns roles.

3. Comms Lead posts initial status (“investigating”) within 15 min.
 4. Restore Operator follows the relevant §5 branch; Scribe logs.
 5. IC declares recovery **only after §4.3 verification + §3.5 reconciliation pass**.
 6. Comms Lead posts resolution.
 7. **Post-incident (within 48 hr):** compare actual vs target RTO/RPO, log lessons, file follow-ups (automate manual steps, fix gaps). Update this runbook in the same PR.
-

Appendix — quick command reference

```
# Backups
pgbackrest --stanza=printflow --type=full backup      # full
pgbackrest --stanza=printflow --type=incr backup      # incremental
pgbackrest --stanza=printflow check                   # verify archiving
php artisan backup:run --only-db                      # spatie logical dump (once installed)

# Restore
pgbackrest --stanza=printflow --delta restore          # latest
pgbackrest --stanza=printflow --type=time --target="YYYY-MM-DD HH:MM:SS+00" --target-action=promote --delta
restore # PITR
pg_restore -d live_db --clean --if-exists --jobs=4 file.dump # from logical dump

# Object storage
aws s3api list-object-versions --bucket printflow360 --prefix "<prefix>/"
aws s3 sync s3://printflow360-replica/ s3://printflow360/ --source-region eu-west-1

# App
php artisan down --secret="restore-2026" / php artisan up
curl -fsS http://localhost:8000/up                    # liveness
```

Docker Containerization Plan — Print-Flow-360

Source: [readme/research/engineering-ops/DOCKER_CONTAINERIZATION_PLAN_2026-06-16.md](#)

IN SHORT — KEY TAKEAWAYS

This document is a plan to package the entire Print-Flow-360 platform — all five apps plus the database and file storage — into self-contained “boxes” called Docker containers. Right now, setting up the platform on a new server is a manual, error-prone process. The goal is to reach a point where one command starts everything correctly, every time, on any server.

The main parts explained simply:

- **Docker containers** — Think of each container as a sealed box that includes an app and everything it needs to run. You ship the box, not a list of setup instructions. This makes deployments faster and more reliable.
- **The five apps** — The platform has five separate programs: the main backend (Laravel API), the admin dashboard, the customer-facing storefront, the design studio, and a PDF/image processing service. The plan covers packaging all five.
- **The existing Docker file is broken** — There is already a Docker setup file, but it is wired to the wrong database type (MySQL instead of PostgreSQL, which is what the platform actually uses). It also leaves out three of the five apps entirely. The plan fixes all of this before adding anything new.
- **Database and storage services** — The containers also need to run a database (PostgreSQL), a fast memory cache (Redis, used for speed and background jobs), and a file storage service (MinIO for local development, real Amazon S3 in production). These are shared by all apps.
- **Reverse proxy / traffic router** — A “traffic director” (Traefik or nginx) sits in front of everything and sends each web request to the right app. It also handles the subdomain routing that lets each tenant store have its own web address (e.g. [myshop.example.com](#)).
- **Shared secrets must match** — Several apps share security keys. If those keys don't match exactly, requests silently fail with no clear error. The plan lists which keys must be kept in sync and warns that some real keys were accidentally saved in code files and need to be changed immediately.
- **Four-phase rollout** — The work is split into phases: (0) fix the broken database wiring, (1) package the backend, (2) package the frontends, (3) add the traffic router, (4) harden for production. Each phase is independently testable.

What to do with this:

The most urgent action is fixing the existing broken Docker file (wrong database, missing apps) before any other container work begins. Separately, any real passwords or API keys that were accidentally saved in code files should be rotated (replaced) right away, as they may already be exposed.

Status: Strategy / design doc. Docker is greenfield here — nothing in this plan is built yet. **Audience:** Developers + ops. **Companion doc:** [DOCKER_RUNBOOK_2026-06-16.md](#) — the hands-on build/run/operate guide. This doc is the *why* and the *target*; the runbook is the *how*. **Date:** 2026-06-16

1. Why containerize — current pain & goals

Print-Flow-360 is five apps in one repo (Laravel API, admin Nuxt, storefront Nuxt, designer SPA, pdf-service) plus Postgres, Redis, and S3. Today there is **no usable container story**:

- The real production path is **bare-metal + PM2** (`deploy/deploy.sh` , PM2 process names `printflow-api/admin/store/queue/schedule`). There is no committed PM2 ecosystem file (`deploy/pm2/ecosystem.config.cjs` is referenced but missing) and no nginx config in `deploy/` .
- The one committed `docker-compose.yml` is the **stock Laravel Sail file** with a bolted-on pdf-service stack, and it is broken on its core assumption (MySQL vs PostgreSQL — see §2).
- Standing up a dev environment means manually running 4–5 processes, wiring env vars by hand, and remembering the SSR-vs-browser API split and the tenant-host quirks.

Goals

Goal	What “done” looks like
One-command local environment	<code>docker compose up</code> brings up Postgres, Redis, MinIO, the API, both Nuxt apps, and pdf-service, correctly wired.
Reproducible, immutable images	Each app builds to a self-contained image; no host bind-mounts of source in prod.
Runtime-configurable images	One image per app works across environments via injected <code>NEXT_*</code> / <code>DB_*</code> / <code>AWS_*</code> env — no rebuild to change an API URL.
Correct datastore topology	Single shared PostgreSQL 16, single Redis (logical-DB split), optional MinIO for local S3.
Tenant-aware routing	A reverse proxy handles wildcard tenant subdomains and preserves <code>Host</code> → <code>X-Tenant</code> resolution.
Horizontal scalability	Stateless web/SSR tiers scale by replica count; durable state lives only in Postgres / Redis / S3.

2. Current state — audit of what exists

2.1 `docker-compose.yml` (the existing file)

It is **Laravel Sail’s default `laravelnuxt.api` service** (builds from `./vendor/laravel/sail/runtimes/8.3` , image `sail-8.3/app` , bind-mounts the whole repo at `./var/www/html` , runs `php artisan octane:start --watch`) plus a hand-added `pdf-service` / `pdf-worker` / `pdf-redis` trio.

What’s **WRONG** or missing:

#	Problem	Detail	Fix
1	MySQL wiring (headline bug)	The pdf-service is wired to MySQL: <code>DB_HOST=\${MYSQL_DB_HOST:-host.docker.internal}</code> , <code>DB_PORT=\${MYSQL_DB_PORT:-3306}</code> , <code>DB_DATABASE=\${MYSQL_DB_DATABASE:-printing_702}</code> . The platform is PostgreSQL everywhere (CLAUDE.md §6, <code>.env</code> <code>DB_CONNECTION=pgsql</code> on 5432). The <code>MYSQL_DB_*</code> vars don't even exist in <code>.env.example</code> .	Pass <code>DB_DIALECT=postgres</code> , <code>DB_HOST=postgres</code> , <code>DB_PORT=5432</code> , <code>DB_DATABASE=live_db</code> .
2	No Postgres service	<code>laravelnuxt.api</code> has <code>depends_on: {}</code> and the compose declares no database. Both Laravel and pdf-service expect an external Postgres at <code>host.docker.internal</code> .	Add a <code>postgres:16-alpine</code> service.
3	No admin / storefront / designer containers	The compose only covers API + pdf. The Nuxt admin (:3000), storefront (:3001), and designer SPA are absent. The API service even maps <code>\${PORT:-3000}</code> as a leftover but builds nothing for Nuxt.	Add <code>admin</code> , <code>storefront</code> services; bake designer <code>dist</code> into web roots.
4	Dev-mode-only API image	Runs <code>octane:start --watch</code> against a bind-mount of source — a dev convenience, not a prod image. No <code>composer install --no-dev</code> , no <code>nuxt build</code> , no <code>config:cache</code> .	Build a real multi-stage PHP/Octane image.
5	No <code>/api/health</code> route	<code>deploy.sh</code> and any orchestrator probing <code>\$API_URL/api/health</code> will 404 — that route does not exist. (Only <code>routes/store-api.php</code> has an unrelated SEO <code>health</code> .) pdf-service correctly has <code>/health</code> .	Add a real <code>GET /api/health</code> route before wiring a Laravel healthcheck.
6	<code>admin_pgsql</code> port default	<code>config/database.php</code> landlord connection <code>admin_pgsql</code> is <code>driver=pgsql</code> but <code>'port' => env('ADMIN_DB_PORT', '3306')</code> — defaults to the MySQL port.	Set <code>ADMIN_DB_PORT=5432</code> explicitly in every container env.

2.2 pdf-service/Dockerfile (the one good file)

The **only correct, finished Dockerfile** in the repo:

- Multi-stage `node:20-alpine`; build deps `vips-dev` `build-base` `python3` `pkgconfig` (for `sharp` /`canvas`); runtime libs `vips` `tini` `curl`; non-root `app` user; `tini` as PID 1; `HEALTHCHECK` on `/health`.
- Deps: `sharp`, `pdftk`, `bullmq`, `ioredis`, `@aws-sdk/client-s3`, `sequelize` + `pg`.
- One image, two roles: **API** (`WORKER_ENABLED=false`, HTTP :4000) and **worker** (`WORKER_ENABLED=true`, BullMQ consumer, `WORKER_CONCURRENCY`). The compose already splits them — good pattern, keep it.

Reuse as-is. Note `pdf-service/package.json` still lists both `mysql2` and `pg` / `pg-hstore`; the live path is `pg` (harmless leftover).

3. Target architecture — service topology

All services on a single user-defined bridge network (rename the existing `sail` network to `printflow`).

Durable state lives **only** in named volumes on Postgres / Redis / MinIO. The web/SSR tiers are stateless and scale by replica count.

Service	Image / base	Build context	Build steps	Runtime cmd	Port(s)	Depends on	Persistent volume	Healthcheck
postgres	postgres:16-alpine	—	—	postgres	5432 (internal)	—	pgdata:/var/lib/postgresql/data	pg_isready -U <user>
redis	redis:7-alpine (appendonly yes)	—	—	redis-server	6379 (internal)	—	redis-data:/data	redis-cli ping
minio (dev only)	minio/minio	—	—	server /data - - console - address :9001	9000 / 9001	—	minio-data:/data	/minio/health/ready
api (Laravel/Octane)	multi-stage: composer → php8.2 (pdo_pgsql , gd / imagick , zip , pcntl , redis , RoadRunner)	repo root	composer install - -no-dev - -optimize-autoloader ; key:generate (once); storage:link ; migrate --force ; config:cache route:cache view:cache event:cache ; copy designer-dist → public/designer	php artisan octane:start - - host=0.0.0 - - port=8000	8000	postgres, redis, (minio)	storage/ , bootstrap/cache	add GET /api/health then probe it
queue-worker	same image as api	—	—	php artisan queue:work --	—	redis, postgres	—	process up

Service	Image / base	Build context	Build steps	Runtime cmd	Port(s)	Depends on	Persistent volume	Healthcheck
				tries=3 -- timeout=90				
scheduler	same image as api	—	—	php artisan schedule:work	—	redis, postgres	—	—
admin (Nuxt 3)	multi-stage node:22-bookworm-slim → .output	repo root (rootDir: nuxt/)	npm ci && npm run build (NODE_OPTIONS=--max-old-space-size=4096)	node nuxt/.output/server/index.mjs	3000	api	—	TCP 3000 / GET /
storefront (Nuxt 4)	multi-stage node:22-slim; copy designer dist → frontstore/public/designer	frontstore/	npm ci && npm run build:themes && nuxt build	node frontstore/.output/server/index.mjs	3001	api, redis	—	TCP 3001 / GET /
designer	build-stage only (Vite) — NOT a runtime service	designer/	publish-designer.sh (VITE_APP_BASE_API baked at build) → dist copied into api + storefront web roots	— (static)	—	—	—	—
pdf-service	pdf-service/Dockerfile (existing)	pdf-service/	npm ci -omit=dev (multi-stage, vips)	node src/index.js (WORKER_ENABLE=false)	4000	postgres, redis	fonts/, tmp/	/health (built-in)

Service	Image / base	Build context	Build steps	Runtime cmd	Port(s)	Depends on	Persistent volume	Healthcheck
pdf-worker	same image as pdf-service	—	—	<code>node src/index.js (WORKER_ENABLE_D=true, WORKER_CONCURRENCY)</code>	—	redis, pdf-service	<code>fonts/</code> , <code>tmp/</code>	process up
proxy	<code>traefik:v3</code> or <code>nginx</code>	—	—	—	80 / 443	api, admin, storefront	<code>letsencrypt/</code> (Traefik ACME)	dashboard / <code>GET /</code>

Named volumes: `pgdata`, `redis-data`, `minio-data`, plus `pdf-fonts` / `pdf-tmp` (or bind mounts as today).

3.1 Per-app build notes (the easy-to-get-wrong bits)

Admin Nuxt (`nuxt/`)

- Driven by the **repo-root** `package.json` and **repo-root** `nuxt.config.ts` (`rootDir: 'nuxt/'`). The `nuxt/` folder has only a stub `package-lock.json` — **all deps come from the root `package.json`**. The build context must include the repo-root files.
- Output is `nuxt/.output/`, NOT `.output/` (because of `rootDir`). Easy to break the runtime-stage `COPY`.
- Build is memory-heavy (fullcalendar, apexcharts, tiptap, quill, firebase, pdfjs) — set `NODE_OPTIONS=--max-old-space-size=4096` in the builder or it can OOM in constrained CI.
- Runtime stage copies **only** `nuxt/.output/` — Nitro output is self-contained, no `node_modules` at runtime.
- `node:22-bookworm-slim` (per `.nvmrc = 22`); prefer Debian slim over Alpine for `sharp`-adjacent `@nuxt/image` (Alpine needs `libc6-compat`).
- SSR is ON in prod (`$production: { ssr: true }`). **Do not static-export.** `npm run generate` exists but is not how this app deploys.
- `GOOGLE_MAPS_PLACE_API_KEY` and the `meet.printflow360.com` (Jitsi) script are **baked into `<head>` at build** from `import.meta.env` — not runtime-swappable. Bake the prod maps key as a build arg.
- The admin has **no DB / Redis / S3 dependency** — it is a pure SSR frontend proxying everything through Laravel `/api/v1`.

Storefront Nuxt (`frontstore/`)

- Own `package.json`. `npm run build:themes` compiles per-theme CSS (runs automatically via the prebuild hook). Output `frontstore/.output/`.
- **Must set `NITRO_REDIS_URL`** for multi-replica deploys — without it, ISR/SWR cache is in-memory (lost on restart, not shared across replicas).

- Per-tenant cache key varies on `x-tenant-host` + `x-device-type` (`server/plugins/tenant-cache-key.ts`). Private routes (`/cart` , `/checkout/**` , `/profile/**` , `/order/**` , `/login` , `/api/**`) are `no-store` ; public routes are ISR 600s.

Designer SPA (`designer/`)

- **Not a runtime service.** `publish-designer.sh` builds the Vite SPA to `designer/dist` and copies it into **both** `public/designer/` (Laravel) and `frontstore/public/designer/` (served static, embedded via iframe).
- `VITE_APP_BASE_API` is **baked at build** (`DESIGNER_BASE_API` → e.g. `https://api.printflow360.com/api/v1/storefront`). Changing the API URL requires a rebuild. Build in a build stage, copy `dist` into the api + storefront images.

4. Cross-cutting concerns

4.1 SSR vs browser API base (the docker-network linchpin)

The admin's `nuxt/app/plugins/app.ts` → `buildBaseURL()` chooses the base by render context:

```
return baseURL
? baseURL
: import.meta.server
  ? config.apiLocal + config.public.apiPrefix // SSR → API_LOCAL_URL
  : config.public.apiBase + config.public.apiPrefix // browser → APP_URL
```

- **SSR (inside the container)** fetches Laravel via `apiLocal` (`API_LOCAL_URL`). In Docker this **MUST** be the **in-network service hostname**, e.g. `http://laravelnuxt.api:8000` — NOT `localhost:8000` (which is the admin container itself).
- **Browser** calls use `apiBase` (`APP_URL`) — the **publicly reachable** Laravel URL, e.g. `https://api.example.com` .
- Getting only one right yields a “works in browser, white-screen on SSR” (or vice-versa) failure. **Both are required and different.**

4.2 Tenant-host resolution & per-tenant subdomain routing

- Storefront requests carry the store hostname as the `X-Tenant` header (the storefront sets this via `useStorefrontHeaders()`). SSR reads request headers and must **prefer** `x-tenant-host` , **NOT** `useRequestURL().hostname` — behind a proxy/Nitro that returns `localhost` , which breaks tenancy (“Storefront not found”). This is a repeatedly-bitten failure mode in this codebase.
- `config/tenancy.php` `central_domains = ['127.0.0.1', 'localhost']` are the non-tenant (landlord/admin) hosts. Stores live on subdomains (e.g. `myshop.localhost`).
- **A reverse proxy is required (none exists).** Recommend **Traefik** (label-driven, easy wildcard + Host-rule routing, ACME auto-TLS) or **nginx** (`server_name *.domain` , `proxy_set_header Host $host` , manual certs). Routing:
 - `*.<storefront-domain>` (wildcard tenant subdomains) → storefront :3001, **passing the real** `Host` so SSR derives `x-tenant-host` . With Traefik, add a middleware that sets `X-Tenant` / `x-tenant-host`

from `Host` if the client didn't.

- `admin.<domain>` (or central domain) → admin :3000.
- `api.<domain>` → Laravel Octane :8000 (the API also reads `X-Tenant` for `/api/v1/storefront/*`).
- `/designer/**`, `/storage/**`, `/images/**`, `/themes/**` are static / aggressively cached.
- **Local dev:** `*.localhost` resolves to 127.0.0.1 in modern browsers, so `printdesign.localhost:3001` works without `/etc/hosts` edits — but the wildcard proxy still needs to route `myshop.localhost`.
- The admin is the **landlord/back-office app and is NOT host-resolved per tenant** — tenant context flows through API calls / store selection, not the hostname. The storefront's "SSR sees localhost" bug class does not apply to admin. The only admin host concern is `buildHeaders` forwarding `referer` / `x-forwarded-for` / `user-agent` on SSR — ensure the proxy sets `X-Forwarded-For` correctly.

4.3 Cache-purge loopback

`FRONTSTORE_CACHE_PURGE_INTERNAL_URL` must hit the Nuxt process **directly**, NOT the proxy/CDN (Cloudflare returns 403 on server-side purge calls). In compose, set it to `http://storefront:3001`.

4.4 Shared secrets (mismatches = silent 401s)

Secret	Must match across	Failure if mismatched
<code>APP_KEY</code>	Laravel ≡ pdf-service (byte-identical)	Silent 401 on delegated file ops
<code>PDF_SERVICE_INTERNAL_SECRET</code> (Laravel) ≡ <code>INTERNAL_API_SECRET</code> (pdf-service)	Laravel ↔ pdf-service	Silent 401
<code>INTERNAL_API_SECRET</code> (Laravel <code>X-Internal-Secret</code>) + <code>FRONTSTORE_CACHE_PURGE_TOKEN</code>	Laravel ↔ storefront	Cache purge / subscription calls rejected

Secrets hygiene: the committed `.env` / `.env.example` contain **real-looking live secrets** (AWS keys, Stripe keys, Brevo SMTP password, Google Maps key, Zoho refresh token). **Never bake these into image layers.** Mount via Docker secrets / env-file kept out of image layers. **Rotate the leaked ones.** The designer's `VITE_APP_BASE_API` is a build arg (build-time), not a runtime env.

4.5 Driver defaults for containers

Current `.env` is single-box defaults: `CACHE_STORE=database`, `QUEUE_CONNECTION=sync`, `SESSION_DRIVER=file`. `SETUP.md` §10.2 flips all three to `redis` for production. **For containers, switch to `redis`** — and then the **queue-worker and scheduler sidecars become mandatory** (jobs no longer run inline).

4.6 Queue worker + scheduler containers

Run from the **same image as `api`**:

- **queue-worker:** `php artisan queue:work --tries=3 --timeout=90` (PM2 name `printflow-queue`).
- **scheduler:** `php artisan schedule:work` (PM2 name `printflow-schedule`).

4.7 Octane server

`config/octane.php` defaults to **RoadRunner** (`'server' => env('OCTANE_SERVER', 'roadrunner')`). Either install the RoadRunner binary in the image, or set `OCTANE_SERVER=swoole / frankenphp` and install that extension. **FrankenPHP is the cleanest single-binary option for a container.**

4.8 Datastore details

- **Postgres:** one Postgres 16 service, **shared-schema, single database**. Every `stanc1/tenancy` bootstrapper in `config/tenancy.php` is commented out — **no per-tenant database, no schema switching**; isolation is by `tenant_id` column only. Two logical DBs in env: tenant `DB_DATABASE=live_db` and landlord `ADMIN_DB_DATABASE=admin_db` (`admin_pgsql`) — both can live in the same instance.
- **Redis:** one instance with separate logical DBs — Laravel cache/queue/session (db 0/1) + BullMQ (db 2/3). BullMQ technically prefers its own instance for eviction policy (`allkeys-lru maxmemory 256mb`) vs Laravel cache (`noeviction`); for strict isolation use two Redis services, otherwise one with per-prefix keys is fine for small deployments. Also feeds storefront `NITRO_REDIS_URL` .
- **S3 / MinIO:** `FILESYSTEM_DISK=s3` , tenant files as **relative paths** under per-tenant prefixes in **one bucket** (no per-tenant bucket). pdf-service writes to the same S3. For local, `config/filesystems.php` supports `AWS_ENDPOINT` + `AWS_USE_PATH_STYLE_ENDPOINT` → use a **MinIO** container (`AWS_ENDPOINT=http://minio:9000` , path-style true, create the `printflow360` bucket on boot). Real S3 in prod.
- **dompdf fonts:** the Laravel image needs fonts + a writable font-cache dir; the pdf-service mirror needs its `fonts/` volume.

5. Phased migration plan

Phase 0 — Fix the compose & stand up Postgres

The current compose is actively wrong; make it *correct* before adding apps.

- [] Add a `postgres:16-alpine` service with `pgdata` volume + `pg_isready` healthcheck.
- [] Rip out the **MySQL wiring**; point pdf-service at Postgres: `DB_DIALECT=postgres` , `DB_HOST=postgres` , `DB_PORT=5432` , `DB_DATABASE=live_db` .
- [] Set `ADMIN_DB_PORT=5432` explicitly (fixes the `admin_pgsql` `3306` default).
- [] Add a **real GET /api/health route** in Laravel (does not exist today).
- [] Confirm pdf-service + pdf-worker + pdf-redis still come up green against Postgres.
- [] Rename the `sail` network to `printflow` .

Phase 1 — Backend + infra tier

- [] Build a **multi-stage Laravel/Octane image** (composer stage → php-runtime with `pdo_pgsql` , `gd / imagick` , `zip` , `pcntl` , `redis` , RoadRunner or FrankenPHP).
- [] Build steps: `composer install --no-dev` , `key:generate` (once), `storage:link` , `migrate --force` , `config/route/view/event:cache` .
- [] Switch driver defaults: `CACHE_STORE=redis` , `QUEUE_CONNECTION=redis` , `SESSION_DRIVER=redis` .

- [] Add **redis** service (`appendonly yes` , `redis-data` volume, `redis-cli ping` healthcheck).
- [] Add **queue-worker** and **scheduler** sidecars (same image as api).
- [] Add **minio** (dev) with bucket-create on boot; set `AWS_ENDPOINT` + path-style.
- [] Verify shared `APP_KEY` + `PDF_SERVICE_INTERNAL_SECRET` across Laravel ↔ pdf-service.
- [] Probe `GET /api/health` from the orchestrator.

Phase 2 — Frontends

- [] Build the **designer** build-stage (`publish-designer.sh` , `VITE_APP_BASE_API` baked); copy `dist` into the api + storefront web roots.
- [] Build the **admin** image (multi-stage node:22-slim, `rootDir: nuxt/` , `NODE_OPTIONS=--max-old-space-size=4096` ; runtime copies only `nuxt/.output/`).
 - [] Set `NUXT_API_LOCAL=http://laravelnuxt.api:8000` (SSR) and `NUXT_PUBLIC_API_BASE=public` Laravel URL (browser).
 - [] Bake `GOOGLE_MAPS_PLACE_API_KEY` + prod `APP_URL` as build args.
- [] Build the **storefront** image (`npm run build:themes && nuxt build`); copy designer `dist` into `frontstore/public/designer` .
 - [] Set `NITRO_REDIS_URL` for shared ISR cache.
 - [] Set `FRONTSTORE_CACHE_PURGE_INTERNAL_URL=http://storefront:3001` .
- [] Map `admin 3000:3000` , `storefront 3001:3001` ; `depends_on: [api]` (storefront also `redis`).

Phase 3 — Reverse proxy + tenant routing

- [] Add a **Traefik (or nginx)** service on 80/443.
- [] Route `*.<storefront-domain>` → storefront :3001, preserving `Host` ; inject `X-Tenant` / `x-tenant-host` if absent.
- [] Route `admin.<domain>` → admin :3000; `api.<domain>` → api :8000.
- [] Cache `/designer/**` , `/storage/**` , `/themes/**` .
- [] Ensure `X-Forwarded-For` is set for admin SSR `buildHeaders` .
- [] Verify end-to-end: a tenant subdomain renders SSR'd storefront (no “Storefront not found”).

Phase 4 — Prod hardening

- [] Move all secrets to Docker secrets / external env-file (out of image layers); **rotate leaked** `.env` secrets.
- [] Choose Octane server (RoadRunner vs FrankenPHP) and validate under load.
- [] Add ACME/TLS (Traefik) or managed certs (nginx); `letsencrypt/` volume.
- [] Healthchecks on every service; restart policies; replica counts for stateless tiers.
- [] Decide single-Redis vs split-Redis for BullMQ eviction isolation.
- [] Switch MinIO → real S3 in prod env; verify per-tenant relative-path uploads round-trip.
- [] Flip pdf-service feature flags (`PDF_SERVICE_FEAT_*`) only after the service is proven (default OFF = Laravel dompdf path; safe to ship the proxy+web+db tier first and add pdf later).
- [] Commit a CI pipeline that builds/pushes all images with `NUXT_TELEMETRY_DISABLED=1` for hermetic builds.

6. Open questions / decisions needed

#	Question	Notes / lean
1	Octane server: RoadRunner, Swoole, or FrankenPHP?	Default is RoadRunner. FrankenPHP is the cleanest single-binary for a container — recommend evaluating it first.
2	One Redis or two?	One with logical-DB split + per-prefix keys is fine for small deployments. BullMQ prefers <code>allkeys-lru</code> eviction vs Laravel cache <code>noeviction</code> — split only if isolation matters.
3	Storefront domain & TLS model	Wildcard cert for <code>*.<storefront-domain></code> (ACME DNS-01 or a wildcard cert). Decide the apex/subdomain scheme for tenants vs <code>admin.</code> / <code>api.</code>
4	Where do secrets live?	Docker secrets vs external env-file vs a secrets manager. Must rotate the committed real-looking secrets regardless.
5	MinIO in dev only, real S3 in prod — or MinIO in prod too?	Lean: MinIO for local, real AWS S3 in prod. Confirm bucket naming (<code>printf</code> low360) and per-tenant prefix convention.
6	Migrations on boot vs separate job	Running <code>migrate --force</code> in the api build vs a one-shot init container/job. A separate job avoids racing multiple api replicas.
7	Designer rebuild trigger	<code>VITE_APP_BASE_API</code> is build-time. Decide how a prod API-URL change re-triggers the designer build + recopy into both web roots.
8	Keep PM2 bare-metal as the prod target, or migrate fully to containers?	The team's real prod path is PM2 today. Decide whether Docker is for local/CI only, or the new prod runtime.
9	Health route shape	<code>GET /api/health</code> must exist; decide whether it also checks Postgres/Redis reachability (deeper probe) vs a flat 200.

See `DOCKER_RUNBOOK_2026-06-16.md` for the concrete Dockerfiles, compose snippets, build/run commands, and troubleshooting.

Docker Runbook — Print-Flow-360

Source: [readme/research/engineering-ops/DOCKER_RUNBOOK_2026-06-16.md](#)

IN SHORT — KEY TAKEAWAYS

This document is a step-by-step guide for the technical team to package the entire Print-Flow-360 platform into Docker containers — self-contained boxes that make the software easy to deploy and run anywhere. It matters because today the platform runs on a manual bare-metal setup that is harder to reproduce, scale, or hand off; containers fix that. The guide starts by fixing a critical existing mistake before covering every service, every setting, and every known failure.

The main parts explained simply:

- **The broken starting point** — The one Docker file already in the codebase is wired to the wrong database (MySQL on port 3306), but the platform actually uses PostgreSQL on port 5432. Nothing works until this is fixed. The guide lists exactly four things to correct before anything else.
- **Dockerfiles (packaging recipes)** — Each part of the platform needs its own recipe that tells Docker how to build and run it. This document provides new recipes for the Laravel API, the admin dashboard, the customer storefront, and the PDF service. Only the PDF service already has a good recipe; the rest are new.
- **docker-compose.yml (the master on/off switch)** — A single file that starts all the services together in the right order, with the right settings. The existing one is broken; this document replaces it with a corrected version that includes the database, a cache layer (Redis), a local file store (MinIO), and all the app services.
- **Environment variables (the settings sheet)** — Every service needs a list of secret keys and connection addresses. Some of these must be identical across services or things silently fail — for example, the `APP_KEY` must be the exact same value in both the Laravel API and the PDF service, or file operations return a silent error with no useful message.
- **SSR vs. browser API addresses** — The admin dashboard talks to the Laravel API in two different ways: from inside the server (using the internal container name `api:8000`) and from the user's browser (using the public web address). If only one is set correctly, the dashboard either shows a blank page on load or breaks in the browser. Both must be set.
- **Reverse proxy (the traffic director)** — A proxy sits in front of all services and sends each request to the right place. Tenant stores (each store owner's customer-facing site) use their own subdomain, and the proxy must pass the real store address through unchanged or the wrong store loads. The document shows how to set this up with either Traefik or nginx.
- **Rollout order (the safe launch sequence)** — The guide recommends bringing services up in a specific order — database first, then the API, then workers, then the frontends, then the proxy, and the PDF service last — so that each step is low-risk and the old bare-metal setup remains available as a fallback throughout.
- **Troubleshooting table** — A plain list of 17 known failure symptoms (for example: blank admin screen, 401 errors, cache not clearing, jobs never running) with the exact cause and fix for each.

What to do with this:

Fix the four database issues listed in Section 0 first — nothing else works until those are corrected. Then follow the rollout order in Section 7, keeping the existing bare-metal setup running until the container path is fully tested.

Hands-on runbook for containerizing the **whole** Print-Flow-360 stack: concrete Dockerfiles, a corrected `docker-compose.yml`, the env-var matrix, build/run commands, a reverse-proxy sketch, and a troubleshooting table.

Status disclaimer. Today the repo has **no usable container story**. The single committed `docker-compose.yml` is the stock Laravel Sail file with a bolted-on pdf-service stack, and it is **wired to MySQL/3306** even though the platform is **PostgreSQL** everywhere. The only correct, finished Dockerfile in the repo is `pdf-service/Dockerfile`. The real prod path the team uses today is **bare-metal via `deploy/deploy.sh` + PM2**, not Docker. Everything below marked **RECOMMENDED** is greenfield; items marked **EXISTING** already live in the repo.

Audience: developers + ops. Read alongside `CLAUDE.md`, `SETUP.md`, and `readme/PDF_SERVICE.md`.

0. The one bug that blocks everything

The committed compose wires the database clients to **MySQL** (`DB_HOST=${MYSQL_DB_HOST}`, `DB_PORT=3306`, `DB_DATABASE=printing_702`). The platform runs on **PostgreSQL 16**, single shared database, isolation by `tenant_id` column only (all `stanc1/tenancy` bootstrappers are commented out — there is **no per-tenant database**).

Before anything in this runbook works you must:

1. Add a `postgres` service (none exists today).
2. Pass `DB_DIALECT=postgres`, `DB_HOST=postgres`, `DB_PORT=5432`, `DB_DATABASE=live_db` to **pdf-service** (its code defaults to Postgres anyway; the compose env actively overrides it wrong).
3. Set `ADMIN_DB_PORT=5432` explicitly — `config/database.php` `admin_pgsql` defaults its port to `3306` (latent MySQL bug).
4. Add a real `GET /api/health` route (it does not exist — orchestrator probes 404 today).

1. Per-service Dockerfile recommendations

All Dockerfiles below are **RECOMMENDED** (new), except `pdf-service/Dockerfile` which is **EXISTING** and good — reuse it as-is.

1.1 Laravel API (Octane) — `deploy/docker/api.Dockerfile` (RECOMMENDED)

Multi-stage: composer deps → PHP runtime with the extensions listed in `SETUP.md §2` plus `pdo_pgsql`, `gd / imagick`, `zip`, `pcntl / posix`, `redis`. Octane defaults to **RoadRunner** (`config/octane.php` → `env('OCTANE_SERVER', 'roadrunner')`); the cleanest single-binary container option is **FrankenPHP**, shown here. (Alternative: install the RoadRunner binary, or set `OCTANE_SERVER=swoole` + the swoole extension.)

The designer SPA (static, **build-time API URL baked**) is built in its own stage and copied into `public/designer/`.

```
# ---- designer build stage (static SPA, baked API URL) ----
FROM node:22-bookworm-slim AS designer
WORKDIR /designer
COPY designer/package*.json ./
RUN npm ci
COPY designer/ ./
# VITE_APP_BASE_API is BAKED at build time — pass the prod storefront API URL.
ARG DESIGNER_BASE_API=https://api.printflow360.com/api/v1/storefront
ENV VITE_APP_BASE_API=$DESIGNER_BASE_API
RUN npm run build          # -> designer/dist

# ---- composer / PHP deps stage ----
FROM composer:2 AS vendor
WORKDIR /app
COPY composer.json composer.lock ./
RUN composer install --no-dev --optimize-autoloader --no-scripts --ignore-platform-reqs

# ---- runtime: FrankenPHP (PHP 8.2+, Octane) ----
FROM dunglas/frankenphp:php8.3-bookworm AS runtime
# Required PHP extensions (SETUP.md §2 + Postgres + image + zip + queue + redis)
RUN install-php-extensions \
    pdo_pgsql pgsql bcmath gd zip exif pcntl intl opcache redis
WORKDIR /var/www/html

# App source + vendored deps + designer dist
COPY . .
COPY --from=vendor /app/vendor ./vendor
COPY --from=designer /designer/dist ./public/designer

# dompdf needs a writable font cache; storage/bootstrap must be writable
RUN chown -R www-data:www-data storage bootstrap/cache public/designer \
    && mkdir -p storage/fonts && chown -R www-data:www-data storage/fonts

ENV OCTANE_SERVER=frankenphp
EXPOSE 8000
# Prod: NO --watch (that's the Sail dev convenience). Caches built at deploy time (see §4).
CMD ["php", "artisan", "octane:start", "--host=0.0.0.0", "--port=8000"]
```

Gotchas

- `--watch` is dev-only. Production uses the plain `octane:start`.
- Run `migrate --force + config:cache route:cache view:cache event:cache` at **deploy time** (an entrypoint or a one-shot job), not in the image build — they need live env/DB. See §4.
- **dompdf needs fonts** and a writable cache dir in the image (`storage/fonts` above).
- **Never bake `.env`** — the committed `.env` / `.env.example` contain real-looking secrets. Mount env at runtime.

1.2 Queue worker + scheduler (REUSE API image, different command) (RECOMMENDED)

No separate Dockerfile. Same image as the API, override the command. Required because the move to containers means flipping `QUEUE_CONNECTION=sync` → `redis` (jobs no longer run inline).


```
# queue-worker (deploy/deploy.sh PM2 name: printflow-queue)
CMD ["php", "artisan", "queue:work", "--tries=3", "--timeout=90"]

# scheduler (deploy/deploy.sh PM2 name: printflow-schedule)
CMD ["php", "artisan", "schedule:work"]
```

(Both set as `command:` overrides in compose — see §2.)

1.3 Nuxt 3 Admin (`nuxt/` , :3000) — `deploy/docker/admin.Dockerfile` (RECOMMENDED)

Multi-stage node build → slim runtime serving the self-contained Nitro output.

Critical gotchas for this app

- Deps come from the **ROOT** `package.json` + root `nuxt.config.ts` (`rootDir: 'nuxt/'`). The `nuxt/package-lock.json` is a stub — build context must include repo-root files.
- Output is `nuxt/.output/` , not `.output/` (because of `rootDir`).
- SSR uses `API_LOCAL_URL` (internal), browser uses `APP_URL` (public). Both required, different in Docker.
- `GOOGLE_MAPS_PLACE_API_KEY` + the `meet.printflow360.com` Jitsi script are **baked at build** from `import.meta.env` — not runtime-swappable. `APP_URL` is also baked into `image.domains` + CSP.
- Build is memory-heavy (fullcalendar/apexcharts/tiptap/quill/firebase/pdfjs) — set `--max-old-space-size=4096` .

```
# ---- build stage ----
FROM node:22-bookworm-slim AS build
WORKDIR /app
# Root files drive the build (rootDir: 'nuxt/').
COPY package.json package-lock.json nuxt.config.ts tsconfig.json ./
RUN npm ci # postinstall runs `nuxt prepare`
COPY nuxt/ ./nuxt/
# Build-time bakes: APP_URL (CSP/image domains), maps key, env label, timezone.
ARG APP_URL=https://api.printflow360.com
ARG GOOGLE_MAPS_PLACE_API_KEY=
ARG ENVIRONMENT=PRODUCTION
ARG APP_TIMEZONE=Asia/Kolkata
ENV APP_URL=$APP_URL \
    GOOGLE_MAPS_PLACE_API_KEY=$GOOGLE_MAPS_PLACE_API_KEY \
    ENVIRONMENT=$ENVIRONMENT \
    APP_TIMEZONE=$APP_TIMEZONE \
    NUXT_TELEMETRY_DISABLED=1 \
    NODE_OPTIONS=--max-old-space-size=4096
RUN npm run build # -> nuxt/.output/ (NOTE: under nuxt/)

# ---- runtime stage ----
FROM node:22-bookworm-slim AS runtime
WORKDIR /app
RUN useradd -m -u 10001 app
# Nitro output is self-contained — no node_modules needed at runtime.
COPY --from=build /app/nuxt/.output ./nuxt/.output
USER app
ENV PORT=3000 HOST=0.0.0.0
EXPOSE 3000
# Runtime-overridable (set per env): NUXT_API_LOCAL, NUXT_PUBLIC_API_BASE, etc. (see §3)
CMD ["node", "nuxt/.output/server/index.mjs"]
```

1.4 Nuxt 4 Storefront (`frontstore/` , :3001) — `deploy/docker/storefront.Dockerfile` (RECOMMENDED)

Own `package.json`. The `build:themes` step compiles per-theme CSS and runs **automatically via the prebuild hook**, so `nuxt build` triggers it — but run it explicitly to be safe. Copies the designer `dist` into `frontstore/public/designer` (static, iframe-embedded).

Critical gotchas

- Set `NITRO_REDIS_URL` at runtime for shared/persistent ISR cache — without it every restart cold-starts and replicas don't share cache.
- Per-tenant cache key varies on `x-tenant-host` + `x-device-type` (`server/plugins/tenant-cache-key.ts`). The proxy **must preserve the real Host** .
- `FRONTSTORE_CACHE_PURGE_INTERNAL_URL` must hit the Nuxt process **directly** (`http://storefront:3001`), never via the proxy/Cloudflare (returns 403).

```
# ---- designer build stage (reuse same as API) ----
FROM node:22-bookworm-slim AS designer
WORKDIR /designer
COPY designer/package*.json ./
RUN npm ci
COPY designer/ ./
ARG DESIGNER_BASE_API=https://api.printflow360.com/api/v1/storefront
ENV VITE_APP_BASE_API=$DESIGNER_BASE_API
RUN npm run build

# ---- storefront build stage ----
FROM node:22-bookworm-slim AS build
WORKDIR /app
COPY frontstore/package*.json ./frontstore/
WORKDIR /app/frontstore
RUN npm ci
COPY frontstore/ ./
ENV NUXT_TELEMETRY_DISABLED=1 NODE_OPTIONS=--max-old-space-size=4096
RUN npm run build:themes && npx nuxt build # prebuild hook also runs build:themes
# Designer SPA served as static from frontstore web root
COPY --from=designer /designer/dist ./public/designer

# ---- runtime stage ----
FROM node:22-bookworm-slim AS runtime
WORKDIR /app
RUN useradd -m -u 10002 app
COPY --from=build /app/frontstore/.output ./frontstore/.output
USER app
ENV PORT=3001 HOST=0.0.0.0
EXPOSE 3001
CMD ["node", "frontstore/.output/server/index.mjs"]
```

1.5 Designer SPA (`designer/` , build-only) (RECOMMENDED, as a stage)

Not a runtime service. It is a Vite SPA whose `VITE_APP_BASE_API` is **baked at build time** (`DESIGNER_BASE_API` env). In prod it is built once and the `dist/` is copied into both the Laravel `public/designer/` and the storefront `frontstore/public/designer/` web roots (mirrors `scripts/publish-designer.sh`). It is shown above as the `designer` build stage inside both the API and storefront Dockerfiles — there is no standalone designer container.

Do **not** run a designer build casually ([CLAUDE.md §5](#) — a hook even blocks it). Only build it in CI/image build for a release.

1.6 pdf-service (`pdf-service/` , :4000) — `pdf-service/Dockerfile` (EXISTING — reuse)

The **only correct, finished Dockerfile in the repo**. Multi-stage `node:20-alpine` , build deps `vips-dev` `build-base` `python3` `pkgconf` (for `sharp` /`canvas`), runtime libs `vips` `tini` `curl` , non-root `app` user, `tini` as PID 1, `HEALTHCHECK` on `/health` . **Do not rewrite it.**

One image serves two roles via env:

- **API:** `WORKER_ENABLED=false` , HTTP :4000.
- **Worker:** `WORKER_ENABLED=true` , BullMQ consumer, `WORKER_CONCURRENCY` .

The only **fix needed** is the *compose env* (not the Dockerfile): pass `DB_DIALECT=postgres` , `DB_HOST=postgres` , `DB_PORT=5432` , `DB_DATABASE=live_db` (see §0 and §2).

2. Corrected, complete `docker-compose.yml` (dev) (RECOMMENDED)

Replaces the broken Sail file. Fixes the MySQL → PostgreSQL bug, adds postgres/redis/minio/proxy, splits pdf API vs worker, adds queue-worker + scheduler sidecars, uses YAML anchors for shared env, and adds healthchecks + named volumes.

```

# docker-compose.yml – dev stack for Print-Flow-360 (RECOMMENDED, replaces Sail stub)
x-laravel-env: &laravel-env
  APP_KEY: ${APP_KEY} # MUST match pdf-service APP_KEY exactly
  DB_CONNECTION: pgsql
  DB_HOST: postgres
  DB_PORT: 5432
  DB_DATABASE: ${DB_DATABASE:-live_db}
  DB_USERNAME: ${DB_USERNAME:-printflow}
  DB_PASSWORD: ${DB_PASSWORD:-secret}
  ADMIN_DB_CONNECTION: admin_pgsql
  ADMIN_DB_HOST: postgres
  ADMIN_DB_PORT: 5432 # FIX: config default is wrongly 3306
  ADMIN_DB_DATABASE: ${ADMIN_DB_DATABASE:-admin_db}
  REDIS_HOST: redis
  REDIS_PORT: 6379
  REDIS_CLIENT: phpredis
  CACHE_STORE: redis # was: database
  QUEUE_CONNECTION: redis # was: sync (so workers become mandatory)
  SESSION_DRIVER: redis # was: file
  FILESYSTEM_DISK: s3
  AWS_ACCESS_KEY_ID: ${AWS_ACCESS_KEY_ID:-printflow}
  AWS_SECRET_ACCESS_KEY: ${AWS_SECRET_ACCESS_KEY:-printflow-secret}
  AWS_DEFAULT_REGION: ${AWS_DEFAULT_REGION:-us-east-1}
  AWS_BUCKET: ${AWS_BUCKET:-printflow360}
  AWS_ENDPOINT: http://minio:9000 # MinIO for local S3
  AWS_USE_PATH_STYLE_ENDPOINT: "true"
  PDF_SERVICE_INTERNAL_SECRET: ${PDF_SERVICE_INTERNAL_SECRET} # == pdf INTERNAL_API_SECRET

services:
  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: ${DB_USERNAME:-printflow}
      POSTGRES_PASSWORD: ${DB_PASSWORD:-secret}
      POSTGRES_DB: ${DB_DATABASE:-live_db}
    volumes:
      - pgdata:/var/lib/postgresql/data
      # one-shot: create the landlord DB too (admin_db) – both live in one instance
      - ./deploy/docker/initdb:/docker-entrypoint-initdb.d:ro
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${DB_USERNAME:-printflow}"]
      interval: 10s
      timeout: 5s
      retries: 5
    networks: [printflow]

  redis:
    image: redis:7-alpine
    command: ["redis-server", "--appendonly", "yes"]
    volumes:
      - redis-data:/data
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 10s
      timeout: 3s
      retries: 5
    networks: [printflow]

  minio:
    # dev only – real S3 in prod
    image: minio/minio
    command: server /data --console-address ":9001"
    environment:

```

```

    MINIO_ROOT_USER: ${AWS_ACCESS_KEY_ID:-printfLOW}
    MINIO_ROOT_PASSWORD: ${AWS_SECRET_ACCESS_KEY:-printfLOW-secret}
volumes:
  - minio-data:/data
ports: ["9000:9000", "9001:9001"]
healthcheck:
  test: ["CMD", "curl", "-f", "http://localhost:9000/minio/health/ready"]
  interval: 15s
  timeout: 5s
  retries: 5
networks: [printfLOW]

api:
  build:
    context: .
    dockerfile: deploy/docker/api.Dockerfile
  environment:
    <<: *laravel-env
    APP_URL: ${APP_URL:-http://api.localHOST}
    OCTANE_SERVER: frankenphp
  depends_on:
    postgres: { condition: service_healthy }
    redis: { condition: service_healthy }
    minio: { condition: service_healthy }
  volumes:
    - api-storage:/var/www/html/storage # logs (durable local state when S3 is on)
  healthcheck:
    test: ["CMD", "curl", "-f", "http://localhost:8000/api/health"] # ADD this route
    interval: 15s
    timeout: 5s
    retries: 5
  networks: [printfLOW]

queue-worker:
  build: { context: ., dockerfile: deploy/docker/api.Dockerfile }
  command: ["php", "artisan", "queue:work", "--tries=3", "--timeout=90"]
  environment:
    <<: *laravel-env
    APP_URL: ${APP_URL:-http://api.localHOST}
  depends_on:
    postgres: { condition: service_healthy }
    redis: { condition: service_healthy }
  networks: [printfLOW]

scheduler:
  build: { context: ., dockerfile: deploy/docker/api.Dockerfile }
  command: ["php", "artisan", "schedule:work"]
  environment:
    <<: *laravel-env
    APP_URL: ${APP_URL:-http://api.localHOST}
  depends_on:
    postgres: { condition: service_healthy }
    redis: { condition: service_healthy }
  networks: [printfLOW]

admin:
  build:
    context: .
    dockerfile: deploy/docker/admin.Dockerfile
  args:
    APP_URL: ${APP_URL:-http://api.localHOST} # baked (CSP/image domains)
    GOOGLE_MAPS_PLACE_API_KEY: ${GOOGLE_MAPS_PLACE_API_KEY:-}
    ENVIRONMENT: ${ENVIRONMENT:-DEV}

```

```

    APP_TIMEZONE: ${APP_TIMEZONE:-Asia/Kolkata}
environment:
    NUXT_API_LOCAL: http://api:8000 # SSR / internal
    NUXT_PUBLIC_API_BASE: ${APP_URL:-http://api.localhost} # browser / public
    NUXT_PUBLIC_WS_URL: ${NUXT_PUBLIC_WS_URL:-ws://localhost:3001}
    NUXT_PUBLIC_ENVIRONMENT: ${ENVIRONMENT:-DEV}
    NUXT_PUBLIC_STORAGE_BASE: ${NUXT_PUBLIC_STORAGE_BASE:-}
    NUXT_PUBLIC_DESIGNER_APP_URL: /designer/
depends_on:
    api: { condition: service_healthy }
healthcheck:
    test: ["CMD-SHELL", "node -e \"fetch('http://localhost:3000/').then(r=>process.exit(r.ok?
0:1)).catch(()=>process.exit(1))\""]
    interval: 20s
    timeout: 5s
    retries: 5
networks: [printflow]

storefront:
    build:
        context: .
        dockerfile: deploy/docker/storefront.Dockerfile
    environment:
        FRONT_STORE_APP_BASE_API_URL: http://api:8000 # SSR/internal Laravel
        FRONT_STORE_APP_API_PREFIX: /api/v1/storefront
        FRONT_STORE_APP_STORAGE_BASE: ${NUXT_PUBLIC_STORAGE_BASE:-}
        DESIGNER_APP: /designer/
        NITRO_REDIS_URL: redis://redis:6379/4 # shared ISR cache across replicas
        FRONTSTORE_CACHE_PURGE_TOKEN: ${FRONTSTORE_CACHE_PURGE_TOKEN}
        FRONTSTORE_CACHE_PURGE_INTERNAL_URL: http://storefront:3001 # bypass proxy/CDN
        BASE_DOMAIN: ${BASE_DOMAIN:-localhost}
    depends_on:
        api: { condition: service_healthy }
        redis: { condition: service_healthy }
    healthcheck:
        test: ["CMD-SHELL", "node -e \"fetch('http://localhost:3001/').then(r=>process.exit(r.ok?
0:1)).catch(()=>process.exit(1))\""]
        interval: 20s
        timeout: 5s
        retries: 5
    networks: [printflow]

pdf-service:
    build: { context: ./pdf-service, dockerfile: Dockerfile } # EXISTING Dockerfile
    environment:
        WORKER_ENABLED: "false"
        DB_DIALECT: postgres # FIX: compose previously passed MySQL
        DB_HOST: postgres
        DB_PORT: 5432
        DB_DATABASE: ${DB_DATABASE:-live_db}
        DB_USERNAME: ${DB_USERNAME:-printflow}
        DB_PASSWORD: ${DB_PASSWORD:-secret}
        APP_KEY: ${APP_KEY} # == Laravel APP_KEY
        INTERNAL_API_SECRET: ${PDF_SERVICE_INTERNAL_SECRET} # == Laravel PDF_SERVICE_INTERNAL_SECRET
        REDIS_HOST: redis
        REDIS_PORT: 6379
        AWS_ENDPOINT: http://minio:9000
        AWS_ACCESS_KEY_ID: ${AWS_ACCESS_KEY_ID:-printflow}
        AWS_SECRET_ACCESS_KEY: ${AWS_SECRET_ACCESS_KEY:-printflow-secret}
        AWS_BUCKET: ${AWS_BUCKET:-printflow360}
        STORAGE_BASE_URL: ${STORAGE_BASE_URL:-http://minio:9000/printflow360}
    depends_on:
        postgres: { condition: service_healthy }

```

```

    redis:    { condition: service_healthy }
  volumes:
    - ./pdf-service/fonts:/app/fonts
    - pdf-tmp:/app/tmp
  networks: [printflow]          # /health is built into the existing Dockerfile

pdf-worker:
  build: { context: ./pdf-service, dockerfile: Dockerfile }
  environment:
    WORKER_ENABLED: "true"
    WORKER_CONCURRENCY: ${WORKER_CONCURRENCY:-2}
    DB_DIALECT: postgres
    DB_HOST: postgres
    DB_PORT: 5432
    DB_DATABASE: ${DB_DATABASE:-live_db}
    DB_USERNAME: ${DB_USERNAME:-printflow}
    DB_PASSWORD: ${DB_PASSWORD:-secret}
    APP_KEY: ${APP_KEY}
    INTERNAL_API_SECRET: ${PDF_SERVICE_INTERNAL_SECRET}
    REDIS_HOST: redis
    REDIS_PORT: 6379
    AWS_ENDPOINT: http://minio:9000
    AWS_ACCESS_KEY_ID: ${AWS_ACCESS_KEY_ID:-printflow}
    AWS_SECRET_ACCESS_KEY: ${AWS_SECRET_ACCESS_KEY:-printflow-secret}
    AWS_BUCKET: ${AWS_BUCKET:-printflow360}
    STORAGE_BASE_URL: ${STORAGE_BASE_URL:-http://minio:9000/printflow360}
  depends_on:
    redis:    { condition: service_healthy }
    pdf-service: { condition: service_started }
  volumes:
    - ./pdf-service/fonts:/app/fonts
    - pdf-tmp:/app/tmp
  networks: [printflow]

proxy:
  image: traefik:v3
  command:
    - --providers.docker=true
    - --providers.docker.exposedbydefault=false
    - --entrypoints.web.address=:80
  ports: ["80:80", "8080:8080"]
  volumes:
    - /var/run/docker.sock:/var/run/docker.sock:ro
  depends_on: [api, admin, storefront]
  networks: [printflow]

volumes:
  pgdata:
  redis-data:
  minio-data:
  api-storage:
  pdf-tmp:

networks:
  printflow:
    driver: bridge

```

The Traefik routing labels (Host rules + X-Tenant header injection) are sketched in §5; add them onto the `api` / `admin` / `storefront` services.

3. Env-var matrix — who needs what, and what MUST match

3.1 Secrets that MUST be byte-identical across services

Var	Laravel	pdf-service	storefront	Failure if mismatched
APP_KEY	APP_KEY	APP_KEY	—	silent 401 on every delegated file op
PDF_SERVICE_INTERNAL_SECRET ↔ INTERNAL_API_SECRET	PDF_SERVICE_INTERNAL_SECRET	INTERNAL_API_SECRET	—	401 between Laravel ↔ pdf-service
FRONTSTORE_CACHE_PURGE_TOKEN	set	—	FRONTSTORE_CACHE_PURGE_TOKEN	cache purge rejected
INTERNAL_API_SECRET (subscription X-Internal-Secret)	set	—	set (if used)	subscription verify fails

3.2 Per-service env

Service	Key env it reads	Notes
api	<code>APP_KEY</code> , <code>APP_URL</code> , <code>DB_*</code> (pgsql/5432), <code>ADMIN_DB_*</code> (5432, not 3306), <code>REDIS_*</code> , <code>CACHE_STORE=redis</code> , <code>QUEUE_CONNECTION=redis</code> , <code>SESSION_DRIVER=redis</code> , <code>FILESYSTEM_DISK=s3</code> (+ <code>AWS_ENDPOINT</code> / <code>AWS_USE_PATH_STYLE_ENDPOINT</code> for MinIO), <code>OCTANE_SERVER</code> , <code>PDF_SERVICE_*</code>	switch sync/database/file → redis for containers
queue-worker / scheduler	same as api	different command only
admin (Nuxt 3)	build-time <code>APP_URL</code> , <code>GOOGLE_MAPS_PLACE_API_KEY</code> , <code>ENVIRONMENT</code> , <code>APP_TIMEZONE</code> ; runtime <code>NUXT_API_LOCAL</code> (SSR/internal → <code>http://api:8000</code>), <code>NUXT_PUBLIC_API_BASE</code> (browser/public → public Laravel URL), <code>NUXT_PUBLIC_WS_URL</code> , <code>NUXT_PUBLIC_STORAGE_BASE</code> , <code>NUXT_PUBLIC_DESIGNER_APP_URL</code> , <code>NUXT_PUBLIC_ENVIRONMENT</code> , <code>NUXT_PUBLIC_TIMEZONE</code>	API prefix fixed at <code>/api/v1</code> . Maps key + Jitsi script baked at build
storefront (Nuxt 4)	<code>FRONT_STORE_APP_BASE_API_URL</code> (internal → <code>http://api:8000</code>), <code>FRONT_STORE_APP_API_PREFIX</code> , <code>FRONT_STORE_APP_STORAGE_BASE</code> , <code>DESIGNER_APP</code> , <code>NITRO_REDIS_URL</code> , <code>FRONTSTORE_CACHE_PURGE_TOKEN</code> , <code>FRONTSTORE_CACHE_PURGE_INTERNAL_URL</code> , <code>BASE_DOMAIN</code> / <code>FRONTSTORE_BASE_URL_WITHOUT_PROTOCOL</code>	<code>NITRO_REDIS_URL</code> mandatory for multi-replica; purge URL must bypass proxy
designer	<code>DESIGNER_BASE_API</code> / <code>VITE_APP_BASE_API</code> (build arg only)	rebuild to change the API URL
pdf-service / pdf-worker	<code>DB_DIALECT=postgres</code> , <code>DB_HOST=postgres</code> , <code>DB_PORT=5432</code> , <code>DB_DATABASE=live_db</code> , <code>APP_KEY</code> , <code>INTERNAL_API_SECRET</code> , <code>REDIS_*</code> , <code>AWS_*</code> , <code>STORAGE_BASE_URL</code> , <code>WORKER_ENABLED</code> , <code>WORKER_CONCURRENCY</code> , <code>SENTRY_DSN</code>	feature flags <code>PDF_SERVICE_FEAT_*</code> default OFF (Laravel keeps using dompdf)

3.3 SSR-vs-browser API base (the docker-network linchpin)

`nuxt/app/plugins/app.ts` → `buildBaseURL()` chooses:

```
import.meta.server
  ? config.apiLocal + config.public.apiPrefix // SSR → API_LOCAL_URL (NUXT_API_LOCAL)
  : config.public.apiBase + config.public.apiPrefix // browser → APP_URL (NUXT_PUBLIC_API_BASE)
```

- **SSR (inside the admin container)** → `NUXT_API_LOCAL` must be the in-network service name (`http://api:8000`). `localhost` here is the admin container itself → white-screen on SSR.
- **Browser** → `NUXT_PUBLIC_API_BASE` must be the publicly reachable Laravel URL.
- Getting only one right yields “works in browser, blank on SSR” (or vice-versa).

3.4 Secrets hygiene

The committed `.env` / `.env.example` contain **real-looking live secrets** (AWS keys, Stripe keys, Brevo SMTP password, Google Maps key, Zoho refresh token). **Never bake these into image layers.** Use Docker secrets / an env-file kept out of the image, and rotate the leaked ones.

4. Build & run commands

All commands run from repo root. `APP_KEY`, `PDF_SERVICE_INTERNAL_SECRET`, etc. should live in an out-of-image `.env` consumed by compose, not in the images.

```
# 0. One-time: generate an APP_KEY if you don't have one (reuse it for pdf-service)
docker compose run --rm api php artisan key:generate --show # copy into your .env

# 1. Build all images
docker compose build

# 2. Bring up datastores first, then the rest
docker compose up -d postgres redis minio
docker compose up -d

# 3. Create the MinIO bucket (dev S3) — one-shot
docker compose exec minio sh -c \
  "mc alias set local http://localhost:9000 $AWS_ACCESS_KEY_ID $AWS_SECRET_ACCESS_KEY && \
  mc mb -p local/printflow360"

# 4. Laravel deploy-time steps (run AFTER postgres is healthy)
docker compose exec api php artisan migrate --force
docker compose exec api php artisan storage:link
docker compose exec api php artisan config:cache route:cache view:cache event:cache

# 5. Storefront per-theme CSS is built during the image build (build:themes via prebuild
#   hook). To rebuild after a theme change:
docker compose build storefront

# 6. Designer is built INTO the api + storefront images (static dist, baked API URL).
#   Mirrors scripts/publish-designer.sh. To change its API URL, rebuild with the build arg:
docker compose build --build-arg DESIGNER_BASE_API=https://api.example.com/api/v1/storefront \
  api storefront

# 7. Tail logs
docker compose logs -f api
docker compose logs -f storefront admin
docker compose logs -f pdf-service pdf-worker

# 8. Check pdf-service wiring + feature flags (Artisan helper, EXISTING)
docker compose exec api php artisan pdf-service:status
```

Memory note for the builders: the admin and storefront builds are memory-heavy — `NODE_OPTIONS=- -max-old-space-size=4096` is set inside their Dockerfiles. Laravel's bare-metal build uses the same flag in `deploy/deploy.sh`.

5. Reverse-proxy sketch — tenant subdomains + X-Tenant (RECOMMENDED)

No proxy exists in the repo. Tenant resolution: **storefront requests carry the store hostname as the X-Tenant header** (the storefront Nuxt sets it via `useStorefrontHeaders()` ; SSR reads request headers and prefers `x-tenant-host` , **not** `useRequestURL().hostname` which returns `localhost` behind a proxy and breaks tenancy). `config/tenancy.php` `central_domains = ['127.0.0.1', 'localhost']` are the non-tenant (admin/landlord) hosts.

Routing rules:

Host pattern	→ service	Must do
<code>*.<storefront-domain></code> (wildcard tenant subdomains)	storefront :3001	pass real <code>Host</code> through; inject <code>X-Tenant / x-tenant-host</code> from <code>Host</code> if absent
<code>admin.<domain></code> (or central domain)	admin :3000	normal proxy
<code>api.<domain></code>	Laravel Octane :8000	API also reads <code>X-Tenant</code> for <code>/api/v1/storefront/*</code>
<code>/designer/**</code> , <code>/storage/**</code> , <code>/images/**</code> , <code>/themes/**</code>	static / cached	aggressive cache (storefront route rules already cache <code>/designer/**</code> + <code>/themes/**</code>)

Traefik labels (add onto compose services)

```
storefront:
  labels:
    - traefik.enable=true
    - "traefik.http.routers.store.rule=HostRegexp(`{sub:[a-z0-9-]+}.${STOREFRONT_DOMAIN}`)"
    - traefik.http.routers.store.entrypoints=web
    - traefik.http.services.store.loadbalancer.server.port=3001
    # Set X-Tenant from the Host if the client didn't send it:
    - "traefik.http.middlewares.xtenant.headers.customrequestheaders.X-Tenant=${HOST_PLACEHOLDER}"
    - traefik.http.routers.store.middlewares=xtenant
admin:
  labels:
    - traefik.enable=true
    - "traefik.http.routers.admin.rule=Host(`admin.${BASE_DOMAIN}`)"
    - traefik.http.services.admin.loadbalancer.server.port=3000
api:
  labels:
    - traefik.enable=true
    - "traefik.http.routers.api.rule=Host(`api.${BASE_DOMAIN}`)"
    - traefik.http.services.api.loadbalancer.server.port=8000
```

Traefik can't templativize a header from `Host` purely via labels — for production-grade `X-Tenant=Host` injection use a small `ForwardAuth`/plugin, or do it in `nginx` (below). The storefront Nuxt already derives the tenant from the forwarded `Host / x-tenant-host` , so the key requirement is **preserve the real Host** .

nginx equivalent (explicit header injection)

```
# wildcard tenant subdomains -> storefront, inject X-Tenant from Host
server {
    server_name ~^(?<tenant>.+)\.example\.com$;
    location / {
        proxy_pass http://storefront:3001;
        proxy_set_header Host $host;           # preserve real Host
        proxy_set_header X-Tenant $host;       # tenant resolution
        proxy_set_header x-tenant-host $host;  # SSR prefers this
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}

server {                                # admin
    server_name admin.example.com;
    location / { proxy_pass http://admin:3000; proxy_set_header Host $host; }
}

server {                                # api
    server_name api.example.com;
    location / {
        proxy_pass http://api:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
```

Local dev: `*.localhost` resolves to `127.0.0.1` automatically in modern browsers, so `printdesign.localhost` works without `/etc/hosts` edits — but the wildcard proxy still needs the Host rule. `.env` : `TEST_STORE_SUBDOMAIN=printdesign.localhost:3001` , `BASE_DOMAIN=localhost:3001` .

Cache-purge loopback: `FRONTSTORE_CACHE_PURGE_INTERNAL_URL=http://storefront:3001` must hit Nuxt **directly**, never the proxy/Cloudflare (returns 403).

6. Troubleshooting

Symptom	Cause	Fix
Silent 401 on every delegated PDF/image op	<code>APP_KEY</code> differs between Laravel and pdf-service	Make <code>APP_KEY</code> byte-identical in both; restart both
401 between Laravel ↔ pdf-service (non- <code>APP_KEY</code>)	<code>PDF_SERVICE_INTERNAL_SECRET</code> (Laravel) ≠ <code>INTERNAL_API_SECRET</code> (pdf-service)	Set both to the same value
pdf-service can't reach the DB / connects to nonexistent <code>printing_702</code> or speaks Postgres to port 3306	Compose still passes <code>MYSQL_DB_*</code> / <code>DB_PORT=3306</code>	Pass <code>DB_DIALECT=postgres</code> , <code>DB_HOST=postgres</code> , <code>DB_PORT=5432</code> , <code>DB_DATABASE=live_db</code>
Landlord (<code>admin_pgsql</code>) connection fails on 3306	<code>config/database.php</code> <code>admin_pgsql</code> port default is <code>3306</code>	Set <code>ADMIN_DB_PORT=5432</code> explicitly in env
Storefront shows "Storefront not found" / tenant resolves as <code>localhost</code>	SSR fell back to <code>useRequestURL().hostname</code> (returns <code>localhost</code> behind proxy/Nitro)	Proxy must forward real <code>Host</code> ; inject <code>X-Tenant / x-tenant-host</code> ; ensure SSR reads <code>x-tenant-host</code>
Admin white-screen on SSR but fine in browser (or vice-versa)	Only one of <code>NUXT_API_LOCAL</code> / <code>NUXT_PUBLIC_API_BASE</code> is correct	SSR → <code>NUXT_API_LOCAL=http://api:8000</code> ; browser → <code>NUXT_PUBLIC_API_BASE=</code> public Laravel URL
Orchestrator / <code>deploy.sh</code> health probe 404s on <code>/api/health</code>	No real health route exists	Add <code>GET /api/health</code> returning 200 before wiring the Laravel healthcheck
Storefront cache cold on every restart / replicas show stale-vs-fresh inconsistently	<code>NITRO_REDIS_URL</code> unset → in-memory ISR cache (lost on restart, not shared)	Set <code>NITRO_REDIS_URL=redis://redis:6379/4</code>
Cache purge fails / 403	Purge call routed through proxy/Cloudflare	Set <code>FRONTSTORE_CACHE_PURGE_INTERNAL_URL=http://storefront:3001</code> (direct)
Admin build OOMs in CI	Huge dep tree (fullcalendar/apexcharts/tiptap/quill/firebase/pdfjs)	<code>NODE_OPTIONS=--max-old-space-size=4096</code> in builder stage
<code>COPY</code> finds no .output for the admin image	Output is under <code>nuxt/.output/</code> (because <code>rootDir: 'nuxt/'</code>), not <code>../.output/</code>	Copy <code>nuxt/.output</code> ; ensure build context has root <code>package.json</code> + <code>nuxt.config.ts</code>
Admin missing deps at build	Deps live in root <code>package.json</code> ; <code>nuxt/package-lock.json</code> is a stub	Build context must include repo-root files; <code>npm ci</code> at root

Symptom	Cause	Fix
Designer iframe calls wrong API	<code>VITE_APP_BASE_API</code> is baked at build time	Rebuild with <code>--build-arg DESIGNER_BASE_API=...</code>
Queued jobs never run in containers	<code>QUEUE_CONNECTION=sync</code> (inline) carried over from <code>.env</code>	Set <code>QUEUE_CONNECTION=redis</code> and run the <code>queue-worker</code> + <code>scheduler</code> sidecars
dompdf export fails / missing fonts	Font cache dir not present/writable in the Laravel image	Ship fonts + a writable <code>storage/fonts</code> cache dir
Octane won't start	Image has no RoadRunner binary (<code>OCTANE_SERVER</code> default)	Install RoadRunner, or set <code>OCTANE_SERVER=frankenphp</code> / <code>swoole</code> and install that runtime
MinIO uploads 403 / bucket missing	Path-style/endpoint not set, or bucket not created	<code>AWS_ENDPOINT=http://minio:9000</code> , <code>AWS_USE_PATH_STYLE_ENDPOINT=true</code> , <code>mc mb local/printflow360</code>
pdf-service appears dormant (no effect)	<code>PDF_SERVICE_FEAT_*</code> flags default OFF — Laravel uses its own dompdf path	Expected; flip the relevant flags to route work to pdf-service

7. Rollout order (de-risked)

Because **pdf-service feature flags default OFF**, Laravel keeps using its own dompdf path — so you can ship the **proxy + web + db tier first** and add the pdf tier later:

1. `postgres` + `redis` (+ `minio` for dev) → healthchecks green.
2. `api` (with the **new** `/api/health` route + Postgres wiring) → `migrate --force` + caches.
3. `queue-worker` + `scheduler` (now that `QUEUE_CONNECTION=redis`).
4. `admin` + `storefront` (mind SSR-vs-browser API bases; set `NITRO_REDIS_URL`).
5. `proxy` (wildcard tenant routing, preserve `Host`).
6. `pdf-service` + `pdf-worker` last (low-risk; flip `PDF_SERVICE_FEAT_*` when ready).

Keep `deploy/deploy.sh` + PM2 as the bare-metal fallback until the container path is proven — note `deploy/pm2/ecosystem.config.cjs` is referenced but **missing**, and there is **no nginx config in** `deploy/` , so Docker here is genuinely greenfield.

Legal, Compliance & Data Privacy

Source: [readme/research/engineering-ops/LEGAL_COMPLIANCE_AND_DATA_PRIVACY_2026-06-16.md](#)

IN SHORT — KEY TAKEAWAYS

This document is about the legal risks the Print-Flow-360 platform faces right now — and what must be fixed before real customers start using it. It covers privacy laws, copyright rules for printed goods, and the agreements the platform needs with store owners. If these issues are not fixed, the business could face large fines or lawsuits.

The main parts explained simply:

- **Urgent problems to fix first** — Six issues ranked by how dangerous they are. The worst two are: (1) the platform prints customer-uploaded artwork onto physical goods with zero legal protection in place, and (2) credit card security codes (CVV) are being saved in the database, which is a serious breach of card-network rules. Both must be fixed before any real customer uses the system.
- **Who “owns” each person’s data** — When a store owner’s customer places an order, the store owner is responsible for that person’s data — the platform just handles it on their behalf. But if the platform ever uses that customer data for its own purposes (like marketing), it becomes legally responsible too. The document maps out exactly which data belongs to whom.
- **GDPR and CCPA (privacy laws)** — European and California privacy laws give people the right to see, correct, or delete their personal data. Right now, the platform has no tools to do any of that. It must build a way for store owners to export or delete a customer’s full record, and it must handle “forget me” requests properly without wiping records that are legally required to be kept (like invoice history).
- **Consent — recording that someone agreed** — When a customer ticks “I agree” at signup or checkout, that agreement is currently never saved anywhere. The law requires a timestamped record of exactly what was agreed to and when. This needs to be stored properly.
- **Cookie consent banner** — EU and UK visitors must be asked before any non-essential tracking starts. US visitors must be able to opt out. Right now the storefront has no cookie banner at all.
- **Marketing data sent without permission** — Every time a new customer is created, their personal information is automatically sent to an external marketing system — with no check that the customer agreed to this. This needs a consent gate before it can run.
- **Copyright and printed goods** — When the platform prints a customer’s uploaded image onto a product, it becomes a manufacturer, not just a website host. The legal rule that protects websites from copyright claims (the DMCA “safe harbor”) does NOT protect a company that physically makes the product. The platform needs: a form for copyright owners to report stolen artwork, a clear process to remove flagged files quickly, a counter for repeat offenders, and a checkbox at upload time where customers confirm they own the image.
- **Data Processing Agreement (DPA)** — Before the platform processes any EU/UK customer’s data on behalf of a store owner, a formal written contract (called a DPA) is legally required. None exists yet.
- **Terms of Service and Privacy Policy** — Store pages currently have no real legal text — a customer might land on an empty Terms page. Default legal page templates need to ship so no store goes live without them.
- **What needs a lawyer** — Thirteen items in the document cannot be handled by engineers alone: writing the actual contract text, registering the copyright agent with the US government, deciding how much content screening to do, and more.

What to do with this: Fix the CVV storage and add an upload ownership checkbox immediately — these are the two issues that could produce the largest legal penalties the fastest. Then hire a lawyer to draft the Data Processing Agreement and register the DMCA agent before onboarding any paying merchant.

This is internal research and guidance, not legal advice. It maps what the Print-Flow-360 codebase actually does today against GDPR/UK GDPR, CCPA/CPRA, ePrivacy/cookie law, and US/EU copyright-and-trademark law for print-on-demand. Every item marked “needs a lawyer” must be reviewed by qualified counsel before relying on it. Dated **2026-06-16**.

Audience: founders, engineering leads, and ops. Plain-language where possible; file paths included so engineers can find the exact code.

Related docs: `readme/SECURITY_GUIDELINES.md` , `readme/PAYMENT_GATEWAYS.md` , `readme/NOTIFICATIONS_AND_QUEUES.md` , `readme/CUSTOM_FIELDS_AND_ADDRESS_ATTENTION_SCOPE_2026-06-07.md` , `CLAUDE.md` §5 (tenancy + data-integrity invariants).

Table of Contents

1. TL;DR — non-negotiables before onboarding real customers
 2. Our data-processing model
 3. GDPR / CCPA obligations
 4. Terms of Service & Privacy Policy
 5. Cookie consent
 6. IP / copyright liability for uploaded designs
 7. DPA (Data Processing Agreement)
 8. Action checklist
 9. What needs a qualified lawyer
-

1. TL;DR — non-negotiables before onboarding real customers

Ranked by risk. The first item is the one that can produce a six-or-seven-figure judgment against the **company itself**, not just a tenant.

#	Risk	Why it's the top of the list	Status today
1	Printing customer-uploaded copyrighted/trademarked artwork.	Print-Flow-360 doesn't just <i>host</i> images — it manufactures physical goods bearing them (the pdf-service builds print-ready PDFs; orders go to production). US courts have repeatedly held that the DMCA “safe harbor” that protects an online <i>host</i> does not protect the <i>manufacture and sale</i> of a t-shirt/poster/card bearing an infringing image (<i>Greg Young Publishing v. Zazzle</i>). [^zazzle] Trademark has no DMCA safe harbor at all. Because the SaaS operator runs the print pipeline for every tenant store, it structurally sits in the “manufacturer” seat (the <i>Zazzle/RageOn</i> loser seat), not the “passive marketplace” seat.	No DMCA agent, no takedown workflow, no upload ownership attestation, no prohibited-content policy, no repeat-infringer policy. The “I agree to Terms” checkbox at signup is UI-only and never saved. P0.
2	Storing card CVV in plaintext. <code>payment_profiles</code> has a <code>cvv</code> integer column (migration <code>2025_04_09_173425</code>), and full <code>card_number</code> + <code>cvv</code> are accepted server-side in <code>PaymentProfileController.php</code> , <code>InvoiceController.php</code> , and <code>Admin/TenantController.php</code> and relayed to <code>Authorize.Net</code> with no client-side hosted-fields tokenization. Reading nulls the CVV (<code>PaymentProfileController.php:49</code>) — but storing CVV at all, even transiently, is a hard PCI-DSS prohibition.	This is a contractual breach with the card networks and a major breach-liability multiplier.	Live in code. P0 — stop storing CVV; move to gateway hosted fields / tokenization. See <code>readme/PAYMENT_GATEWAYS.md</code> .
3	No Data Processing Agreement (DPA) with merchant tenants. Each merchant is the controller of their shoppers' data; the platform is the processor . GDPR Art. 28 makes a written DPA mandatory before processing. [^art28]	Without it, every EU/UK merchant you onboard is technically non-compliant <i>and so are we</i> . Enterprise/B2B merchants will demand it in procurement.	None exists. P0 (Legal).
4	No way to honor a data-subject request. No data export, no self-service account deletion, no erasure that respects order/invoice snapshots. GDPR requires a response within one month ; CCPA within 45 days .	A single “delete my data” email we can't action is a reportable failure.	None exists. P0–P1 (Engineering).
5	No cookie-consent banner. Source search is clean — no banner, no consent manager, no GPC handling. EU/UK visitors require prior opt-in before any non-essential script; US visitors require a Do-Not-Sell/Share path + Global Privacy Control handling.	Direct ePrivacy/PECR and CCPA exposure; the FR regulator has issued nine-figure fines for banner design alone.	None exists. P1.

#	Risk	Why it's the top of the list	Status today
6	Marketing data leaves the platform with no consent gate. <code>Customer::booted()</code> auto-dispatches <code>SyncCustomerToMarketingJob</code> on every customer create (<code>app/Models/Customer.php:105</code>) — customer PII flows to an external marketing system the moment a record is created, with no recorded consent.	Third-party data sharing without a lawful basis.	Live in code. P1.
7	Tenant isolation is row-level only. Single shared PostgreSQL DB; all <code>stanc1/tenancy</code> DB/cache/filesystem bootstrappers are commented out in <code>config/tenancy.php</code> . One missing <code>BelongsToTenant</code> scope = cross-tenant PII leak. Per-tenant S3 is opt-in; otherwise artwork shares one local <code>public</code> disk.	A leak here is a breach across <i>many</i> merchants at once.	High-trust invariant; needs automated cross-tenant tests. P1.

Bottom line: items 1 and 2 are existential and must be fixed before real customer traffic. Items 3–4 are launch-blockers for any EU/UK or California-touching merchant.

2. Our data-processing model

The single most important legal fact about this architecture is the **controller/processor split**, and Print-Flow-360 runs **two roles in parallel**.

2.1 Two roles, decided by *who chooses the purpose*

The line is set by **who determines the purposes and means** of the processing (GDPR Art. 4(7)/(8)), not by who holds the data or who is bigger.[^art28]

Data	Controller	Processor	Contract needed
A merchant's storefront shoppers — their customers' names, emails, addresses, orders, designs	The merchant tenant (they decide why/how)	The platform (acts on the merchant's instructions)	Art. 28 DPA (platform ← merchant)
The platform's own business data — merchant account holders, store-owner contact/billing, platform analytics, marketing to merchants, support tickets, staff	The platform	—	Platform's own privacy notice + lawful basis (not a DPA)

The trap: the moment the platform uses *tenant customer data for its own purposes* — cross-tenant analytics, training models, marketing — it stops being a processor and becomes a **controller** for that processing, inheriting full controller liability.[^controller] The `SyncCustomerToMarketingJob` auto-sync (gap

#6 above) and any cross-tenant analytics are exactly the kind of processing that can flip this switch. Keep tenant customer PII walled to the merchant's purposes.

2.2 The PII we actually hold (from the survey)

Tenant-scoped (merchant is controller, platform is processor):

Category	Where	Notable fields
Customers	<code>app/Models/Customers.php</code> ; migrations <code>2025_01_26_112622</code> , <code>2026_02_14_100001</code> , <code>2026_06_04_000002</code>	company_name, primary_contact_name/email/number(+ext), additional_contacts, account_number, hashed password, email_verified_at, remember_token, OAuth provider/provider_id, last_login_at, login_count, custom_field_values (JSON), notification_preferences
Addresses	<code>app/Models/Address.php</code> ; migrations <code>2025_01_26_131218</code> , <code>2026_03_15_100003</code>	address_line1–4, city, region, zip, country, attention_to (recipient name), latitude/longitude, custom_field_values
B2B accounts	<code>app/Models/B2B/*</code>	company accounts, departments, b2b_role
Extra contacts	<code>Contact</code> / <code>ContactEmail</code> / <code>ContactPhone</code>	named contacts with emails + phones
Quotes & orders	migrations <code>2025_02_01</code> , <code>2025_12_15_201408</code>	customer_note, special_info, billing/shipping/install attention_to, address snapshots
Newsletter	<code>newsletter_subscribers</code> (<code>2026_03_15_100002</code>)	email, subscribed_at, unsubscribed_at
Reviews / logs	<code>ProductReview</code> (customer_id), <code>StorefrontErrorLog</code> context	customer-linked content
Uploaded artwork & designs	<code>uploads</code> (<code>2025_07_26_222949</code>), <code>designer_documents</code> (<code>app/Models/Design.php</code>)	customer artwork (PDF/AI/EPS/PSD/TIFF up to 200 MB), designer_json, preview_image_url, print_ready_file, customer_id — may contain third-party creative IP

Central/landlord (platform is controller) — note these have *no* `tenant_id` :

Category	Where	Notable fields
Staff	<code>users</code>	email, hashed password
Store-owner businesses	<code>tenants</code> (<code>2019_09_15_000010</code>)	business name, street/city/state, logo, <code>allowed_ips</code> , <code>Authorize.Net keys</code> , <code>bucket_name</code>
Payment data	<code>payment_profiles</code> (<code>2025_03_30_...</code> , + <code>cvv</code> <code>2025_04_09_173425</code>), <code>payments</code> (<code>2025_02_05_...</code>)	<code>card_last_four</code> , <code>card_type</code> , <code>expiration_date</code> , <code>billing_information</code> (JSON), <code>cvv</code> ← remove ; payments store <code>transaction_id/amount/status/payload</code>
Gateways	<code>payment_gateways</code> (<code>2025_11_02</code>)	per-tenant gateway config (JSON)

2.3 Sub-processors (must be disclosed and flow-down-bound)

Every external service that touches tenant customer PII is a **sub-processor** under Art. 28(4) and must be (a) bound to obligations equivalent to our merchant DPA, and (b) listed publicly with advance notice of changes.

Sub-processor	Role	Data it sees	Notes
Payment gateways — <code>Authorize.Net</code> (order/invoice CIM), Stripe, Razorpay, PayPal, PayTM (billing)	Payment processing	Card data, billing info	<code>app/Services/PaymentService.php</code> , <code>app/Services/Billing/SubscriptionGatewayRegistry.php</code> . PCI: move to their hosted fields so PAN/CVV never transit our app.
Email / SMS provider	Transactional + marketing delivery	Names, emails, phone	See <code>readme/NOTIFICATIONS_AND_QUEUES.md</code>
External marketing system	<code>SyncCustomerToMarketingJob</code> target	Customer PII	No consent gate today — gap #6.
Cloud storage / S3	File storage	Artwork, designs, snapshots	Per-tenant bucket <code>tenants.bucket_name</code> <i>only if configured</i> ; else shared local <code>public</code> disk
pdf-service (Node, port 4000)	Print-ready PDF + thumbnail generation	Artwork + design data	Shares the same PostgreSQL DB (read-only) and per-tenant S3. In-scope sub-processor/internal processing component — must inherit the same encryption, isolation, access control. <code>pdf-service/src/services/tenant/tenantContext.js</code> , <code>pdf/pdfStorageService.js</code> , <code>storage/S3Storage.js</code> .

3. GDPR / CCPA obligations

For each obligation: the rule, then “**what we must build**” tied to this codebase.

3.1 Data-subject rights — access, export (portability), rectification, erasure

- **GDPR:** respond within **one month** (extendable +2 months for complex requests).[^erasure] As a **processor**, our duty is to give the **merchant-controller the tooling** to action these; the merchant decides and replies to their shopper.
- **CCPA/CPRA:** rights to **Know**, **Delete**, **Correct**, **Opt-out of sale/share**, **Limit sensitive-PI**, **Equal treatment**. Confirm receipt in **10 business days**, substantive response in **45 days** (extendable to 90).[^ccpa]

What we must build:

- **Per-tenant DSAR export** — an action in the *merchant store admin* (not global settings — UX rule: a setting about a customer belongs near customers) that exports one shopper's full record as machine-readable JSON/CSV: profile, addresses, order/quote history, reviews, **and custom_field_values**. Reuse the existing tenant-scoped query layer so it stays controller-isolated. *Round-trip test (per the silent-drop rule): assert every collected field, including custom fields, is in the export.*
- **Self-service preference + request entry** in the storefront profile (request export / deletion / correction).
- **Tenant-isolation guard on every DSAR/export/delete path** so one merchant can never read or erase another merchant's customer — extend **BelongsToTenant** and add a test proving cross-tenant access **fails** (this is the design-API customer-scope leak risk class).

3.2 Erasure vs. snapshots — the hard part, call it out

GDPR's right to erasure is **not absolute**. Data kept for a **legal obligation** (tax/accounting on invoices) or for the **establishment/exercise/defence of legal claims** is **exempt**.[^erasure] Today we only have **SoftDeletes** on Customer/Quote/Invoice — that **retains** data, it does not erase it, and there is no hard-delete/anonymization path.

The correct mechanic (do NOT hard-DELETE everything):

1. **Anonymize the live Customer record** — replace name/email/phone and linked address fields with placeholders, sever auth (password, remember_token, OAuth ids).
2. **Retain the legally-required snapshots** — order/invoice ***_address_snapshot** and **customer_snapshot** stay, in **minimised** form, for the statutory tax/dispute window only.
3. This intersects the existing snapshot-aware code paths (**CLAUDE.md** §5 “records that snapshot data”) — **extend** them, don't bypass them.

What we must build: an erasure workflow on the Customer model that anonymizes live PII, retains minimised snapshots, and is covered by a test asserting **live PII is gone but the legally-required snapshot persists**.

3.3 Consent & lawful basis

- **Order fulfilment** rests on **contract** or **legitimate interest** (the latter needs a documented 3-part balancing test).[^consent]
- **Marketing email/SMS and non-essential cookies** require **opt-in consent** — specific, informed, freely given, unambiguous, **no pre-ticked boxes**, withdrawable.[^consent]
- Today: the signup “I agree to Terms/Privacy” checkbox (**login.vue** , **LoginModal.vue**) is **UI-only, never persisted** — no consent column/table/timestamp/version anywhere.

What we must build:

- A **consent record** (purpose, timestamp, source, notice version) for marketing and cookies.
- A **storefront preference centre**; wire **withdrawal** to suppress future sends.
- Gate `SyncCustomerToMarketingJob` and the email-campaign module on recorded marketing consent — fixes gap #6.

3.4 Breach notification — the 72-hour chain

- A breach = accidental/unlawful destruction, loss, alteration, or unauthorised disclosure/access.^[^breach]
- **Controller** notifies the supervisory authority **within 72 hours** of becoming aware (unless unlikely to be a risk); high-risk breaches also require notifying affected individuals.
- **We are the processor** → we must notify the affected merchant-controller **“without undue delay”** (contract this to 24–48h). *That* starts the merchant's 72-hour clock.

What we must build: breach-detection logging + an internal incident workflow recording **who/what/when** that can generate a per-tenant notification. **Reuse the existing audit/activity-log mechanism** (`CLAUDE.md` §5) — do not invent a new one.

3.5 Retention & minimisation

GDPR Art. 5 requires keeping PII no longer than necessary. Today, soft-deletes retain data indefinitely and there's no retention mechanism.

What we must build: per-category retention timers (scheduled jobs) — anonymize inactive customers after a defined period, purge abandoned-cart PII, enforce the invoice-retention window — and surface the schedule in an admin-visible setting. **Retention values are a lawyer input** (statutory tax/dispute periods).

3.6 CCPA specifics (only if thresholds are crossed)

CCPA/CPRA applies to a for-profit business doing business in California meeting **any** of: >\$25M revenue; buys/sells/shares PI of **100,000+** CA residents/households; or **≥50%** of revenue from selling/sharing PI.^[^ccpa-thresh] We may not cross these yet — **but a merchant tenant might**, and will demand contractual support. Routine processing under our merchant contracts makes us a **“service provider”** (the CCPA analogue of a processor), which keeps tenant transfers from being “sales” — *provided we never use tenant data for our own purposes*.

4. Terms of Service & Privacy Policy

There are **two tiers** of agreement, and today neither is properly implemented (the `/privacy-policy` and `/terms` links resolve only via the generic store-authored CMS `[slug].vue` page renderer — content may be **absent or empty by default**).

4.1 Tier 1 — Platform ↔ Merchant (the store owner signs up)

- **Master Services Agreement / Platform ToS** — service scope, fees, acceptable use, **IP ownership and indemnity flow-down** (merchants must attest their customers own uploaded artwork and indemnify the platform), liability caps, termination/data-return.
- **Data Processing Agreement** — see §7. Mandatory, separate document, referenced by the ToS.

- **Where to surface:** merchant onboarding / store-creation flow, with a **persisted acceptance record** (who, which version, when).

4.2 Tier 2 — Store ↔ Shopper (per storefront)

- **Storefront Terms of Sale + Privacy Policy + Acceptable-Use / Prohibited-Content + DMCA / IP policy + Cookie Policy.**
- The merchant is the controller, so these are nominally the *merchant's* documents — but the platform must (a) provide sensible **default templates**, (b) make them real, reachable pages, and © **persist shopper acceptance** at signup/checkout.

What we must build:

- Replace the UI-only checkbox with a **persisted consent/acceptance record** (purpose, ToS/Privacy version, timestamp, source) at storefront signup and checkout.
- Ship **default Privacy/ToS/Acceptable-Use templates** so a store is never live with empty legal pages; render the DMCA agent + prohibited-content policy as real footer-linked pages (see §6).
- **Lawyer drafts the actual wording** for both tiers.

5. Cookie consent

There is no cookie-consent banner, consent manager, or GPC handling anywhere (source-clean; the only “consent”-like UI is the non-persisted login checkbox). Cookies/storage in use today:

- **Cookies:** `customer_token` (Sanctum auth, 30d, secure+strict), `token`, `dismissed_promo_bars`.
- **localStorage:** `tenant`, `storefront-cart-session`, `storefront-designer-auth-token`, `tax_rate`, `chat_token`, `exitPopupSeen`.
- **sessionStorage:** `offerBannerDismissed`, product config snapshots, checkout drafts, category cache.

5.1 What's required

- **EU/UK (ePrivacy Art. 5(3) — this is what triggers the banner, not GDPR alone):** prior, granular, per-category opt-in before any non-essential cookie or script. **Reject-All must be as easy as Accept-All** on the first layer; no pre-ticked boxes; persistent “Cookie preferences” link; re-prompt no sooner than ~6 months (FR) / ~12 months (general).^{[^banner][^reject]}
- **US (CCPA/CPRA + ~12 states):** opt-out model — cookies may fire by default, but provide a “Do Not Sell or Share My Personal Information” link and **automatically honor the Global Privacy Control (GPC)** signal (`Sec-GPC: 1` header / `navigator.globalPrivacyControl`). GPC **overrides** a banner “accept”. From **Jan 1 2026**, California requires a user-facing **confirmation** that the opt-out was applied.^[^gpc]

5.2 Categories

Category	Consent?	Examples
Strictly necessary	No (disclose only)	<code>customer_token</code> , cart session, the consent record itself
Functional / preferences	Yes (EU)	<code>dismissed_promo_bars</code> , UI prefs
Analytics	Yes (EU)	GA/GTM
Marketing / advertising	Always	ad pixels, retargeting

5.3 Nuxt storefront implementation note

- Build a **geo-aware gate**: EU/UK → opt-in (block scripts pre-consent); US → opt-out + Do-Not-Sell + GPC; **default to opt-in when geo is unknown**.
- **Block all non-essential scripts until the matching category is accepted** — use Nuxt Scripts `useScriptTriggerConsent()` / `defaultConsent` with Google Consent Mode v2; never hardcode `<script>` tags that fire on load.^[^nuxt]
- **SSR-safe**: `document.cookie` is unavailable during SSR — guard client reads (`import.meta.client` / `<ClientOnly>` / `onMounted`) to avoid hydration mismatch; read the consent cookie server-side for consistent SSR tag output. **Keep consent paths no-store** so one visitor's choice can't be cached per-tenant and served to another (mind `frontstore/nuxt.config.ts` route-cache rules).
- Reserve layout space + show a loading state for the banner per the project's **async third-party-widget UX rule** (`CLAUDE.md` §0); test at **375px / 768px**.
- A **GPC listener** (server header + client property) that auto-applies and persists the opt-out and overrides a conflicting accept.

6. IP / copyright liability for uploaded designs

This is the central risk. Print-Flow-360 reproduces customer-uploaded artwork onto physical goods for every tenant — it is a **manufacturer**, not a passive host.

6.1 Who is liable

Three parties can be liable when an infringing upload is printed: the **customer** (uploader, primarily responsible for owning rights), the **merchant** (lists/sells it), and the **platform** (hosts **and manufactures**). The uploader remains primarily liable, but the printer's liability is real and fact-dependent — it is **not** a blanket rule that the printer is always responsible. Outcomes split sharply on **how active the platform is**.

6.2 The DMCA nuance that matters for *physical print*

- **DMCA §512© safe harbor can protect the online *display* of a user image — but multiple courts have held it does NOT protect *manufacturing and selling a physical good bearing that image***, because making a t-shirt/poster is not “storage at the direction of a user” (*Greg Young Publishing v. Zazzle*).^[^zazzle] **Caveat:** this rests on **district-court rulings**, not a uniform circuit rule — *Pixels* (a truly passive POD operator) *kept* the §512© harbor.^[^pixels] The honest posture: the physical-manufacture-isn't-storage theory

is **real and has succeeded**, but it is fact-dependent. Because we run the print pipeline for every tenant, we should assume we sit on the **exposed** side.

- **What kills the harbor:** direct profit + right/ability to control what's sold + slow takedowns (*Feingold v. RageOn* — 18–23 day removals, **one** factor alongside control/profit); **stripping copyright metadata** on upload and curating/pricing/syndicating user content (*Gardner v. CafePress*) and **willful blindness** to obvious red flags.
- **Trademark has NO DMCA safe harbor.** A platform that “brings trademark-offending products into being” can face **direct** “use in commerce” liability (*Ohio State v. Redbubble*, 6th Cir.) — but a marketplace with a **real, enforced** takedown + repeat-infringer process **can still win** (*Atari v. Redbubble* jury verdict). Counterfeit marks carry **statutory damages up to \$2M per mark** (willful) and risk **customs seizure**.
- **Right of publicity / likeness** (celebrity faces, athletes) is a **separate state-law tort** with **no** DMCA process — a classic POD exposure, handled via the prohibited-content policy.
- **§512(f) over-removal liability:** acting on **bad-faith or fair-use-blind** takedowns creates its *own* liability (*Lenz v. Universal*). Don't auto-strike legitimate parody/fair-use/public-domain art — the **counter-notice path must be genuinely available**.

6.3 Required safeguards

Process/legal:

- **Register a Designated DMCA Agent** with the U.S. Copyright Office and **renew every 3 years** (an expired registration voids the harbor); publish name, physical address, phone, email on every storefront.
- **Notice-and-takedown** that removes flagged material **expeditiously** (hours/days), a **counter-notice** path, and an **adopted and enforced repeat-infringer termination** policy.
- A clear **prohibited-content / acceptable-use policy** (copyright, trademarks/logos, counterfeits, rights of publicity), surfaced where users upload.
- **Do NOT strip image metadata** in the upload/thumbnail/pdf pipeline (it defeated CafePress) — preserve original or audit.

What we must build:

- **Upload-time ownership attestation** — “I own or am licensed to use this artwork,” timestamped and logged, before the upload enters the print pipeline. **Wire it through the two-step Upload::consumeUpload flow** so it round-trips and persists (no silent drop). Files:
`app/Http/Controllers/Api/Storefront/StorefrontArtworkUploadController.php` ,
`app/Models/Upload.php` .
- **Public IP-complaint intake form** (unauthenticated, under `routes/public-api.php`) capturing all statutory notice fields, validated server-side, routed to a moderation queue.
- **Moderation/takedown dashboard** for staff: view a flagged design, see which orders/jobs reference it, disable it + block new prints, notify merchant/customer in plain language, record who/what/when (**reuse the audit log**).
- **Design state machine** on `app/Models/Design.php` — `flagged` / `under_review` / `removed_ip` / `restored` — so a flagged file is blocked from new jobs but history (snapshots) is preserved. Plus an **order/job IP-hold** to pause fulfillment.
- **Repeat-infringer strike counter** on customer (and merchant) with a configurable suspension threshold.

- **Designated-agent + policy pages** rendered as real footer-linked storefront pages.
- Optional **proactive screening hook** in the pdf-service (perceptual-hash / brand deny-list) — but **gate carefully**: building detection can create “red-flag knowledge” that *defeats* the harbor. This is a legal-strategy call (see §9), not an engineering default.
- **EU DSA parity**: the same intake usable for “illegal content / infringing goods” reports, plus trader-info capture for B2B merchants.

6.4 Contractual shields (intent of the clauses — lawyer writes the wording)

- **User reps/warranties**: the uploader owns or is licensed to use everything uploaded; it infringes no copyright, trademark, privacy, or publicity right.
- **Indemnification**: the uploader (and, realistically, the **merchant** — individual-customer indemnities are often uncollectable) defends/indemnifies the platform against third-party IP claims.
- **Content licence**: a non-exclusive, worldwide, royalty-free licence to the platform/merchant to **reproduce, print, and fulfil** the work — broad enough to *manufacture goods*, not just display.

7. DPA (Data Processing Agreement)

A written **Art. 28 DPA is mandatory** with every merchant tenant before processing their shoppers' data.[^art28]

7.1 Art. 28(3) required-contents checklist

The DPA must state subject-matter, duration, nature/purpose, data types, and categories of data subjects — then bind the platform (processor) to:

- [] (a) process **only on the controller's documented instructions** (incl. transfers)
- [] (b) ensure **personnel confidentiality**
- [] © implement **Art. 32 security** (encryption in transit + at rest, tenant isolation, RBAC, resilience/restore, periodic testing)
- [] (d) respect **sub-processor** authorisation rules
- [] (e) **assist the controller with data-subject requests** (the §3.1 tooling)
- [] (f) assist with **Art. 32–36** (security, breach notice, DPIAs)
- [] (g) **delete or return** all personal data at end of service
- [] (h) make available info to **demonstrate compliance** and allow **audits**

7.2 Sub-processor list + change notification

- Publish and maintain a **/sub-processors** page (name, purpose, location, “last updated”): payment gateways, email/SMS, external marketing system, S3/storage, **pdf-service** (§2.3).
- Use **general written authorisation** with **advance notice (~30 days)** before adding/replacing a sub-processor + a mechanism for merchants to **object**.
- Every sub-processor must be **flow-down-bound** to equivalent obligations; the platform stays **fully liable** for their failures.

7.3 International transfers (post-Schrems II)

If any host or sub-processor is outside the EEA/UK, a transfer mechanism + documented assessment is required:[^transfer]

- **EU → US:** rely on **EU-US Data Privacy Framework** self-certification, **or** EU SCCs (2021 modular) + a **Transfer Impact Assessment**.
- **UK:** the **UK IDTA** or the **UK Addendum** to EU SCCs + a Transfer Risk Assessment. (UK → serving both EU and UK merchants may need both.)
- **Engineering input:** a **data-flow inventory** — which fields each sub-processor receives and where it's hosted — feeds the sub-processor list and the transfer assessments. **Lawyer selects/executes the mechanism.**

7.4 Engineering to support the DPA

- **Per-merchant data export** ("return all data") + **hard-delete on offboarding** (Art. 28(3)(g)) — cascade deletes to S3 + pdf-service artifacts.
- **Prove tenant isolation** with automated cross-tenant-denial tests; TLS everywhere + at-rest encryption for Postgres and per-tenant S3; **RBAC + access logging** for staff touching tenant data; **breach-detection pipeline** that can identify affected tenants.

8. Action checklist

Separated into **Engineering build tasks** and **Legal/Ops tasks**. Priority: **P0** = before onboarding real customers · **P1** = soon after launch · **P2** = ongoing hardening.

8.1 Engineering

#	Item	Owner	Priority
E1	Stop storing CVV — drop the <code>cvv</code> column; move card capture to gateway hosted fields/tokenization ; remove plaintext PAN/CVV handling in <code>PaymentProfileController</code> / <code>InvoiceController</code> / <code>Admin/TenantController</code>	Engineering	P0
E2	Upload-time ownership attestation wired through <code>Upload::consumeUpload</code> (timestamped, persisted, no silent drop)	Engineering	P0
E3	DMCA intake form (public route) + takedown/moderation dashboard + Design state machine (flagged/removed/restored) + order/job IP-hold	Engineering	P0
E4	Repeat-infringer strike counter + suspension threshold (customer + merchant)	Engineering	P1
E5	Per-tenant DSAR export (JSON/CSV incl. <code>custom_field_values</code>) in store admin + round-trip test	Engineering	P0–P1
E6	Erasure-by-anonymization workflow respecting order/invoice snapshots + test (live PII gone, snapshot kept)	Engineering	P0–P1
E7	Cross-tenant isolation tests on every DSAR/export/delete + general scope audit	Engineering	P0–P1
E8	Consent record + storefront preference centre ; persist signup/checkout ToS acceptance (replace UI-only checkbox)	Engineering	P1
E9	Gate <code>SyncCustomerToMarketingJob</code> + email campaigns on recorded consent	Engineering	P1
E10	Cookie-consent banner (geo-aware opt-in/opt-out, script gating, GPC listener, CA confirmation, preference centre)	Engineering	P1
E11	Breach-detection logging + incident workflow (reuse audit log)	Engineering	P1
E12	Retention timers (inactive customers, abandoned carts, invoice window) + admin-visible schedule	Engineering	P1–P2
E13	<code>/sub-processors</code> page + change-notification mechanism; data-flow inventory	Engineering	P1
E14	Per-merchant export + hard-delete on offboarding (cascade to S3 + pdf-service)	Engineering	P1
E15	Default Privacy/ToS/Acceptable-Use/DMCA storefront page templates so stores are never live with empty legal pages	Engineering	P1
E16	Do NOT strip image metadata in upload/pdf pipeline (audit current behavior)	Engineering	P1
E17	Enforce per-tenant S3 + at-rest encryption + RBAC/access logging ; treat pdf-service as in-scope sub-processor	Engineering	P2

8.2 Legal / Ops

#	Item	Owner	Priority
L1	Draft + sign the Art. 28 DPA (platform ↔ every merchant) and the platform ↔ merchant Master Services Agreement	Legal	P0
L2	Register the Designated DMCA Agent with the U.S. Copyright Office; calendar the 3-year renewal	Legal/Ops	P0
L3	Draft the contractual shields — upload reps/warranties, indemnity (incl. merchant flow-down), content licence to print/fulfil	Legal	P0
L4	Draft customer-facing Privacy Policy, ToS, Acceptable-Use/Prohibited-Content, Cookie Policy, DMCA policy (both tiers)	Legal	P0–P1
L5	Lawful-basis determination per purpose + Legitimate Interests Assessment sign-off	Legal	P1
L6	CCPA threshold + “sale/share” analysis (platform and per-merchant); GPC/Do-Not-Sell applicability	Legal	P1
L7	Statutory retention periods (tax/dispute) → feed E12 timer values	Legal	P1
L8	Transfer mechanism (DPF / EU SCCs + TIA / UK IDTA + TRA) selection + signing	Legal	P1
L9	Art. 30 Records of Processing (processor-side + platform controller-side) + sub-processor list maintenance	Legal/Ops	P1
L10	DPIA / DPO / EU-UK representative scoping	Legal	P1
L11	DSA scope determination (hosting service / online marketplace / KYBC / EU point of contact)	Legal	P1–P2
L12	Proactive-screening posture decision (detection vs. red-flag-knowledge trade-off)	Legal	P2

9. What needs a qualified lawyer

Reminder: this document is research and engineering guidance, not legal advice. The items below require sign-off from privacy / IP / internet counsel before the company relies on them.

- Drafting + signing the Art. 28 DPA, Master Services Agreement, and Standard Contractual Clauses / IDTA** — engineering cannot author binding contract text.
- The contractual IP shields** — exact wording of upload reps/warranties, indemnification scope (and merchant flow-down + insurance), and the content-licence grant broad enough to *manufacture goods*.
- DMCA agent registration** and a candid assessment that **§512© likely does not shield the physical-manufacture step** — so the company knows its real exposure.
- Proactive-vs-reactive moderation posture** — how much screening to do without creating “knowledge”/red-flag awareness that *defeats* the safe harbor (a legal-strategy call).
- Trademark + counterfeit + right-of-publicity strategy** (no safe harbor) — brand allow/deny lists, counterfeit handling, celebrity-likeness rules.

6. **Lawful-basis determination** per processing purpose and Legitimate Interests Assessment sign-off.
 7. **CCPA/CPRA threshold + “sale/share” classification**, GPC obligations, and applicability of other US state laws (Virginia, Colorado, etc.).
 8. **Cookie classification** (necessary vs. functional vs. analytics vs. marketing — the main legal failure mode), cookie-wall/legitimate-interest edge cases, FR 6-month vs. UK post-DUAA divergences, and consent-banner wording.
 9. **Statutory retention periods** that override erasure → set the values engineering implements.
 10. **International-transfer mechanism selection** + TIA/TRA conclusions and supplementary measures.
 11. **DPIA / DPO / EU-UK representative** requirements; **DSA scope** (hosting service / online marketplace / KYBC / EU point of contact).
 12. **Privacy policy + all customer-facing notice wording** (both tiers).
 13. **Liability allocation and breach-notification responsibilities** between platform and merchants.
-

Footnotes / sources

[^zazzle]: DMCA safe harbor disappears once infringing images are printed on physical items — <https://techdirt.com/articles/20170629/16141537702/court-says-dmca-safe-harbors-disappear-once-infringing-images-are-printed-physical-items.shtml> [^pixels]: POD website not directly liable due to lack of volitional conduct (Sid Avery v. Pixels) — <https://advertisinglaw.fkks.com/post/102gsj3/print-on-demand-website-not-liable-for-direct-copyright-infringement-due-to-lack> [^rageon]: POD vendor denied DMCA safe harbor (Feingold v. RageOn) — <https://blog.ericgoldman.org/archives/2020/07/print-on-demand-vendor-doesnt-qualify-for-dmca-safe-harbor-feingold-v-rageon.htm> [^cafepress]: Gardner v. CafePress (metadata stripping + active control defeat 512©) — <https://cdas.com/cafepress-cannot-beat-copyright/> [^osu]: Ohio State University v. Redbubble (direct “use in commerce”, 6th Cir.) — <https://malloylaw.com/ohio-state-university-v-redbubble-inc-a-potential-sigh-of-relief-for-trademark-owners/> [^atari]: Jury finds for online marketplace over Atari — <https://www.morganlewis.com/pubs/2021/11/jury-finds-for-online-marketplace-over-atari-in-trademark-infringement-case> [^512]: U.S. Copyright Office §512 — designated agent + repeat-infringer policy — <https://www.copyright.gov/512/> [^art28]: GDPR Art. 28 (DPA + processor obligations) — <https://gdpr-info.eu/art-28-gdpr/> [^controller]: ICO — what it means to be a processor (and when you become a controller) — <https://ico.org.uk/for-organisations/uk-gdpr-guidance-and-resources/controllers-and-processors/controllers-and-processors/what-does-it-mean-if-you-are-a-processor/> [^erasure]: ICO — right to erasure (and its exemptions) — <https://ico.org.uk/for-organisations/uk-gdpr-guidance-and-resources/individual-rights/individual-rights/right-to-erasure/> [^consent]: ICO — when is consent appropriate — <https://ico.org.uk/for-organisations/uk-gdpr-guidance-and-resources/lawful-basis/consent/when-is-consent-appropriate/> [^breach]: GDPR Art. 33 — breach notification — <https://gdpr.eu/article-33-notification-of-a-personal-data-breach/> [^ccpa]: CCPA FAQ — CCPA rights + timelines — <https://coppa.ca.gov/faq.html> [^ccpa-thresh]: California AG — CCPA thresholds + service-provider status — <https://oag.ca.gov/privacy/ccpa> [^banner]: ePrivacy / GDPR cookie consent 2026 — https://www.consentio.com/knowledge-hub/GDPR/gdpr_cookie_consent_2026 [^reject]: “Reject All” must be as easy as “Accept All” (EDPB task force) — <https://techgdpr.com/blog/data-protection-digest-3062025-the-reject-all-button-is-a-must-legitimate-interest-as-the-data-controllers-initiative/> [^gpc]: Global Privacy Control 2026 — <https://www.didomi.io/blog/global-privacy-control-gpc-2026> [^nuxt]: Nuxt Scripts — consent management — <https://scripts.nuxt.com/docs/guides/consent> [^transfer]: European Commission — new Standard Contractual

Clauses Q&A — https://commission.europa.eu/law/law-topic/data-protection/international-dimension-data-protection/new-standard-contractual-clauses-questions-and-answers-overview_en

Payments, Billing & Financial Correctness

Source: [readme/research/engineering-ops/PAYMENTS_BILLING_FINANCIAL_CORRECTNESS.md](#)

IN SHORT — KEY TAKEAWAYS

This document is a detailed audit of every way the platform handles money — taking payments, issuing refunds, charging the right amount in the right currency, and collecting the correct tax. It matters because several serious bugs mean customers can be charged the wrong amount, refunds that appear to go through never actually move any money, and sensitive card details are stored in a way that breaks security rules.

The main parts explained simply:

- **Card data security (PCI-DSS)** — There are strict industry rules about how card numbers and the 3-digit security code (CVV) on the back of a card may be stored. Right now, the platform stores both in plain text in the database, which is a serious violation. The customer-facing checkout is safe (it redirects to Stripe), but an older admin payment path is not.
- **Refunds — the silent-lie bug** — When a store admin clicks “Refund” on an order, the screen says “Refund processed.” No money is actually returned to the customer. The button does nothing except show a success message. Only stores using one specific old payment provider ([Authorize.Net](#)) can issue real refunds at all.
- **Chargebacks (disputed payments)** — When a customer disputes a charge with their bank, the platform only changes a status label. The store owner is never notified, no deadline is surfaced, and orders in dispute keep moving to print. This is the most financially urgent problem.
- **Currency and charge amounts** — When a customer checks out, the platform charges in whatever currency the payment gateway happens to be configured for, not necessarily the currency of the order. For some currencies (like Japanese Yen), a hardcoded calculation also multiplies the amount by 100 too many times, causing a 100x overcharge.
- **Tax calculation** — The platform uses a single flat tax rate for all products in all locations. It does not account for different tax rules by region (US states, EU VAT, India GST) and incorrectly charges tax on products that are marked as tax-exempt when purchased through the storefront.
- **Hidden surcharge** — An older payment path silently adds a 3.5% fee on top of the invoice amount with no disclosure to the customer or the store owner.
- **Webhook reliability** — Webhooks are automated messages from the payment provider confirming what happened to a payment. The platform's webhook handling is mostly solid, but a gap means a webhook from one store could accidentally update the wrong store's records.

What to do with this: The two most urgent fixes are (1) stop storing card security codes (CVV) immediately — this is a legal violation — and (2) fix the refund button so it actually moves money. Everything else in the document is ordered by urgency in the action plan at the end.

The canonical reference for how Print-Flow-360 takes money, what the law/standards require, and where today's code falls short. **Last researched: 2026-06-15**

How to read this. Each topic section pairs *the external standard* (what correct looks like, with citeable facts) against *what the code does today* (with `file:line` citations preserved from the audit), then lists *gaps* (severity-tagged) and *recommendations* (ordered, pointing at the exact files to change). If you only have five minutes, read the Executive Summary risk table (§1) and the Silent-Lie Watchlist (§7). If you are about to touch money code, read the relevant topic section in full first. Severity tags: **[CRITICAL]** = active money/compliance loss or violation; **[HIGH]** = realistic wrong-money or audit-failure path; **[MEDIUM]** = drift/inconsistency that will bite; **[LOW]** = hygiene.

Disclaimer (read this). The PCI and tax content below is *engineering guidance*, not legal, tax, or compliance advice. A real PCI assessment requires a Qualified Security Assessor (QSA); a real tax position requires a qualified tax professional in each jurisdiction. Nothing here certifies compliance — it tells engineers what to build so a QSA/tax pro has a fighting chance of signing off.

1. Executive summary

Print-Flow-360 runs **two independent payment subsystems that share no code**: (1) a SaaS/landlord subscription-billing layer (`app/Services/Billing/*` , driven by `BillingService` + `SubscriptionGatewayRegistry` , reading landlord-DB config), and (2) a tenant storefront customer-payment layer (`app/Services/Integrations/Payment/*` , driven by `PaymentGatewayInterface` / `AbstractPaymentGateway` , a registry, `CheckoutPaymentService` , and a signature-verified webhook pipeline). The storefront layer is genuinely well-engineered. The damage lives in a **third, legacy `Authorize.Net` path** (`app/Services/PaymentService.php`) wired into admin order/invoice/quote/customer/tenant controllers, plus several gaps where the good pipeline stops short of finishing the money story.

Area	Current state	Risk level	Top gap
PCI-DSS scope	Storefront is correctly tokenized (Stripe Checkout redirect, SAQ-A); a parallel legacy <code>Authorize.Net</code> path accepts raw PAN+CVV over the API, stores full PAN + CVV in plaintext , and reveals them in the admin UI behind an eye-toggle	CRITICAL	CVV/PAN stored in plaintext (<code>PaymentService.php:269,272,181,184</code>); whole platform is realistically SAQ-D + a hard Req-3.2 violation
Webhooks / idempotency	Unified, signature-verified, two-phase idempotent pipeline keyed on (<code>gateway_key</code> , <code>event_id</code>); genuinely good	MEDIUM-HIGH	Webhook runs with tenancy NOT initialized → gateway-config + Order lookups run unscoped across all tenants; no dead-letter sweep; no state-machine guard
Refunds / chargebacks	Admin Orders refund modal is a pure silent-lie (fields dropped, “Refund processed” shown, no money moves); only working refund is legacy <code>Authorize.Net</code> -only; 5 gateway <code>processRefund()</code> impls are dead code; dispute handling is a status-flip stub	CRITICAL	A <code>non-Authorize.Net</code> store cannot refund at all ; chargebacks silently flip a flag nobody sees
Currency / rounding	Currency is now snapshotted onto documents (memory note stale); admin <code>formatMoney</code> is currency-aware	CRITICAL	Gateway charge currency read from gateway config, not the order (<code>CheckoutPaymentService.php:87</code>); hardcoded <code>*100</code> minor-unit conversion breaks JPY/KWD; <code>payments</code> table has no currency column
Sales tax / VAT / GST	Single flat per-store rate (<code>tax_enabled</code> + <code>tax_rate</code> + <code>prices_include_tax</code>), snapshotted per line; no <code>TaxService</code> , no destination/nexus logic	HIGH	No destination-based tax, no nexus/OSS/GST logic, no tax-exempt customers; storefront ignores per-product <code>is_taxable</code> and taxes everything

2. PCI-DSS scope

The standard / what correct looks like

- **Governing version:** PCI DSS **v4.0.1**, effective **1 April 2025** (superseded v4.0; v3.2.1 retired 31 March 2024). The 51 “future-dated” requirements lost their grace period on **31 March 2025** — as of 2026 every assessment runs against the complete v4.0.1 with no grace period.
- **Cheapest path = SAQ-A**, achievable only if the platform **never stores, processes, or transmits cardholder data** on its own systems. Qualifying patterns: redirect to the provider’s hosted page, OR an iframe/embedded form where **all** payment fields originate directly from the compliant provider (Stripe Elements/Checkout served from Stripe’s domain; Razorpay hosted Checkout). Both Stripe and Razorpay are PCI **Level 1** providers.
- **The dividing line:** if any merchant-served code can touch the card fields, you fall out of SAQ-A into **SAQ-A-EP** (~191 reqs) or **SAQ-D** (300+ reqs). Posting raw PAN to a gateway’s server-to-server API ⇒ SAQ-D.

- **Prohibited storage (Req 3.3, formerly 3.2):** Sensitive Authentication Data (SAD) — **full track data, CVV/CVC/CID, and PINs/PIN blocks** — **must NEVER be stored after authorization, regardless of encryption.** CVV may never be persisted to disk, log, cache, or DB for any reason once authorized.
- **Allowed storage (Req 3.4/3.5):** provider **token**, **masked PAN** (first6 + last4), **brand**, **expiry**. Full PAN only if rendered unreadable (hash/truncation/tokenization/strong crypto) and displayed masked to ≤ first6+last4, full visible only on documented business need. Best practice and the SAQ-A model: store **only the token + last4 + brand + expiry**, never the full PAN.
- **The 2025 iframe trap (FAQ 1588):** v4.0.1 added two SAQ-A eligibility conditions — (1) all payment-page elements originate only/directly from a compliant TPSP; (2) the merchant confirms its site is not susceptible to script attacks. Condition (2) invokes **Req 6.4.3** (payment-page script inventory + integrity/authorization) and **Req 11.6.1** (payment-page + security-header tamper detection, evaluated ≥ weekly) — and **applies to embedded iframe forms** (Elements/Razorpay modal), NOT to pure redirect/hosted-page integrations. Satisfy it either by self-implementing 6.4.3/11.6.1 (SRI/CSP/monitoring) **or** by retaining the provider's written confirmation that their embedded solution protects against script attacks.
- **Always:** TLS on all payment pages; no card data in logs/Sentry/queue payloads; SAQ-A is still an annual self-assessment + AOC per acquirer (Stripe auto-generates a combined SAQ-A+AOC when Checkout/Elements is the sole collector).

What Print-Flow-360 does today

- **Storefront path is correct (SAQ-A).** `StorefrontCheckoutController` → `CheckoutPaymentService::createPaymentSession` (`app/Services/Storefront/Checkout/CheckoutPaymentService.php:38`) → `StripeGateway::createCheckoutSession` (`StripeGateway.php:134`) returns `PaymentSessionResult::redirect($session->url)`; `getRenderMode()` is `'redirect'` (`StripeGateway.php:113`). The storefront collects **no** card fields — `CheckoutPayment.vue:126` only shows a method selector and “You'll be securely redirected”; the embedded `StripeElementsDriver.vue:1-15` is an explicit unimplemented stub. A repo-wide grep of `frontstore/app` found no `card_number` / `cvv` / `exp_month`. Laravel stores only `gateway_reference` / `transaction_id` on the `Payment` row.
- **The legacy Authorize.Net path is non-compliant.** `PaymentService::addPaymentMethod` (`app/Services/PaymentService.php:221`) and `addPaymentMethodToTenant` (`:120`) write raw card data to the DB:
 - `PaymentProfile` created with `'card_last_four' => $cardNumber` (the **full** number into a column named `last_four`, `PaymentService.php:269`), `'expiration_date'`, and `'cvv' => $cardCode` (`:272`).
 - `addPaymentMethodToTenant` writes `'card_number' => $cardNumber` and `'cvv' => $cardCode` to `tenant_payment_profiles` (`:181`, `:184`).
 - Schema confirms plaintext: `2025_04_09_173425_add_cvv_column_in_payment_profiles_table.php` adds an unencrypted integer `cvv`; `create_payment_profiles_table.php` stores card fields as plain strings; `TenantPaymentProfile.php:16-27` lists `card_number` / `last_four` in `$fillable`. **No model anywhere uses an `encrypted` cast** (grep for `encrypted` returns nothing).
- **Raw PAN+CVV are POSTed to Laravel from many endpoints** and echoed back to the browser: `PaymentProfileController::store` (`:94-116`), Admin `TenantController` (`:827-841`),

`OrderController ( :1219 )`, `InvoiceController ( :931-954 )`, `QuoteController ( :944-947 )`,
`PaymentController::store ( :140-143, :356 )`. The admin UI
`nuxt/app/components/customer/payment/profile.vue:25-43` renders
`card_number / cvv / expiration_date` behind an eye/eye-slash reveal toggle.

- **All gateway secrets are stored in plaintext.** `StoreThirdPartyService` casts `configuration` only as `'array'` — no `encrypted:array ( StoreThirdPartyService.php:30-34 )`; the billing layer's `Admin\ThirdPartySetting` casts `attributes` as plain `'array'`. So `stripe_secret`, `key_secret`, `webhook_secret`, `transaction_key` sit unencrypted in JSON columns.

Gaps & risks

- **[CRITICAL]** CVV/CVV2 stored in the DB (`payment_profiles.cvv` int column; `tenant_payment_profiles.cvv`) — an absolute Req 3.2/3.3 prohibition, encryption-irrelevant.
- **[CRITICAL]** Full PAN stored unencrypted — the full number is written into `card_last_four ( PaymentService.php:269 )` and into `tenant_payment_profiles.card_number ( :181 )`; violates Req 3.4.
- **[CRITICAL]** Multiple Laravel endpoints accept raw PAN+CVV in request bodies, forcing the whole app into SAQ-D and likely failing a QSA audit.
- **[CRITICAL]** All gateway API secrets (live keys, webhook secrets) stored plaintext in JSON config columns — a DB read/backup leak exposes every tenant's live payment credentials.
- **[HIGH]** Stored card data is reflected back to the admin browser behind a reveal toggle (`profile.vue:25-43`) — insider-threat / exfiltration-by-design on top of the storage violation.
- **[HIGH]** Storing full PAN (even if it were encrypted) keeps the platform in cardholder-data scope; the SAQ-A model is token+last4+brand+expiry only.
- **[MEDIUM]** Even the “new” `AuthorizeNetGateway` adapter passes raw `card_number / cvv` arrays server-side (`AuthorizeNetGateway.php:48-51` ; `AuthorizeNetBillingGateway.php:52-54`) rather than using Accept.js tokens — so the registry path can also handle raw card data.
- **[MEDIUM]** Multi-tenant misconfig: if any tenant can be configured with a non-hosted collector, that one tenant breaks the platform's SAQ-A eligibility. Eligibility must be enforced at the platform layer.
- **[MEDIUM]** The storefront custom-field / dynamic-form engine could let a store owner add a field that captures card numbers — reserved-name + field-type controls must prevent SAD capture through custom fields.
- **[LOW]** Embedded-iframe checkout (if Elements is ever enabled) triggers the 6.4.3/11.6.1 script-inventory + tamper-detection obligation; pure redirect avoids it. SAQ-A still needs annual attestation + TLS regardless of the gateway being Level 1.

Recommendations (ordered)

1. **Stop collecting CVV everywhere, now.** Remove `cvv` from every `FormRequest` (`PaymentProfileController`, `TenantController`, `OrderController`, `InvoiceController`, `QuoteController`, `PaymentController`). Drop the `cvv` columns (`payment_profiles`, `tenant_payment_profiles`) in a new migration and **purge existing rows**. Nothing in PCI permits storing it.
2. **Stop storing full PAN.** Fix `PaymentService.php:269` (`card_last_four` must receive only `substr($cardNumber, -4)`) and `:181` (`tenant_payment_profiles.card_number` should not exist — store only `last_four`). Migrate the column out and purge.

3. **Tokenize the `Authorize.Net` path** with `Accept.js` / `Accept Hosted` so raw PAN never reaches Laravel; route `PaymentProfileController` /admin profile creation through tokens. This is what moves the admin path back toward SAQ-A.
4. **Remove the card-reveal UI** in `profile.vue:25-43`; display only masked last4 + brand + expiry.
5. **Encrypt gateway secrets at rest.** Change `StoreThirdPartyService` `configuration` cast to `encrypted:array` (`StoreThirdPartyService.php:30-34`) and `Admin\ThirdPartySetting` `attributes` to `encrypted:array`; provide a one-time re-save/migration command.
6. **Scrub observability:** verify Laravel request logs, Sentry breadcrumbs, queue payloads, and the `*_snapshot` JSON columns never capture card fields.
7. **Lock the custom-field engine** to reject card-number-shaped field names/types.
8. **Decide redirect-vs-iframe deliberately.** Keep the storefront on **redirect** (lowest burden); if Elements is ever turned on, stand up the 6.4.3 script inventory + 11.6.1 tamper detection or hold Stripe's written attestation.

3. Webhook idempotency & failure handling

The standard / what correct looks like

- **Webhook is the source of truth**, not the synchronous API response — card refunds and disputes resolve asynchronously (Stripe: card refunds can fail/re-credit up to 30 days later; Razorpay: `refund.processed` is the definitive final state).
- **Signature-verify on the raw body** with the provider SDK/HMAC (Stripe `constructEvent` validates HMAC + timestamp tolerance; Razorpay HMAC-SHA256 with timing-safe compare). Fail closed on missing/empty secret.
- **Idempotency:** dedupe on the gateway event id (Stripe `event.id`, Razorpay event id) with a DB unique constraint; a redelivered event records once. Amount-mutating ledger operations must be idempotent (don't double-add a refund).
- **Order/state safety:** derive document status from cumulative amounts, not “any event fired”; guard against out-of-order delivery (a late `failed` after `captured` must not regress a paid order).
- **Failure handling:** providers retry on non-2xx; surface a 5xx to trigger retry, but also run a **dead-letter / reconciliation sweep** for events that exhaust the provider's finite retry window. Continuous transaction-level reconciliation against bank settlement.

What Print-Flow-360 does today

- **Single gateway-agnostic endpoint** `POST /api/v1/webhooks/payments/{gateway_key}` (`routes/public-api.php:32`), correctly excluded from auth/CSRF (`bootstrap/app.php:42-43`), → `PaymentWebhookController.php:23` → `PaymentEventDispatcher::handle` (`PaymentEventDispatcher.php:62`).
- **Signature verification is real** and runs once at `PaymentEventDispatcher.php:96` via each gateway's `verifyWebhook()`:

- Stripe `StripeGateway.php:180-198` reads `webhook_secret`, requires `Stripe-Signature`, calls `\Stripe\Webhook::constructEvent($request->getContent(), ...)` on the **raw** body; maps `SignatureVerificationException` → `WebhookVerificationException`.
- Razorpay `RazorpayGateway.php:346-361` requires `X-Razorpay-Signature`, computes `hash_hmac('sha256', $request->getContent(), $secret)` and compares with `hash_equals()`.
- Missing/empty secret throws (fail-closed). Gateways without an override inherit `AbstractPaymentGateway.php:53` which throws `LogicException` — so **PayPal/Paytm/AuthorizeNet have no inbound signature verification implemented yet**. A verification failure → message `signature_invalid` → HTTP 401 (`PaymentWebhookController.php:49`).
- **Idempotency is genuine and two-phase.** Unique index `payment_webhook_events_dedup_unique` on `(gateway_key, event_id)` (`2026_05_02_000001_update_payments_system.php:66`). `processVerifiedEvent` (`PaymentEventDispatcher.php:130-266`): **Phase 1** (`:144-181`) opens a txn, `SELECT ... lockForUpdate()` (`:153`); if the row exists with `processed_at` non-null → status `duplicate` → HTTP 200; else upsert/reuse the row (incrementing `attempts` at `:170`) and commit only the audit row. **Phase 2** (`:209-265`) dispatches the domain event **outside** the dedup txn; on success sets `processed_at=now()`. A **second** idempotency layer is in the listener: `UpdatePaymentAndOrder.php:128-147` uses `Payment::firstOrCreate` keyed on `(gateway_key, idempotency_key=event_id)`, backed by `payments_gateway_idem_unique` (`migration:22`).
- **Webhook-vs-redirect race handled by design:** the redirect-return path (`RazorpayGateway::verifyReturnParams`, `:422-451`) produces the same `VerifiedPaymentEvent` and shares `processVerifiedEvent`; whichever arrives first claims the slot, the second short-circuits as `duplicate`.
- **Failure handling:** on a listener exception, `PaymentEventDispatcher.php:241-265` logs, writes `$row->error`, and **deliberately leaves** `processed_at=null` so the next gateway redelivery re-runs it (`:250-251`); surfaces HTTP 500 to signal retry. Unknown event types (`payment.unknown` or not in `EVENT_MAP`) are stamped `processed + ignored_unknown` + HTTP 200 (`:210-229`) so the gateway stops retrying noise. Customer-email side-effects are isolated in their own try/catch (`UpdatePaymentAndOrder.php:78-87`).

Gaps & risks

- **[HIGH] Cross-tenant gateway-config resolution.** The webhook route runs with **no tenancy middleware**, and `StoreThirdPartyService` uses `BelongsToTenant` (`StoreThirdPartyService.php:14`) whose `TenantScope` is a no-op when tenancy is uninitialized. So the config lookup at `PaymentEventDispatcher.php:69-73` runs **unscoped across all tenants** and returns the first matching active config (`orderByRaw` at `:72` only sorts account-level first). With two tenants on the same gateway key, the webhook can resolve the **wrong** tenant's secret and stamp the event row with the wrong `tenant_id` (`:162-163`).
- **[MEDIUM] No webhook-to-order tenant consistency check.** `resolveOrder` (`UpdatePaymentAndOrder.php:96-121`) and `resolveOrderId` (`PaymentEventDispatcher.php:277,282`) find Orders globally with no assertion that `order.tenant_id` matches the resolved gateway-config tenant. A validly-signed payload from tenant A referencing tenant B's order id could mutate the wrong tenant's order.
- **[MEDIUM] No state-machine guard.** `UpdatePaymentAndOrder.php:58-65` applies any event unconditionally; an out-of-order/late delivery (stale `failed` after `captured`) can regress a PAID order.

`applyRefunded` (:179) is non-idempotent in amount terms — safe only because the event-row dedup blocks same- `event_id` re-runs.

- **[MEDIUM] No dead-letter / reconciliation sweep.** Permanently-failing events stay `processed_at=null` + `error` and depend entirely on the gateway's finite retry window; once exhausted, the payment is silently never reconciled. `PaymentGatewayLogService` is not invoked in the webhook path at all.
- **[LOW] No rate limiting** on the webhook route (`routes/public-api.php:32` has no `throttle`). Signature verification gates abuse, but unsigned garbage still hits DB driver-resolution first.
- **[LOW] PayPal/Paytm/AuthorizeNet** have no `verifyWebhook` override — fail-closed (good) but not wired for inbound webhooks; do not enable those keys for live webhooks until implemented.

Recommendations (ordered)

1. **Re-scope the webhook to a tenant.** After signature verification, resolve the tenant from the verified payload's order/account, `tenancy()->initialize($tenant)`, and **re-run** the config + Order lookups scoped — or enforce exactly one global account-level config per gateway key. Fixes the cross-tenant config and order risks together. (`PaymentEventDispatcher.php:69-73`, :277-282)
2. **Assert tenant consistency** in `resolveOrder` / `findOrCreatePayment` : the resolved Order's `tenant_id` must equal the resolved gateway-config's `tenant_id`, else reject. (`UpdatePaymentAndOrder.php:96-121`)
3. **Add a lifecycle state-machine guard** in `UpdatePaymentAndOrder.php:58-65` : never regress `PAID` → `FAILED` ; derive `payment_status` from cumulative captured/refunded amounts, not from the bare event type.
4. **Add a scheduled reconciliation/dead-letter sweep** over `payment_webhook_events` where `processed_at IS NULL AND error IS NOT NULL` past the provider's retry window; alert + allow manual replay. Wire `PaymentGatewayLogService` into the webhook path for auditable history.
5. **Add `throttle` middleware** to the webhook route and reject obviously-malformed bodies before DB resolution.
6. Implement `verifyWebhook` for PayPal/Paytm/AuthorizeNet before enabling their live webhook keys.

4. Refunds & chargebacks

The standard / what correct looks like

- **Stripe & Razorpay both support full + multiple partial refunds**; cumulative refunds may not exceed the captured amount (the gateway also enforces this). Refunds always return to the original method.
- **Refunds are asynchronous.** Stripe Refund status: `requires_action` → `pending` → `succeeded` / `failed` / `canceled` (a card refund can `fail` and re-credit the merchant up to **30 days** later). Razorpay: `created` → `processed` / `failed` (`refund.processed` is the definitive final state; a payment stays `captured` until **fully** refunded, then becomes `refunded`).
- **Webhook is source of truth.** Stripe: `refund.created/updated/failed` (fire for all refunds since the 2024-10-28 Acacia change) + `charge.refunded` . Razorpay: `refund.created/processed/failed/speed_changed` . Do **not** decrement invoice `paid_amount` or flip order status on the synchronous create response — wait for the success webhook.

- **Idempotency on refund creation** prevents double-refunds: Stripe `Idempotency-Key` header ($\geq 24h$ retention, param-mismatch errors); Razorpay idempotent refund endpoints. Before creating: check `cumulative_refunded + new ≤ captured`. Persist + unique-constrain the gateway refund id so redelivered webhooks don't double-count.
- **Fees are NOT recovered on refunds** (both providers retain the original processing fee); a ledger assuming 100% return mis-states P&L.
- **Disputes/chargebacks debit funds immediately.** Stripe debits the disputed amount + a **non-refundable dispute received fee** at `charge.dispute.created`; the matching “countered” fee (since 17 Jun 2025) is returned only on a win; `charge.dispute.funds_reinstated` on a win. Razorpay temporarily debits `amount_deducted` at `payment.dispute.created`.
- **Short, network-driven response window** (Stripe 7–21 days; Razorpay `respond_by` Unix timestamp). Miss it and you auto-lose. Drive an internal SLA + reminders off the deadline. Win on documentary evidence: delivery/tracking proof, signed invoice, customer comms, your refund/cancellation policy, AVS/CVC, IP/device.
- **Status from cumulative amounts:** a partially-refunded order is *partially* refunded, not fully.

What Print-Flow-360 does today

- **The admin Orders refund modal is a pure silent-lie.** `nuxt/app/pages/orders/[id].vue:281-290` (`saveRefund`) sends `refund_amount/refund_reason/refund_note/notify_customer_refund` via `updateOrder()` → `PUT /orders/{id}` (`useOrdersApi.ts:31-42`). `OrderController::update` (`OrderController.php:618-650`) validates a whitelist that includes **none** of the refund fields, builds `$payload` from `$validated` only (`:656`) then `$order->fill($payload)` (`:701`). A grep for `refund_amount / refund_reason / refund_note / notify_customer_refund` across `app/` and `database/` returns **zero** hits. So all four fields are silently discarded — no gateway call, no Payment row, no status change, no audit, no notification — yet the UI unconditionally shows `showSuccess('Refund processed')` on a 200 (`orders/[id].vue:288`).
- **The only working refund is legacy + Authorize.Net-only.** `PaymentController::refund` (`PaymentController.php:478-567`, route `routes/api.php:448` `payments/{id}/refund`) hardcodes the legacy `App\Services\PaymentService` (`Authorize.Net`), constructed from `tenant->authorize_api_key/authorize_transaction_key` (`:480-489`), aborting 400 if unset. It supports partial refunds (rejects `amount > original` at `:502-504`; flips Invoice → REFUNDED only when full at `:525`), guards eligibility (`credit_card + PAID` at `:498`), requires a stored profile (`:506-511`). `PaymentService::processRefund` (`PaymentService.php:496-516`) refuses pre-settlement refunds (the void branch is commented out at `:506`). **This path is not reachable from the Orders UI.**
- **Five gateway `processRefund()` impls are dead code.** Stripe (`StripeGateway.php:77-95`, caps `refunds + partial_refunds` `true` `:125-133`), PayPal (`PayPalGateway.php:164-235`), Razorpay (`RazorpayGateway.php:195-262`), Paytm (`PaytmGateway.php:289-361`), AuthorizeNet (`AuthorizeNetGateway.php:66-91`). A grep for `->processRefund()` callers finds the **only** non-self caller is `PaymentController.php:512` calling the legacy `PaymentService` — **never** the gateway registry. So Stripe/PayPal/Razorpay/Paytm refund code is unreachable and the advertised `partial_refunds` capability is honored by no UI/endpoint.
- **Refund webhook is idempotent + additive but disconnected.** `UpdatePaymentAndOrder.php:128-147` keys Payment on `(gateway_key, event_id)` (idempotent on redelivery); `applyRefunded` (`:176-181`) sets

- `payment->status=REFUNDED` and **accumulates** `refunded_amount += amount`, sets `order->payment_status=REFUNDED`, in a DB txn (:55-74). **But:** (a) a partial-refund webhook always flips order/payment to full `REFUNDED` and never uses the existing `PaymentStatusEnum::REFUNDED_PARTIAL` (`PaymentStatusEnum.php:38`) despite accumulating a partial amount — a \$10 refund on a \$100 order is mislabeled fully refunded; (b) the manual `PaymentController::refund` path creates a separate `refund` -type Payment not keyed on any webhook event_id, so a later `charge.refunded` webhook creates **yet another** Payment row (no cross-path dedup); © the Orders-UI path writes nothing.
- **Chargeback/dispute is a status-flip stub.** `PaymentDisputed` (`app/Events/Payment/PaymentDisputed.php`) is an empty subclass. Stripe `charge.dispute.created` (`StripeGateway.php:204`) and PayPal `CUSTOMER.DISPUTE.*` (`PayPalGateway.php:382`) → `payment.disputed` → `PaymentDisputed` (`PaymentEventDispatcher.php:41`) → `applyDisputed` (`UpdatePaymentAndOrder.php:183-187`) which does **only** `payment->status=DISPUTED` and `order->payment_status=DISPUTED`. There is **no store-owner notification** (`NotificationTypeEnum` has no dispute/chargeback type — only `INVOICE_REFUNDED` / `ORDER_REFUNDED` at :13,:31), **no order/production freeze, no accounting adjustment, no audit log, and no `respond_by` deadline surfaced.**
 - **Audit logging exists only on the legacy refund path.** `PaymentController::refund` logs `payment_refunded` and `refund_created` via `AuditLogger` (`PaymentController.php:551-559`). The webhook refund/dispute path writes **no** `AuditLogger` entry (only stamps the `PaymentWebhookEvent` row). `Order::markAsPaid` writes `OrderHistory` (`Order.php:190-202`) but there is no `markAsRefunded` / `markAsDisputed`.

Gaps & risks

- **[CRITICAL]** Admin Orders refund modal silently discards all refund fields yet shows “Refund processed”. Money is NOT returned but staff believe it was — the worst bug class per [CLAUDE.md](#).
- **[CRITICAL]** Chargeback/dispute handling is a stub: only flips status. No store-owner alert (no dispute `NotificationType` exists), no `respond_by` SLA, no production freeze, no accounting, no audit. The most financially-urgent event silently flips a flag nobody sees, and disputed orders keep flowing to production.
- **[HIGH]** Gateway `processRefund()` for Stripe/PayPal/Razorpay/Paytm + advertised `partial_refunds` are dead code. **Any store not on Authorize.Net cannot issue a refund through the product at all.**
- **[HIGH]** Partial refund via webhook sets status to full `REFUNDED` and never uses `REFUNDED_PARTIAL` despite accumulating `refunded_amount` — corrupts reporting/accounting.
- **[MEDIUM]** No idempotency link between the manual refund and the gateway refund webhook → a later `charge.refunded` creates a duplicate Payment row (double-counts `refunded_amount`).
- **[MEDIUM]** `PaymentService::processRefund` refuses pre-settlement refunds (void branch commented out, :506) — admins can’t void/refund a same-day unsettled charge; they hit a dead-end error with no void fallback.
- **[MEDIUM]** Ledger assumes refunds/disputes are final on the synchronous response; per the standard, a refund can fail/reverse up to 30 days later and a dispute debit reverses on a win — wait for webhooks and record the immediate dispute debit + reinstatement, or balance reconciliation drifts.

Recommendations (ordered)

1. **Wire the Orders refund modal to a real refund service.** Create a `RefundService` (controller stays thin) that: resolves the gateway via `PaymentGatewayRegistry`, calls `processRefund()` with an **idempotency**

- key**, records a pending refund Payment, and only finalizes on the success webhook. Add the refund fields to a proper FormRequest. (`OrderController.php:618-650` , `orders/[id].vue:281-290`)
2. **Route refunds through the gateway abstraction**, not the legacy `PaymentService` , so Stripe/Razorpay/PayPal/Paytm stores can refund. Keep `Authorize.Net` working via its adapter.
 3. **Honor partial refunds:** in `applyRefunded` (`UpdatePaymentAndOrder.php:176-181`), set `REFUNDED_PARTIAL` when `cumulative_refunded < captured` , `REFUNDED` only when equal; derive order status from cumulative amounts.
 4. **Treat the webhook as source of truth:** do not decrement invoice paid_amount / flip order status on the synchronous refund-create response; record an intent/pending entry and finalize on `refund.updated--succeeded` (Stripe) / `refund.processed` (Razorpay).
 5. **Cross-path dedup:** when the manual refund later receives a `charge.refunded` webhook, match it to the existing refund Payment (e.g. store the gateway refund id on the manual row) rather than creating a duplicate.
 6. **Build real chargeback handling:** add a `CHARGEBACK / DISPUTE NotificationType` , alert the store owner immediately with the `respond_by` /Stripe due-date countdown, freeze the order's production/print jobs, record the immediate dispute debit (+ fee) and the reinstatement on a win, and write an `AuditLogger` entry. Surface a "respond by" SLA in the admin UI. (`PaymentDisputed` , `UpdatePaymentAndOrder.php:183-187` , `NotificationTypeEnum.php`)
 7. **Add `markAsRefunded / markAsDisputed`** to `Order` writing `OrderHistory` , and emit `AuditLogger` from the webhook refund/dispute path (satisfies the `CLAUDE.md` audit invariant).
 8. **Add a void fallback** for pre-settlement cancellations instead of the dead-end error (`PaymentService.php:506`).
-

5. Currency & rounding

The standard / what correct looks like

- Charge the customer in the **document's** currency, validated against what the gateway supports — never silently substitute a config default.
- **Minor-unit conversion is currency-specific:** most currencies have 2 decimals (×100), but **zero-decimal** currencies (JPY, KRW) pass the integer amount as-is and **three-decimal** currencies (KWD, BHD) use ×1000. A hardcoded ×100 over/undercharges by 100×/10×.
- Persist the **actual captured currency** on the payment record for reconciliation.
- Snapshot the display currency at document creation so a later store-currency change doesn't rewrite history.
- Prefer integer minor units or `bcmath` /decimal arithmetic over float; round consistently to the currency's decimal places.

What Print-Flow-360 does today

- **The memory note is stale:** currency is now snapshotted onto documents.
`2026_06_04_0000001_add_currency_snapshot_to_documents.php` adds nullable `currency_code/currency_symbol/currency_decimals` to orders/quotes/invoices, populated in `creating()` hooks (`Quote.php:45-53` , `Order.php:61-69` , `Invoice.php:46-54` — Invoice copies from its

source Quote), backfilled at read time (`OrderController.php:223-225, :370-373`). Source of truth: `store_settings.default_currency_id → currencies` , resolved via `CurrencyHelper.php:22` and `StorefrontCurrencyService.php:39` .

- Admin `formatMoney` is currency-aware (`nuxt/app/utils/formatMoney.ts:17` takes `(value, symbol='$', decimals=2)` ; `useStoreCurrency.ts:17`). Detail pages pass the snapshot (`formatMoney(invoice.total_amount, invoice.currency_symbol, invoice.currency_decimals)`). But many call-sites pass only the value and fall back to `$ : plan.monthly_charge` , `row.base_price` , `rate.rate` (shipping), `tx.amount` (billing HistoryTabs), `t.unit_price` .
- **CRITICAL: the gateway charge currency comes from the gateway config, not the order.** In `CheckoutPaymentService.php:86-101` , the amount is the order grand total (in store currency) but `currency = serviceRow->configuration['currency'] ?? 'USD'` — read from the **payment-gateway config row**, never from `$order->currency_code` . No comparison between order and gateway currency; `supportsCurrency()` (default true, `AbstractPaymentGateway.php`) is never called in checkout.
- **Hardcoded ×100** at `StripeGateway.php:45, :142` and `RazorpayGateway.php:50, :316` (`(int) round($amount * 100)`) — wrong for non-2-decimal currencies.
- The **payments** table has no currency column (`2025_02_05_185503_create_payments_table.php` , `2026_05_02_000001_update_payments_system.php`); `Payment.php` has no currency field. The gateway-returned captured currency (`StripeGateway.php:220`) is **discarded** rather than stored/compared.
- **Totals re-round to a hardcoded 2 decimals** ignoring `currency_decimals` : `Quote::getTotalAmountAttribute()` (`Quote.php:219, :231`) does `round(sum(line+tax), 2)` . Amounts are `DECIMAL(10-14,2)` floats; the pricing engine rounds to 4dp (`ProductPricingCalculator.php:209-210`) then the model re-rounds to 2dp; no `bcmath` anywhere.

Gaps & risks

- **[CRITICAL]** Gateway charge currency read from config (`CheckoutPaymentService.php:87` , default `'USD'`), never from `$order->currency_code` , and `supportsCurrency()` never checked. A store totalling in currency X with a gateway configured for Y charges the X-amount **labelled** as Y — a silent wrong-currency charge with no validation.
- **[CRITICAL]** Hardcoded ×100 (`StripeGateway.php:45, :142` ; `RazorpayGateway.php:50, :316`) ignores decimal places — JPY/KRW overcharged 100×, KWD/BHD undercharged 10×.
- **[HIGH]** `payments` table has no currency column and the webhook-parsed captured currency is discarded — the wrong-currency bugs above are **undetectable** from stored data; no reconciliation possible.
- **[MEDIUM]** Total accessors round to hardcoded 2 decimals (`Quote.php:231`) ignoring `currency_decimals` — mis-rounds 0-/3-decimal currencies before they ever reach the gateway.
- **[MEDIUM]** Many admin `formatMoney()` call-sites omit symbol/decimals and fall back to `$ /2` — wrong display for non-USD stores.
- **[LOW]** All money math is float-based; calculator → line → total → payment rounding can drift a cent on large multi-item orders.

Recommendations (ordered)

1. **Charge in the order's currency.** In `CheckoutPaymentService.php:86-101` , use `$order->currency_code` (not the config default); call `$gateway->supportsCurrency($order->currency_code)` and throw a clear

- `PaymentSessionException` if unsupported (matches the existing “never silently degrade” pattern).
2. **Currency-decimal-aware minor units.** Add a helper that converts to minor units using the currency’s `decimal_places` (0/2/3) and replace the hardcoded $\times 100$ at `StripeGateway.php:45, :142` and `RazorpayGateway.php:50, :316`.
 3. **Add a `currency` column to `payments`** and persist both the charged currency and the gateway-returned captured currency (`StripeGateway.php:220`); reconcile against `order.currency_code` in the webhook.
 4. **Round totals to `currency_decimals`** in `Quote` / `Order` / `Invoice` total accessors (`Quote.php:231`), not hardcoded 2.
 5. **Sweep admin `formatMoney` call-sites** (`plan.monthly_charge` , `row.base_price` , shipping `rate.rate` , billing `tx.amount` , `t.unit_price`) to pass the snapshot symbol/decimals.
 6. **(Later)** Move money math to integer minor units / `bcmath` to eliminate float drift.
-

6. Sales tax / VAT / GST

The standard / what correct looks like

- **Tax is destination-based** — compute at checkout from the validated **ship-to** address (place-of-supply in India). There is **no single national rate** in the US.
- **US (post-Wayfair):** per-state economic nexus, commonly **\$100k** sales (some states `$100k OR 200 transactions` , the 200-count being dropped — ~16 states gone by 1 Jan 2026); CA/TX **\$500k**, NY **\$500k AND 100 txns**. Marketplace-facilitator rules may shift collect/remit to the platform. **Custom printing is generally taxable** tangible personal property (full retail price including labor/materials/delivery).
- **EU:** destination VAT for B2C; **€10k** micro-threshold then **OSS** (single quarterly return); **IOSS** for imports $\leq \text{€}150$; **reverse charge** for B2B (validate VAT no. via VIES). ViDA (adopted 11 Mar 2025) phases changes 2027–2028 (mandatory B2B reverse charge for non-established suppliers; planned abolition of the €150 IOSS threshold).
- **UK:** 20%; £90k threshold for UK businesses but **no threshold for overseas sellers** (register from first B2C sale); imports $\leq \text{£}135$ collected at point of sale.
- **India:** CGST+SGST (intra-state) vs IGST (inter-state) by **place-of-supply**; GSTIN + HSN on invoices; B2B e-invoicing (IRP/IRN/QR) above turnover thresholds.
- **Canada:** 5% GST + provincial HST/PST/QST; CAD \$30k small-supplier threshold; destination by province.
- **Australia:** 10% GST; A\$75k threshold; low-value-imports collected at sale.
- **Display mode is region-specific: tax-inclusive** for EU/UK/AU consumers (legally required), **tax-exclusive** for US/Canada. A single global display setting is non-compliant.
- **Snapshot** the computed tax (rate, jurisdiction, amount, taxable base, registration number) onto the invoice/order at sale time. **Use a tax engine** (Stripe Tax / Avalara / TaxJar) for nexus tracking + jurisdiction rates rather than hand-coding. Distinguish **B2B vs B2C** for reverse-charge + display.

What Print-Flow-360 does today

- **No `TaxService` exists** anywhere in `app/` . Tax is three `store_settings` columns: `tax_enabled` , `tax_rate` (`decimal(5,2)`), `prices_include_tax` , plus display-only `tax_label` / `tax_id`

(2025_12_21_000002_create_store_core_tables.php:40-46).

- **Flat per-store rate, snapshotted per line.** `StorefrontCheckoutController` reads `tax_enabled/tax_rate/prices_include_tax` (:364-366) and computes per line inclusive (`amount - amount/(1+rate/100)` , :377) or exclusive (`amount*rate/100` , :380); same for quotes (:769-817) and `StorefrontCartService.php:50-51` . Computed tax is snapshotted per line as `tax / tax_percentage / is_taxable` ; the rate is stored on the parent (`orders.tax_rate` float + `orders.prices_include_tax`). Admin-created docs respect per-item `is_taxable + tax_percentage` (`OrderController.php:881` , `QuoteController.php:512` , `InvoiceController.php:547`). So tax **is** history-safe — but it's a single flat rate with no region/nexus/VAT/GST logic and no tax-exempt-customer mechanism.
- **Storefront ignores per-product `is_taxable`** . Products carry `is_taxable` (`2025_04_12_103533_add_is_taxable_in_items.php` , `ProductResource.php:22`), and admin honors it per item — but storefront checkout hardcodes `'is_taxable' => $taxEnabled` (`StorefrontCheckoutController.php:394`), taxing **all** items whenever tax is enabled. A store with exempt printed matter over-charges through storefront while admin charges correctly — divergent results for the same product by entry path.
- **No persisted order-level `tax_amount`** . `orders` has `tax_rate + prices_include_tax` but no `tax_amount / total_tax` column; `Order::getTotalAmountAttribute` (`Order.php:265-288`) recomputes from per-line `item.tax` snapshots at read time (`StorefrontCheckoutController.php:1024` references `$order->total_tax ?? null`). History-safe via line snapshots, but there's no single authoritative stored tax-total to reconcile against.

Gaps & risks

- **[HIGH] Merchant-of-record / marketplace-facilitator classification is undecided** — determines whether the platform or each store bears collect/remit liability across all US states (and EU/UK/AU deemed-supplier rules). A legal/tax determination that must precede building tax logic.
- **[HIGH]** Single flat store rate, no destination/region/nexus support — systematically wrong tax (under/over-collected) for any multi-jurisdiction, US state/local, EU VAT, or India GST obligation; no tax-exempt-customer mechanism.
- **[HIGH]** Storefront ignores per-product `is_taxable` (`StorefrontCheckoutController.php:394`) — storefront taxes exempt products that the admin path correctly exempts.
- **[MEDIUM]** Tax-display mode is not region-aware — showing tax-exclusive to EU/UK/AU consumers violates price-indication rules; a single global setting is non-compliant.
- **[MEDIUM]** India place-of-supply (CGST+SGST vs IGST) and B2B e-invoicing (IRP/IRN/QR + HSN + both GSTINs) — a naive “add 18%” produces non-compliant Indian invoices.
- **[MEDIUM]** Moving regulatory targets (ViDA 2027–2028; US dropping the 200-txn count) — hardcoded rates/thresholds will rot; use a maintained engine.
- **[LOW]** Overseas-seller zero-threshold traps (UK/EU non-established B2C) — “wait until we hit the limit” logic is wrong cross-border.
- **[LOW]** No persisted order-level `tax_amount` ; a future change to the total accessor or a missing line snapshot could silently change historical tax with no stored ground-truth.

Recommendations (ordered)

1. **Resolve the merchant-of-record / facilitator question** (legal/tax) before building — it dictates who registers and remits.
 2. **Honor per-product `is_taxable` in storefront checkout** now
(`StorefrontCheckoutController.php:394`): set each line's `is_taxable` from the product, not the store-wide flag — closes the admin ↔ storefront divergence and is a small, high-value fix.
 3. **Introduce a `TaxService`** (controllers stay thin) as the single tax-computation seam; have storefront + admin + cart all call it, so there's one place to evolve.
 4. **Integrate a tax engine** (Stripe Tax / Avalara / TaxJar) behind that service for destination rates + nexus/threshold tracking; avoid baking jurisdiction tables into code.
 5. **Snapshot the full tax detail** (rate, jurisdiction, amount, taxable base, registration number) onto orders/quotes/invoices, and **add a persisted `tax_amount`** column for reconciliation. Prove the round-trip (compute → persist → read → render) with a test, per the silent-drop invariant.
 6. **Make tax-display mode region-aware** (inclusive EU/UK/AU, exclusive US/Canada); distinguish B2B vs B2C to drive reverse-charge + display.
 7. **Capture/display registration numbers** (VAT/GSTIN/HSN/place-of-supply, GST-HST-PST, ABN) on invoices where required; validate EU B2B VAT numbers and apply reverse charge.
-

7. Silent-lie bug watchlist — the calculator → quote → invoice → payment money pipeline

The product owner's stated top fear is a rounding/currency/tax mismatch silently producing a wrong number, or a "success" the system didn't actually do. These are every place in the audit where that can happen, traced down the pipeline:

#	Stage	What silently goes wrong	Evidence	Severity
1	Calculator → line	Pricing engine rounds intermediate unit prices to 4dp then the model re-rounds to 2dp with float math; compounds across many items	<code>ProductPricingCalculator.php:209-210</code> → <code>Quote.php:231</code>	LOW
2	Quote/Order/Invoice total	Total accessor rounds to hardcoded 2 decimals , ignoring snapshotted <code>currency_decimals</code> — wrong for JPY(0)/KWD(3) before the gateway is ever called	<code>Quote.php:219, :231</code>	MEDIUM
3	Tax (storefront)	Storefront taxes all items (<code>is_taxable = tax_enabled</code>) ignoring per-product <code>is_taxable</code> ; same product taxed differently via admin vs storefront	<code>StorefrontCheckoutController.php:394</code> vs <code>OrderController.php:881</code>	HIGH
4	Tax total	No persisted <code>tax_amount</code> ; total tax recomputed from per-line snapshots — a missing/inconsistent line snapshot silently changes the historical total	<code>Order.php:265-288</code> , <code>StorefrontCheckoutController.php:1024</code>	LOW
5	Order → gateway currency	Charge currency read from the gateway config , not <code>\$order->currency_code</code> ; X-amount charged labelled as Y; <code>supportsCurrency()</code> never checked	<code>CheckoutPaymentService.php:86-101</code> , <code>AbstractPaymentGateway.php</code>	CRITICAL
6	Amount → minor units	Hardcoded ×100 over/undercharges non-2-decimal currencies 100×/10×	<code>StripeGateway.php:45, :142</code> ; <code>RazorpayGateway.php:50, :316</code>	CRITICAL
7	Payment record	<code>payments</code> has no currency column; captured currency discarded — the wrong-currency charge is invisible in the DB, so it can't even be detected after the fact	<code>2025_02_05...create_payments_table.php</code> , <code>Payment.php</code> , <code>StripeGateway.php:220</code>	HIGH
8	Surcharge	<code>chargeCustomerProfile</code> silently adds a hardcoded 3.5% fee with no UI disclosure — customer is charged more than the invoice and the store owner can't see why	<code>PaymentService.php:438-440</code> (<code>round(\$amount * 1.035, 2)</code>)	HIGH
9	Refund (admin Orders UI)	All refund fields dropped by <code>OrderController::update</code> whitelist; no money moves but “Refund processed” is shown	<code>orders/[id].vue:281-290, :288</code> ; <code>OrderController.php:618-650, :656, :701</code>	CRITICAL
10	Partial refund (webhook)	Order/payment flipped to full REFUNDED for a partial refund; <code>REFUNDED_PARTIAL</code> never used though <code>refunded_amount</code> accumulates	<code>UpdatePaymentAndOrder.php:176-181</code> ; <code>PaymentStatusEnum.php:38</code>	HIGH

#	Stage	What silently goes wrong	Evidence	Severity
11	Refund double-count	Manual refund Payment not keyed on a webhook event_id → later <code>charge.refunded</code> creates a duplicate Payment row, double-counting <code>refunded_amount</code> in reports	<code>PaymentController.php:512</code> vs <code>UpdatePaymentAndOrder.php:128-147</code>	MEDIUM
12	Chargeback	Dispute only flips a status flag — no alert, no <code>respond_by</code> , no accounting; the store owner never learns of it and disputed orders keep producing	<code>UpdatePaymentAndOrder.php:183-187</code> ; <code>NotificationTypeEnum.php:13,31</code>	CRITICAL
13	Cross-tenant webhook	Unscoped gateway-config lookup can resolve the wrong tenant's config/secret and stamp the event with the wrong <code>tenant_id</code>	<code>PaymentEventDispatcher.php:69-73, :162-163</code>	HIGH
14	Out-of-order events	A late <code>failed</code> after <code>captured</code> can regress a PAID order (no state-machine guard)	<code>UpdatePaymentAndOrder.php:58-65</code>	MEDIUM
15	Dead-letter	A permanently-failing webhook stays <code>processed_at=null</code> forever once the gateway stops retrying — payment silently never reconciled	<code>PaymentEventDispatcher.php:241-265</code>	MEDIUM
16	Display	Admin <code>formatMoney</code> call-sites without symbol/decimals render <code>\$</code> for non-USD stores	<code>formatMoney.ts:17</code> ; <code>plan.monthly_charge / rate.rate / tx.amount</code> call-sites	MEDIUM

The common pattern (and the one the product owner cares about most): the system reports success or a number that looks right while the real money movement either didn't happen (#9), happened in the wrong currency/amount (#5, #6), or was silently marked-up (#8), with no stored evidence to detect it later (#7). Every fix below for these rows should ship with a round-trip test (submit → persist → read back → assert), per the [CLAUDE.md](#) silent-drop invariant.

8. Prioritized action plan

Ordered by (severity, then effort). **Do now** = compliance/active-loss; **Do next** = correctness gaps that produce wrong money; **Later** = hardening.

Priority	Fix	Severity	Effort	Files
Do now	Stop collecting CVV (remove from all FormRequests, drop columns, purge rows)	CRITICAL	M	<code>PaymentService.php</code> , all card-accepting controllers, <code>payment_profiles</code> / <code>tenant_payment_profiles</code> migrations
Do now	Stop storing full PAN; store last4 only	CRITICAL	M	<code>PaymentService.php:269, :181</code> ; <code>TenantPaymentProfile.php</code> ; migration
Do now	Encrypt gateway secrets at rest (<code>encrypted:array</code> casts)	CRITICAL	S	<code>StoreThirdPartyService.php:30-34</code> , <code>Admin\ThirdPartySetting</code>
Do now	Remove admin card-reveal UI; show masked last4 only	HIGH	S	<code>nuxt/app/components/customer/payment/profile.vue:25-43</code>
Do now	Fix admin Orders refund silent-lie — wire to a real <code>RefundService</code> via the gateway registry	CRITICAL	L	<code>orders/[id].vue</code> , <code>OrderController.php:618-650</code> , new <code>RefundService</code>
Do now	Charge in the order's currency + <code>supportsCurrency()</code> check	CRITICAL	S	<code>CheckoutPaymentService.php:86-101</code>
Do now	Currency-decimal-aware minor units (kill hardcoded $\times 100$)	CRITICAL	S	<code>StripeGateway.php:45, :142</code> ; <code>RazorpayGateway.php:50, :316</code>
Do now	Real chargeback handling (alert + <code>respond_by</code> SLA + freeze + accounting + audit)	CRITICAL	L	<code>PaymentDisputed</code> , <code>UpdatePaymentAndOrder.php:183-187</code> , <code>NotificationTypeEnum.php</code>
Do now	Disclose / remove the hidden 3.5% surcharge	HIGH	S	<code>PaymentService.php:438-440</code>
Do next	Route refunds through gateway abstraction (<code>non-Authorize.Net</code> stores can refund)	HIGH	M	<code>PaymentController.php:512</code> , <code>RefundService</code>
Do next	Honor per-product <code>is_taxable</code> in storefront checkout	HIGH	S	<code>StorefrontCheckoutController.php:394</code>
Do next	Add <code>currency</code> column to <code>payments</code> ; persist + reconcile captured currency	HIGH	S	<code>payments</code> migration, <code>Payment.php</code> , <code>StripeGateway.php:220</code>
Do next	Partial-refund status (<code>REFUNDED_PARTIAL</code>) + webhook-as-source-of-truth	HIGH	M	<code>UpdatePaymentAndOrder.php:176-181</code>
Do next	Re-scope webhook to a tenant after signature verify; assert order $\leftrightarrow$ config tenant match	HIGH	M	<code>PaymentEventDispatcher.php:69-73, :277-282</code> , <code>UpdatePaymentAndOrder.php:96-121</code>
Do next	Round document totals to <code>currency_decimals</code>	MEDIUM	S	<code>Quote.php:231</code> , <code>Order</code> / <code>Invoice</code> accessors
Do next	Sweep admin <code>formatMoney</code> call-sites to pass snapshot symbol/decimals	MEDIUM	M	<code>formatMoney.ts</code> call-sites
Do next	Resolve merchant-of-record/facilitator classification (legal/tax)	HIGH	—	(decision, not code)

Priority	Fix	Severity	Effort	Files
Later	Cross-path refund dedup (manual ↔ <code>charge.refunded</code> webhook)	MEDIUM	M	<code>PaymentController.php:512</code> , <code>UpdatePaymentAndOrder.php:128-147</code>
Later	Webhook state-machine guard (no PAID → FAILED regression)	MEDIUM	M	<code>UpdatePaymentAndOrder.php:58-65</code>
Later	Dead-letter / reconciliation sweep + wire <code>PaymentGatewayLogService</code>	MEDIUM	M	<code>PaymentEventDispatcher.php:241-265</code> , scheduled job
Later	<code>TaxService</code> + tax engine (Stripe Tax/Avalara) + region-aware display + tax-exempt customers	HIGH	XL	new <code>TaxService</code> , <code>StorefrontCheckoutController</code> , admin
Later	Void fallback for pre-settlement refunds	MEDIUM	S	<code>PaymentService.php:506</code>
Later	Throttle webhook route; verifyWebhook for PayPal/Paytm/AuthorizeNet	LOW	S	<code>routes/public-api.php:32</code> , gateway classes
Later	Integer minor-units / <code>bcmath</code> money math; consolidate the two gateway trees	LOW	XL	pricing + payment layers

9. Sources

PCI-DSS

- PCI SSC FAQ 1588 — new SAQ-A eligibility criteria (v4.0.1): <https://blog.pcisecuritystandards.org/faq-clarifies-new-saq-a-eligibility-criteria-for-e-commerce-merchants>
- PCI DSS v4.0.1 document library + Requirement 3 (Protect Stored Account Data): https://www.pcisecuritystandards.org/document_library/
- SAQ A-EP v4.0: <https://listings.pcisecuritystandards.org/documents/PCI-DSS-v4-0-SAQ-A-EP.pdf>
- Stripe PCI guidance + AOC: <https://stripe.com/guides/pci-compliance> · <https://stripe.com/resources/more/pci-attestation-requirements-and-process>
- Razorpay PCI/compliance: <https://razorpay.com/blog/payment-gateway-compliance/> · <https://razorpay.com/blog/what-is-pci-compliance-in-ecommerce/> · <https://razorpay.com/blog/what-is-pci-dss-compliance/>
- Reqs 6.4.3 / 11.6.1 (client-side script controls): <https://datadome.co/learning-center/pci-requirements-6-4-3-and-11-6-1/> · <https://www.sikich.com/insight/preparing-for-pci-dss-v4-0-1-requirements-6-4-3-and-11-6-1/> · <https://jscrumbler.com/blog/pci-dss-4-0-1>
- SAQ-A iframe-trap analysis: <https://www.akamai.com/blog/security/pci-dss-v4-0-1-changes-qualify-saq-a> · <https://trustedsec.com/blog/the-hidden-trap-in-the-pci-dss-saq-a-changes> · <https://hyperproof.io/resource/pci-dss-4-0-update-new-saq-a-eligibility-criteria/>

Refunds & disputes

- Stripe: <https://docs.stripe.com/refunds> · <https://docs.stripe.com/api/refunds> · <https://docs.stripe.com/changelog/acacia/2024-10-28/refund-webhook-update> ·

https://docs.stripe.com/api/idempotent_requests · <https://docs.stripe.com/disputes> · <https://docs.stripe.com/disputes/how-disputes-work> · <https://docs.stripe.com/disputes/categories> · <https://support.stripe.com/questions/june-2025-pricing-updates-for-disputes>

- Razorpay: <https://razorpay.com/docs/payments/refunds/> · <https://razorpay.com/docs/api/refunds/> · <https://razorpay.com/docs/webhooks/refunds/> · <https://razorpay.com/docs/payments/disputes/> · <https://razorpay.com/docs/payments/disputes/submit-evidence/> · <https://razorpay.com/docs/webhooks/payloads/disputes/>
- Distributed-payments correctness: <https://medium.com/airbnb-engineering/avoiding-double-payments-in-a-distributed-payments-system-2981f6b070bb>

Tax (sales tax / VAT / GST)

- US economic nexus: <https://www.avalara.com/us/en/learn/guides/state-by-state-guide-economic-nexus-laws.html> · <https://www.avalara.com/blog/en/north-america/2025/06/states-eliminating-economic-nexus-transaction-thresholds.html>
- Stripe Tax: <https://docs.stripe.com/tax/monitoring> · <https://docs.stripe.com/tax/supported-countries/united-states> · <https://docs.stripe.com/tax/registering>
- EU VAT / OSS / ViDA: <https://www.vatcalc.com/eu/eu-limits-e10000-vat-e-commerce-b2c-distance-selling-threshold-use-2025-vat-in-the-digital-age/> · https://taxation-customs.ec.europa.eu/taxation/vat/vat-directive/place-taxation_en · <https://hellotax.com/blog/vat-oss-schemes/> · <https://www.vatcalc.com/eu/eu-2028-scraps-e150-import-consignment-threshold-and-io-ss-limit/> · ViDA package, adopted 11 March 2025
- UK: <https://sterlingandwells.com/blogs/uk-vat-registration-for-overseas-companies/> · UK VAT threshold £90,000 (HMRC, 2025-26)
- India GST: <https://gstforecom.com/igst-vs-cgst-sgst-in-2025-a-guide-for-e-commerce-sellers/> · <https://tallysolutions.com/gst/e-invoicing-threshold/>
- Canada: <https://www.fonoa.com/resources/blog/when-to-charge-gsthst-in-canada>
- Australia: <https://www.ato.gov.au/businesses-and-organisations/international-tax-for-business/gst-for-non-resident-businesses/gst-on-low-value-imported-goods>
- Print-product taxability: <https://www.nj.gov/treasury/taxation/pdf/pubs/sales/anj18.pdf> (NJ printing-industry fact sheet; MN/VA equivalents)

10. Files audited

Contracts & abstraction

- `app/Contracts/PaymentGatewayInterface.php` , `app/Contracts/SubscriptionGatewayInterface.php`
- `app/Services/Integrations/Payment/AbstractPaymentGateway.php` , `PaymentGatewayRegistry.php`
- `app/Services/Integrations/IntegrationResolver.php`

Storefront payment gateways

- `app/Services/Integrations/Payment/{StripeGateway,RazorpayGateway,PayPalGateway,PaytmGateway,AuthorizeNetGateway,ChequeGateway}.php`

Subscription/billing layer

- `app/Services/BillingService.php` ,
`app/Services/Billing/{SubscriptionGatewayRegistry,StripeBillingGateway,RazorpayBillingGateway,AuthorizeNetBillingGateway,ChequeBillingGateway}.php`

Legacy **Authorize.Net** engine + admin payment

- `app/Services/PaymentService.php` ,
`app/Http/Controllers/Api/Payment/{PaymentController,PaymentWebhookController}.php`

Webhook / event pipeline

- `app/Services/Payment/{PaymentEventDispatcher,PaymentGatewayLogService}.php`
- `app/Listeners/Payment/UpdatePaymentAndOrder.php`
- `app/Events/Payment/{AbstractPaymentEvent,PaymentRefunded,PaymentDisputed}.php`
- `app/Support/Payment/VerifiedPaymentEvent.php` , `app/Models/PaymentWebhookEvent.php`
- `app/Providers/{AppServiceProvider,EventServiceProvider}.php` , `bootstrap/app.php` ,
`routes/{public-api,api}.php` , `config/payment-drivers.php`

Checkout / pricing / currency / tax

- `app/Services/Storefront/Checkout/{CheckoutPaymentService,CheckoutPricingService}.php`
- `app/Services/Storefront/Cart/StorefrontCartService.php`
- `app/Services/Storefront/Context/StorefrontCurrencyService.php` ,
`app/Helpers/CurrencyHelper.php`
- `app/Services/Pricing/ProductPricingCalculator.php`
- `app/Http/Controllers/Api/Storefront/StorefrontCheckoutController.php`
- `app/Http/Controllers/Api/{Order/OrderController,Invoice/InvoiceController,Customer/PaymentProfileController,Admin/TenantController}.php`

Models & enums

- `app/Models/{Payment,Order,Quote,Invoice,StoreThirdPartyService,TenantPaymentProfile,PaymentProfile}.php`
- `app/Enums/Invoice/PaymentStatusEnum.php` , `app/Enums/Notification/NotificationTypeEnum.php`

Migrations

- `2025_02_05_185503_create_payments_table.php` , `2026_05_02_000001_update_payments_system.php`
- `2025_03_30_104325_create_payment_profiles_table.php` ,
`2025_04_09_173425_add_cvv_column_in_payment_profiles_table.php` ,
`2025_05_31_182508_create_tenant_payment_profiles_table.php`
- `2026_06_04_000001_add_currency_snapshot_to_documents.php`
- `2025_12_21_000002_create_store_core_tables.php` , `2025_12_15_201408_create_orders_table.php` ,
`2025_04_12_103533_add_is_taxable_in_items.php`

Frontend (Nuxt)

- `nuxt/app/pages/orders/[id].vue` , `nuxt/app/composables/useOrdersApi.ts`
- `nuxt/app/utils/formatMoney.ts` , `nuxt/app/composables/store-management/useStoreCurrency.ts`

- `nuxt/app/components/customer/payment/profile.vue`
- `frontstore/app/composables/{useTheme,useThemeSettings}.ts` , `frontstore/app/utils/price.ts`
- `frontstore/app/components/checkout/CheckoutPayment.vue` ,
`frontstore/app/components/checkout/payment-drivers/StripeElementsDriver.vue`

Existing docs

- `readme/PAYMENT_GATEWAYS.md` , `readme/RAZORPAY_INTEGRATION_GUIDE.md`

Production Readiness & Reliability Reference

Source: [readme/research/engineering-ops/PRODUCTION_READINESS_RELIABILITY_2026-06-15.md](#)

IN SHORT — KEY TAKEAWAYS

This document is an honest check-up of the platform's safety net — what happens if something goes wrong. Right now, if the database server crashes or a file gets lost, there is no way to get the data back. This matters because print jobs are physical: a lost order means a real customer gets the wrong product, or nothing at all, after they already paid.

The main parts explained simply:

- **No database backups** — If the database were deleted or corrupted today, every tenant's orders, customers, payments, and products would be gone forever. There is nothing in place to recover them. This is the most urgent problem in the whole document.
- **Background jobs run inside the web request (sync mode)** — When the server processes a paid order, it also does extra work (like generating a PDF or sending an email) right then and there. If anything goes wrong mid-way, that work is lost silently — even if the customer already paid. Jobs should run separately in the background with automatic retries.
- **Print files can go missing without warning** — When a customer downloads their order files, if a print-ready PDF is missing, the system quietly skips it and sends the download anyway. The customer gets an incomplete ZIP, and the print job can start with no production file — with no alert to staff.
- **Most errors are invisible** — Only the PDF service has proper error tracking. If the main website or the design tool crashes, nobody finds out. Problems go unnoticed until a customer complains.
- **No Redis / job queue** — Redis is a fast memory store that holds background jobs safely. It is not set up, so there is no retry system, no job history, and no way to recover work that failed.
- **No health monitoring or uptime checks** — There is nothing watching the platform from the outside to alert staff if the site goes down, a scheduled task silently stops, or backups fail.
- **Dev settings running in production** — The environment is still configured for development (debug mode on, real payment credentials mixed in). This increases exposure if something goes wrong.
- **No disaster recovery plan** — There is no written, tested runbook for what to do when something breaks. A backup that has never been tested is not a real backup.

What to do with this:

Set up automatic database backups first — this is the single change that prevents permanent data loss. Then switch background jobs to run asynchronously (with Redis and a queue manager) so paid-order work is never lost silently. Everything else in the document builds on those two foundations.

Canonical reliability reference for Print-Flow-360 (Laravel 11 + PostgreSQL, admin Nuxt, storefront Nuxt, Vue designer, Node pdf-service with BullMQ). Authored 2026-06-15. Based on a current-state codebase scan + web best-practice research. **Why this doc is serious:** print jobs are PHYSICAL. A lost order, a missing print-ready file, or a re-run of a paid job costs real money and ships the wrong thing (or nothing) to a paying customer. Reliability here is part of the fulfillment contract, not an ops nicety.

1. Executive summary

In plain language: **the platform can currently lose paying customers' data and their print-ready files with no way to get them back.** The code is well-structured (clean tenancy, services, snapshots) but the reliability *infrastructure* around it is mostly absent or wired to development-grade defaults. One app (`pdf-service`) is genuinely well-instrumented; almost nothing else is.

Top risks, most dangerous first

1. **No database backups at all (CRITICAL — data loss).** No backup package is installed (`spatie/laravel-backup` is absent from `composer.json`), no `pg_dump` / `pg_basebackup` /WAL archiving, no scheduled backup task in `routes/console.php` , and no restore path. A search of the codebase for backup/restore/dump/snapshot returns zero matches. If either PostgreSQL instance (`live_db` central, `admin_db`) is lost or corrupted, **every tenant's orders, customers, payments, and products are gone permanently.** This is the single most dangerous finding.
2. **QUEUE_CONNECTION=sync (CRITICAL — paid-job loss).** All 18 queued jobs run *inline inside the web request* (`.env:57` , `.env.example:57`). There is no async execution, no retry, no `failed_jobs` capture on a mid-request crash, and a PHP timeout silently destroys the work with no trace. For anything that produces or notifies about a paid customer's file, a transient hiccup = permanently lost work, possibly *after* payment succeeded.
3. **Paid orders can be downloaded without their print-ready PDF, silently (CRITICAL — wrong/incomplete fulfillment).** `OrderController.downloadArtwork()` checks each file exists and **silently continues past missing ones** (`OrderController.php:504-513`). If the `print_ready_file` link is lost (the DB update is "best-effort" and never throws — `printReadyPersistService.js:39-59`), the customer's ZIP is missing the production artwork with no warning, and the print job can proceed without production-grade files.
4. **Error monitoring is one-app-only (HIGH — blind to failures).** Only `pdf-service` has real observability (Sentry + Prometheus + health checks + correlation IDs). Laravel has local file logs + an `error_logs` DB table but **no Sentry/APM**. The storefront has Honeybadger (deferred-load, possibly unwired). The designer and docs apps have **zero** error monitoring. Backend API and designer failures are effectively invisible.
5. **Single PostgreSQL host, no replica, no failover, no PITR (HIGH).** `config/database.php` has no `read` replica array and no secondary connection; central and admin DBs sit on the same `localhost:5432` . There is no point-in-time recovery, so even with a future nightly dump the best achievable RPO would be a full day.
6. **Redis not configured; BullMQ effectively off (HIGH).** No `REDIS_*` vars in either app; cache driver is `database` . The pdf-service has full BullMQ retry/backoff infrastructure but it is **disabled by default** (no `REDIS_HOST`), so PDF generation also runs synchronously.

Environment note: `.env` shows `APP_ENV=development` , `APP_DEBUG=true` , file cache/session, but contains real AWS/Stripe credentials — i.e. this is a dev-configured environment in active use. The recommendations below assume you will stand up a properly-configured production environment; do not ship these dev defaults.

2. Current state

2.1 Backups / Disaster Recovery

Area	Exists today	Evidence	Missing / risk
Backup tooling	Nothing	<code>composer.json</code> (no <code>spatie/laravel-backup</code>); <code>app/Console/Commands</code> (26 commands, none backup); <code>grep backup/restore/dump/snapshot = 0</code> hits	No logical or physical backup of any DB
Scheduled backups	None	<code>routes/console.php:1-71</code> (only auth-clear, sanctum-prune, reminders, campaigns, invoices)	No backup window; nothing offsite
PITR / WAL archiving	None	<code>config/database.php</code> (no <code>read /replica/failover</code>); <code>grep WAL = 0</code>	Cannot recover to a point in time
Read replica / HA	None	<code>config/database.php</code> (single host <code>localhost:5432</code>)	DB host is a single point of failure
File (S3) durability	S3 is primary disk	<code>config/filesystems.php:48-59</code> , <code>.env:56</code> , bucket <code>printflow360</code>	Versioning/replication/Object-Lock not verified/enabled ; tenancy filesystem bootstrapper disabled (<code>config/tenancy.php:34</code>) so tenants likely share a bucket without disk-level isolation
Test DB sandbox	Separate <code>printflow360_test</code>	<code>.env.testing:35-45</code>	Useful as a restore-drill target (not yet used as one)
Recovery capability	None	<code>grep</code> results; no backup packages	Only recovery is manual DBA intervention

2.2 Error monitoring / observability

App	Exists today	Evidence	Missing / risk
pdf-service	Excellent: Sentry (uncaught/unhandled/5xx), Prometheus <code>/metrics</code> , <code>/health</code> + <code>/health/ready</code> + <code>/health/startup</code> , X-Request-Id correlation, Pino JSON logs	<code>pdf-service/src/lib/sentry.js:1-105</code> , <code>index.js:43,79,87-98</code> , <code>lib/metrics.js</code> , <code>routes/health.js:13-54</code> , <code>middleware/requestLogger.js:8-28</code>	None significant
Laravel API	Moderate: 11 log channels, <code>ErrorLogService</code> persists 5xx + unhandled to <code>error_logs</code> table, built-in <code>/up</code> health, partial X-Request-Id (forwarded to pdf-service only)	<code>bootstrap/app.php:27,82-134</code> , <code>config/logging.php</code> , <code>app/Services/ErrorLog/ErrorLogService.php:1-165</code> , <code>PdfServiceClient.php:303-305</code>	No Sentry/APM , no global request-id middleware, no <code>request_id</code> on <code>error_logs</code> , <code>/up</code> has no custom checks
Storefront (Nuxt)	Weak: Honeybadger, deferred-loaded on idle; client-error ingest endpoint exists	<code>frontstore/.../honeybadger.client.ts:1-24</code> , <code>StorefrontErrorLogController.php</code> , <code>package.json:40</code>	Honeybadger may not be reporting; no Sentry; no confirmed client → backend error flow
Admin (Nuxt)	(not separately evidenced)	—	No confirmed error monitoring
Designer (Vue)	None	<code>designer/package.json</code> (no Sentry/Honeybadger)	Zero visibility into a customer-facing editor
Docs	None / unknown	<code>docs/package.json</code>	Likely none
Distributed tracing	Partial: X-Request-Id pdf ↔ Laravel only	<code>PdfServiceClient.php:280-309</code>	No Sentry/Datadog/Jaeger trace stitching across the stack

2.3 Queues & background work

Area	Exists today	Evidence	Missing / risk
Laravel queue driver	<code>sync</code> (inline, no async)	<code>.env:57</code> , <code>.env.example:57</code>	No retries, no isolation, mid-request crash = silent loss
failed_jobs table	Defined but only fed on thrown exceptions	<code>0001_01_01_000002_create_jobs_table.php:37-45</code> , <code>config/queue.php:106-110</code>	Useless under <code>sync</code> for crashes/timeouts
Per-job retry/backoff	Almost none (2/18 set anything)	<code>SendEmailCampaignJob.php:23, 25 (\$tries=1)</code> , <code>CampaignDispatcherJob.php:22 (\$timeout=300)</code>	16 jobs have no tries/backoff/timeout
Horizon	Not installed	<code>composer.json</code> (no <code>laravel/horizon</code>)	No queue UI, retry, or worker health
Redis	Not configured	no <code>REDIS_*</code> in <code>.env</code> / <code>.env.example</code> ; <code>config/cache.php:18</code> default database	No persistent/Redis queue; BullMQ can't run
pdf-service BullMQ	Built (3 attempts, exp backoff, 7d failed / 24h completed retention) but disabled	<code>pdf-service/src/queue/pdfQueue.js:35-40</code> , <code>config/env.js:43-53</code>	No <code>REDIS_HOST</code> → PDF jobs run sync
Laravel → pdf-service calls	Synchronous HTTP, retry only on connection error (2x, 5s connect/30s req)	<code>PdfServiceClient.php:268-275</code>	No recovery on generation failure/timeout; imposition also sync (<code>:235-239</code>)
pdf-service flags	All delegation flags default off ; silent fallback to DOMPDF (<code>strict_mode=false</code>)	<code>config/pdf_service.php:46-57</code>	No print jobs use pdf-service yet
Cross-system job tracking	None	<code>pdf-service/src/db/models/PDFJob.js</code> (own DB, not linked to <code>failed_jobs</code>)	No end-to-end job recovery between Laravel and Node
Worker concurrency	<code>WORKER_CONCURRENCY=2</code> (unused while BullMQ off)	<code>pdf-service/src/config/env.js:51</code>	Potential bottleneck for bulk print once enabled

2.4 Print-file durability & integrity

Area	Exists today	Evidence	Missing / risk
Generation & storage	300-DPI PDFs (PDFKit+sharp) → per-tenant S3 or local; new timestamped file per regen	<code>pdfStorageService.js:40-66</code> , <code>S3Storage.js:54-69</code>	Old files never deleted → file proliferation on retries
Link to design	<code>persistPrintReadyFile()</code> writes <code>designer_documents.print_ready_file</code> ; order snapshots it at checkout w/ live fallback	<code>commit 12847afa</code> , <code>printReadyPersistService.js:28-60</code> , <code>StorefrontCheckoutController::resolveDesignPrintReadyFile()</code>	DB link write is best-effort, never throws (<code>:39-59</code>) → silent link loss
Corruption validation	Only <code>buffer.length > 0</code>	<code>pdfStorageService.js:82-84</code> , <code>pdfGenerator.js:205-211</code>	Malformed-but-nonzero PDF can be stored & marked complete
Idempotency	None — each retry regenerates a NEW file	<code>pdfQueue.js:36-39</code> , <code>pdfStorageService.js:25-26,81-94</code>	No dedup; orphan files accumulate
Order download w/ missing file	Silently skipped	<code>OrderController.php:504-513</code>	Customer ZIP missing print PDF, no indication (CRITICAL)
Paid-order-without-file scenario	Snapshot may be null + link may be lost → no artwork, no alert	<code>OrderController.php:428-440,504-513</code> , <code>printReadyPersistService.js:56-59</code>	Print job runs without production artwork (CRITICAL)
Secret coupling	<code>APP_KEY</code> + <code>INTERNAL_API_SECRET</code> must match Laravel exactly	<code>tokenService.js</code> , <code>internalAuth.js</code> , <code>config/env.js:22,41,88-89</code>	Mismatch → 401 on all <code>/internal/*</code> (ops footgun)

2.5 Health / uptime

Area	Exists today	Evidence	Missing / risk
pdf-service probes	liveness/readiness/startup with per-dependency status + latency	<code>routes/health.js:13-54</code> , <code>lib/healthcheck.js:17-124</code>	None
Laravel health	Built-in <code>/up</code> only	<code>bootstrap/app.php:27</code>	No readiness check (DB/Redis/queue/disk/backup), no token-gated JSON
Nuxt apps	—	—	No <code>/livez</code> / <code>/readyz</code> evidenced
External uptime/synthetics	None evidenced	—	No UptimeRobot/Better Stack, no checkout/PDF synthetic, no backup/scheduler heartbeat

2.6 Deploy / infrastructure

Area	Exists today	Evidence	Missing / risk
Environment posture	Dev-grade config in active use	<code>.env:1-50,161</code> (<code>APP_ENV=development</code> , <code>APP_DEBUG=true</code> , file cache/session, <code>SUBSCRIPTION_MODE=local</code>)	Not a hardened production env
Tenancy DR nuance	DB-per-tenant via stancI; <code>DatabaseTenancyBootstrapper</code> disabled	<code>config/tenancy.php:32,42-78</code> , <code>app/Models/Tenant.php:12-19</code>	Restore plan must cover central + tenant data consistently
IaC / secrets backup	None evidenced	—	No infra-as-code / config backup tier
Migration rollback strategy	274 central migrations, no documented rollback/backup-before-migrate	<code>database/migrations</code>	Risky schema changes have no safety net

3. Target architecture & recommendations

Sized for a **small team**: prefer managed and cheap; avoid over-engineering (no active/active, no Kubernetes complexity until revenue demands it).

3.1 Backups / DR — adopt a 3-tier backup model

- **Tier 1 — PostgreSQL PITR (the must-have).** Use **pgBackRest** (gold-standard, all-in-one) or **WAL-G** (simplest to S3, ideal under ~100GB) for *continuous archiving*: weekly base backup + daily incremental + WAL archiving with `archive_timeout ≈ 300s`. This gives ~5-minute RPO. If you'd rather not self-host, a **managed Postgres with PITR** (RDS/Aurora 1s–35d, Cloud SQL, DigitalOcean daily+WAL 7d, Neon) removes the operational load — strongly preferred for a small team. **Avoid Supabase free tier for production data — it has no backups/PITR.**
- **Tier 2 — Offsite logical net.** Add `spatie/laravel-backup` for nightly `pg_dump` ZIPs (DB + app files) to a **separate S3 bucket in another region**, encrypted, with **Object Lock** (immutability). This is portable and easy for single-table/single-tenant restores, but is NOT a PITR replacement (only as fresh as the last dump).
- **Tier 3 — Print-file protection.** Enable **S3 bucket versioning + cross-region (or 2nd-bucket) replication + Object Lock** on the `printflow360` bucket. Make file deletion **soft** (quarantine / lifecycle-expire after a window longer than backup retention) so a restored older DB row still finds its object.
- **Follow 3-2-1-1-0:** 3 copies, 2 media, 1 offsite, 1 immutable, 0 errors. Retention: daily 7–30d / weekly 4–12w / monthly 12mo.
- **DB + object-store consistency on restore (the project's #1 trap).** The DB stores *relative paths* (`HasImageFields` / `FileHelper`); a point-in-time DB restore does NOT roll S3 back, producing orphaned files (safe) or **missing files** (the customer-facing silent-lie class this codebase guards against). Mitigate with versioning + replication + soft-deletes + a **post-restore reconciliation job** that diffs every DB file path against the bucket **per tenant base path**, restores prior versions for missing objects, and lists orphans —

surfaced in plain language, never a broken download. Also re-verify order/quote/invoice *snapshot* file references resolve after restore.

- **“A backup you’ve never restored isn’t a backup.”** Automate a **weekly restore drill** (cron/CI: restore base+WAL into a throwaway DB → integrity checks: key-table row counts, FK/constraint validation, app smoke test → DB-vs-bucket path diff → report → teardown). Use `printfLOW360_test` as the sandbox. Measure and trend: restore duration vs RTO, recovered point vs RPO, validation pass/fail, integrity diff, manual-step count.
- **DR strategy tier:** start at robust **Backup & Restore** with continuous archiving (hits the targets below cheaply). Graduate to **Pilot Light** (always-on replicated DB + IaC-deployable app) only when an hours-long RTO becomes unacceptable.

3.2 Error monitoring / observability — standardize on Sentry across all apps

- **Laravel:** `composer require sentry/sentry-laravel`, `php artisan sentry:publish`, wire `Integration::handles($exceptions)` in `bootstrap/app.php`. Keep `send_default_pii=false`; redact email/phone/address/card via `before_send` + server-side Data Scrubbing. Tag **tenant/store/user by UUID only** (matches `HasUuid`) — never email/name. Add a global request-id middleware and store `request_id` on `error_logs`.
- **Both Nuxt apps:** `@sentry/nuxt` as **separate Sentry projects** (admin vs storefront). Upload source maps on production build (the most common setup failure if skipped). Session Replay: `replaysSessionSampleRate=0`, `replaysOnErrorSampleRate=1.0`, and **do not ship replay on** `/checkout`, `/cart`, `/profile` **without masking review** (customer PII). Server-side Sentry needs the built `sentry.server.config.mjs` loaded via `--import`.
- **Designer (Vue) and docs:** add `@sentry/vue` — currently zero visibility into a customer-facing editor.
- **pdf-service:** already excellent; just add Sentry if not on (`@sentry/node`, `instrument.js` required FIRST, `setupExpressErrorHandler` after routes).
- **Distributed tracing:** propagate `sentry-trace` + `baggage`; add the pdf-service internal host to `trace_propagation_targets` so a Laravel → pdf-service trace stitches. Use **one canonical SENTRY_RELEASE (git SHA) and identical SENTRY_ENVIRONMENT** across all four apps or traces won’t stitch.
- **Cost control:** prefer a `traces_sampler` (1.0 for errors/checkout/admin/slow paths; ~0.05–0.1 normal; 0 for `/up`, health, static, and high-volume `/info` / `/promotional-bars` /ISR routes). Sample traces, not errors.
- **Alerting:** gate on rate/severity/tags, not “any new issue”; ignore expected exceptions (Validation/404/Auth/client-abort); mark `vendor/` / `node_modules` out-of-app for grouping.

3.3 Queues — Redis + Horizon, idempotent jobs, alerting

- Set `QUEUE_CONNECTION=redis` (treat `sync` as dev-only). Run **Laravel Horizon** with a **dedicated supervisor + queue for print-file generation** so an email backlog never starves file work. Keep `failed_jobs` as the dead-letter store; prune only after alerting/retry workflows exist.
- **Dedicated, persistent, HA Redis for queues** (separate instance from cache): `appendonly yes`, `appendfsync everysec`, `maxmemory-policy=noeviction` (the only policy that keeps queue correctness), plus a replica + Sentinel/cluster. Many managed Redis ship without persistence and *with* eviction — you must set these explicitly. A clean worker restart re-runs in-flight jobs (safe if idempotent); Redis data loss without AOF loses all unpersisted jobs.

- **Idempotency is the #1 correctness rule** (both Laravel & BullMQ are at-least-once). The file-producing job must key on `order_id + file_version` (or content hash): on entry, if the print-ready file already exists, **no-op and reuse** — never regenerate, double-charge, or re-notify. Persist a “generated” marker (DB unique constraint/upsert).
- **Transactional enqueue:** set `'after_commit' => true` (or `->afterCommit()`) so a job never runs before the order+payment rows commit; consider the transactional-outbox pattern for the Laravel → Node handoff.
- **Bound every job:** `$tries` (e.g. 5), array `$backoff` with jitter (`[10,30,60,120]`), `$timeout` < worker timeout, `$failOnTimeout=true`. Wrap flaky external calls (pdf-service, S3) with `ThrottlesExceptions`. Implement `failed()` to set the order’s file status to `generation_failed`, notify staff, and give the customer a recovery action — never leave a paid order silently stuck.
- **BullMQ (pdf-service):** enable by setting `REDIS_HOST`; keep attempts + exponential backoff + **jitter**; bound `removeOnComplete` (e.g. `{age:3600,count:1000}`) and keep `removeOnFail` generous (failed print jobs = paid orders needing manual recovery); trap SIGTERM → `worker.close()` for graceful drain on deploy; raise `lockDuration` above worst-case render time to avoid false stalls; return only the **relative/S3 path** in `returnValue` (repo rule). Model the pipeline (preflight → render → thumbnail → notify) with `FlowProducer` + fail-parent.
- **Alert on three signals per queue:** backlog depth, oldest-job age (>5 min on the print-file queue = paid customers waiting), and failure/dead-letter growth. Page a human on any failed paid-order job.

3.4 Print-file durability — close the silent-loss gaps

- **Never silently skip a missing print file.** Fix `OrderController.downloadArtwork()` (`:504-513`) to surface a plain-language error + recovery path (re-generate / contact support) instead of `continue`.
- **Make the print-ready link write reliable.** `persistPrintReadyFile()` must retry and, on persistent failure, fail the job / flag the order — not swallow the error (`printReadyPersistService.js:39-59`).
- **Add PDF integrity validation** beyond `length>0`: verify PDF header/EOF marker (and ideally a checksum) before marking a job complete.
- **Add idempotency + cleanup** so retries reuse the existing file rather than proliferating timestamped orphans.
- **Pre-fulfillment guard:** before a print job is actioned, assert the print-ready artwork resolves in storage; block/alert if not.

3.5 Health / uptime — 3-tier probes + external watchers

- **Laravel:** keep `/up` as liveness; add `spatie/laravel-health` readiness (DB, Redis, queue, disk, backup-age) returning token-gated 200/503 JSON, plus custom S3 + pdf-service checks.
- **Nuxt + Node:** expose `/livez` (process only) and `/readyz` (deps with parallel timeouts). **Never put dependencies in a liveness probe** (a DB blip must not trigger a restart loop).
- **External:** point **UptimeRobot** or **Better Stack** at the readiness endpoints; **Healthchecks.io** heartbeats on backups, the scheduler, and the BullMQ worker (catches “the cron silently stopped”); a **Checkly/Playwright synthetic** for checkout + PDF generation (the revenue paths). Alerts: dedup + escalate + link a runbook.

3.6 Deploy / infrastructure

- Stand up a real production env: `APP_ENV=production`, `APP_DEBUG=false`, Redis-backed cache/session.

- **Infra-as-code in git** + nightly config/secret backup (Tier 2 of the backup model).
- **Backup before migrate** for risky schema changes; document rollback per migration.
- Keep `APP_KEY` and `INTERNAL_API_SECRET` / `PDF_SERVICE_INTERNAL_SECRET` in sync across Laravel and pdf-service (mismatch = 401 storm).
- Write the **DR runbook**: activation criteria; named roles (Incident Commander / Restore Operator / Comms Lead) with backups; call tree (DB host, S3, payment, DNS providers); decision tree by failure type; numbered copy-pasteable restore steps (provision host → restore base+WAL to target time → repoint at S3 → run reconciliation → repoint DNS); validation checklist; post-incident actual-vs-target review. Add a **single-tenant accidental-deletion branch** (S3 versioning + `pg_dump` filtered by `tenant_id`) — far cheaper than full PITR. Keep it in version control; rehearse during game days.

Recommended RTO/RPO targets

Tier	Data / system	RPO target	RTO target	How achieved
1 Critical	PostgreSQL (orders, customers, payments, products)	5 min	1–4 hrs	Daily base backup + WAL archiving (<code>archive_timeout=300s</code>) for PITR; restore to new host
1 Critical	S3 print-ready files / designs / proofs	near-zero (last write)	1–4 hrs	S3 versioning + cross-region/2nd-bucket replication
2 Important	App config, IaC, secrets, queue state	24 hrs	4–8 hrs	Infra-as-code in git + nightly config/secret backup
3 Low	Thumbnails, derived/cache assets, regenerable renders	24 hrs+	best-effort	Regenerate via pdf-service; replicate only if cheap

DB RPO can tighten to ~1 min (lower `archive_timeout` or add streaming replication) but 5 min is the cost/benefit sweet spot. Sub-30-min RTO needs warm standby — overkill until revenue justifies it.

4. Prioritized action plan

P0 = data-loss / paid-job-loss risks. One ordered list.

Priority	Item	Why it matters for this platform (paid-print-job risk)	Rough effort
P0	Stand up PostgreSQL backups: managed PITR or self-hosted pgBackRest/WAL-G (weekly base + daily + WAL to offsite S3, encrypted)	Today a DB loss wipes every tenant's orders/customers/payments permanently — there is no recovery at all	1–3 days (managed faster)
P0	Add Tier-2 <code>spatie/laravel-backup</code> nightly dump to a 2nd-region, Object-Lock bucket	Offsite immutable copy survives ransomware/accidental drop; enables per-tenant restore	0.5 day
P0	Enable S3 versioning + replication + Object Lock on <code>printflow360</code> ; make file deletes soft	Lost/overwritten print-ready files = wrong physical product to a paying customer; lets restore find old objects	0.5–1 day
P0	Switch <code>QUEUE_CONNECTION</code> off <code>sync</code> (→ Redis) + run Horizon with a dedicated print-file queue	Inline jobs lose paid work on any crash/timeout, sometimes after payment, with no <code>failed_jobs</code> trace	1–2 days (+Redis)
P0	Fix silent print-file gaps: error (not silent skip) in <code>downloadArtwork()</code> (<code>:504-513</code>); make <code>persistPrintReadyFile()</code> retry/fail instead of swallow	A paid order can be “downloaded” or printed with the production PDF silently missing	1 day
P0	First automated restore drill + DR runbook (incl. DB ↔ S3 reconciliation, per-tenant)	An untested backup is not a backup; proves you can actually recover paid-order data before you need to	1–2 days
P1	Idempotent file-generation job (key on order+file version; reuse existing file) + <code>afterCommit</code> dispatch	At-least-once retries must not regenerate, double-charge, or re-notify; must not run before payment commits	1 day
P1	Sentry across Laravel + both Nuxt apps + designer (UUID-only tags, PII scrubbed, shared release/env)	Backend/designer failures are currently invisible — paid-order errors go unnoticed	1–2 days
P1	Persistent HA Redis for queues (AOF, <code>noeviction</code> , replica), separate from cache; enable BullMQ in pdf-service	Redis is the durability boundary for queued paid-order work; default eviction/no-persistence loses jobs	1 day
P1	Failure alerting: Horizon failed jobs + BullMQ failed set + queue depth/age → page staff; set order file status + recovery path	A failed paid-order file must page a human and never leave the order silently stuck	1 day
P1	<code>spatie/laravel-health</code> readiness + external uptime (Better Stack/UptimeRobot) + <code>Healthchecks.io</code> heartbeats on backups/scheduler/worker	Detects a silently-dead scheduler/worker/backup before customers do	1 day
P1	Bound all 18 jobs (tries/backoff+jitter/timeout/ <code>failed()</code> handler)	Email/SMS/notification failures currently vanish with no retry	0.5–1 day
P2	PDF integrity validation (header/EOF/checksum) before marking complete	Prevents storing a malformed-but-nonzero “print-ready” file that prints as garbage	0.5 day

Priority	Item	Why it matters for this platform (paid-print-job risk)	Rough effort
P2	Old print-file cleanup / dedup to stop timestamped orphan proliferation	Controls storage cost from repeated regen; complements idempotency	0.5 day
P2	Checkout + PDF-generation synthetic monitor (Checkly/Playwright)	Catches a broken revenue path before a customer hits it	0.5–1 day
P2	Production env hardening (<code>APP_ENV=production</code> , <code>APP_DEBUG=false</code> , IaC, secret backup) + backup-before-migrate	Removes dev-grade exposure and schema-change risk	1 day
P2	Read replica / Pilot Light DB (graduate from backup-restore)	Only when hours-long RTO becomes unacceptable; not yet warranted	2–4 days

5. References

PostgreSQL backups / PITR

- <https://www.postgresql.org/docs/current/continuous-archiving.html>
- <https://www.kunalganglani.com/blog/postgresql-backup-tools-compared>
- <https://www.percona.com/blog/postgresql-backup-strategy-enterprise-grade-environment/>
- <https://www.jusdb.com/blog/postgresql-point-in-time-recovery-pitr>
- <https://dev.to/mohhddhassan/postgresql-backups-and-point-in-time-recovery-with-pgbackrest-13gp>

Backup strategy / 3-2-1 / Laravel backup

- <https://spatie.be/docs/laravel-backup/v10/installation-and-setup>
- <https://opsiocloud.com/blogs/cloud-backup-strategy-3-2-1-rule-guide/>

RTO/RPO & DR strategy

- <https://www.veeam.com/blog/recovery-time-recovery-point-objectives.html>
- <https://www.ninjaone.com/blog/define-rto-and-rpo-across-backup-tiers/>
- <https://acecloud.ai/blog/rto-and-rpo-disaster-recovery/>
- <https://docs.aws.amazon.com/whitepapers/latest/disaster-recovery-workloads-on-aws/disaster-recovery-options-in-the-cloud.html>
- <https://aws.amazon.com/blogs/architecture/disaster-recovery-dr-architecture-on-aws-part-i-strategies-for-recovery-in-the-cloud/>
- <https://learn.microsoft.com/en-us/azure/well-architected/design-guides/disaster-recovery>

Restore testing / runbooks / DB ↔ S3 consistency

- <https://aws.amazon.com/blogs/storage/implementing-restore-testing-for-recovery-validation-using-aws-backup/>
- <https://aws.amazon.com/blogs/aws/automatic-restore-testing-and-validation-is-now-available-in-aws-backup/>
- <https://docs.aws.amazon.com/aws-backup/latest/devguide/restore-testing.html>

- <https://www.baculasystems.com/blog/backup-recovery-testing/>
- <https://aws.amazon.com/blogs/storage/consistent-point-in-time-restore-for-amazon-s3-buckets/>
- <https://docs.aws.amazon.com/aws-backup/latest/devguide/point-in-time-recovery.html>
- https://www.smartsheet.com/sites/default/files/IC-Disaster-Recovery-Runbook-10506_PDF.pdf
- <https://matoffo.com/how-to-create-a-disaster-recovery-dr-runbook-a-comprehensive-guide/>
- <https://www.datto.com/blog/optimize-rto-and-rpo/>

Sentry (Laravel / Nuxt / Express / tracing)

- <https://docs.sentry.io/platforms/php/guides/laravel/>
- <https://docs.sentry.io/platforms/php/guides/laravel/configuration/options/>
- <https://docs.sentry.io/platforms/php/guides/laravel/data-management/sensitive-data/>
- <https://docs.sentry.io/platforms/php/guides/laravel/tracing/distributed-tracing/>
- <https://nuxt.com/modules/sentry>
- <https://docs.sentry.io/platforms/javascript/guides/nuxt/manual-setup/>
- <https://docs.sentry.io/platforms/javascript/guides/nuxt/sourcemaps/>
- <https://docs.sentry.io/platforms/javascript/guides/express/>
- <https://docs.sentry.io/platforms/javascript/guides/vue/tracing/configure-sampling/>
- <https://docs.sentry.io/concepts/data-management/event-grouping/>
- <https://develop.sentry.dev/sdk/foundations/state-management/scopes/>
- <https://blog.sentry.io/automate-group-and-get-alerted-a-best-practices-guide-to-monitoring-your>

Queues — Laravel / Horizon / idempotency / outbox

- <https://laravel.com/docs/12.x/queues>
- <https://laravel.com/docs/11.x/horizon>
- <https://medium.com/@aiman.asfia/idempotency-in-laravel-12-2025-the-complete-guide-that-will-save-you-from-double-charges-3-am-0135d93f6dea>
- <https://medium.com/@mohhddhassan/transactional-outbox-idempotency-in-laravel-exactly-once-effects-over-at-least-once-f4bae734d75f>
- <https://prateeksha.com/blog/queues-that-dont-fail-laravel-queue-design-retries-backoff-observability>

Queues — BullMQ / Redis durability

- <https://docs.bullmq.io/guide/retrying-failing-jobs>
- <https://docs.bullmq.io/guide/workers/stalled-jobs>
- <https://docs.bullmq.io/guide/going-to-production>
- <https://docs.bullmq.io/guide/metrics>
- <https://redis.io/tutorials/operate/redis-at-scale/persistence-and-durability/>
- <https://www.dragonflydb.io/blog/top-5-reasons-why-your-redis-instance-might-fail>

Health / uptime / monitoring

- <https://spatie.be/docs/laravel-health>
- <https://uptimerobot.com/> · <https://betterstack.com/> · <https://healthchecks.io/> · <https://checklyhq.com/>

Local source-of-truth files

- `CLAUDE.md` , `readme/PDF_SERVICE.md` , `app/Services/PdfService/PdfServiceClient.php` ,
`config/pdf_service.php`

Multi-Tenant Isolation & Access-Control Security Review — 2026-06-15

Source: [readme/research/engineering-ops/SECURITY_MULTITENANT_ISOLATION_2026-06-15.md](#)

IN SHORT — KEY TAKEAWAYS

This document is a security check of how Print-Flow-360 keeps each store's data separate from every other store's data. It found several serious problems where one store owner could accidentally — or deliberately — read or change another store's data. These problems are real and confirmed in the code, not just theoretical worries.

The main parts explained simply:

- **Shared database with filters** — All stores share the same database. The only thing keeping Store A from seeing Store B's data is a software filter that runs on every database query. If that filter is ever skipped or turned off, any store can read any other store's data. There is no deeper safety net.
- **Fail-open isolation** — "Fail-open" means: if the filter is not set up correctly, it does nothing and lets everything through. Right now, if a page or background job forgets to set up the filter, all data from all stores is visible. There is no fallback that blocks the leak.
- **Cross-store data tampering (IDOR)** — Two specific pages in the admin panel (for managing order statuses and job statuses) let any store admin change or delete another store's custom statuses just by guessing a number in the web address. This is confirmed and live.
- **Master backdoor OTP** — There is a hardcoded backup login code (702702) baked into the system. If the environment is not configured to override it, anyone who knows this number can bypass two-factor login for any admin account.
- **Designer access tokens never expire** — The security tokens used by the design studio have a bug where expired tokens are accepted forever instead of being rejected. Anyone with an old token keeps access indefinitely.
- **CORS wildcard** — The system allows any website on the internet to make requests to the API on behalf of a logged-in user. This means a malicious website could silently act as the user.
- **No login rate limits on the customer storefront** — The customer login page has no limit on how many times someone can try to guess a password. An attacker can try thousands of passwords automatically.
- **Designer asset library leaks** — Customers' uploaded design files are visible to all customers in the design studio, not just the person who uploaded them.
- **Background jobs run without store context** — Tasks that run in the background (like sending emails) often forget to set up the store filter, so they can accidentally query or affect the wrong store's data.
- **Security tokens do not expire** — Customer login tokens never time out. If a token is stolen, it works forever.

What to do with this:

Fix the three critical items first: (1) add an ownership check to the order-status and job-status edit pages so they refuse to touch another store's records, (2) remove the hardcoded master OTP backdoor entirely, and (3) fix the designer token bug so expired tokens are rejected. These three fixes close the only confirmed live exploits before any other hardening work begins.

Staff security review of how Print-Flow-360 keeps one tenant's data away from another, and where it currently fails. Scope: tenant resolution, Eloquent isolation, Sanctum auth, CORS, rate limiting,

queue/Octane context, and IDOR. Audience: backend engineers + ops. Every claim is grounded with `file:line`. Anything that does not yet exist in code is labelled **RECOMMENDATION**.

1. Executive summary

Isolation model (proven fact, not assumption): Print-Flow-360 is **single-database, shared-schema** multitenancy. This is not a guess — **every stancI/tenancy bootstrapper is commented out** in `config/tenancy.php:31-37` (no `DatabaseTenancyBootstrapper`, no `CacheTenancyBootstrapper`, no `QueueTenancyBootstrapper`, no `FilesystemTenancyBootstrapper`). So `tenancy()->initialize()` only sets the `tenant()` context object; it never swaps the DB connection, cache prefix, or filesystem root. **All 113 tenant-owned models share one Postgres schema and 121 migrations carry a `tenant_id` column.** Isolation depends *entirely* on the `BelongsToTenant` global scope (`vendor/stancI/tenancy/src/Database/TenantScope.php:14-22`) adding `where tenant_id = tenant()->getTenantKey()` to every query.

Two parallel tenant-resolution paths:

- **Admin / store-admin API** — tenant comes from the *authenticated user*, not the host (`app/Http/Middleware/InitializeTenancy.php:19-20`). A stolen admin token therefore cannot be replayed against another tenant's host — the token's own tenant always wins. This is a genuine strength.
- **Storefront API** — tenant comes from the *request* (`app/Http/Middleware/InitializeStorefrontTenancy.php:55-79`): `X-Tenant` header first, then `X-Store-Id` / `store_id`, then `getHost()` last. The header is attacker-controllable and is trusted *ahead* of the real host.

The single biggest risk: the isolation posture is **fail-open**. `TenantScope::apply()` early-returns and filters *nothing* when tenancy was never initialized (`TenantScope.php:17-18`). Combined with two controllers that deliberately strip the scope (`OrderStatusController` / `JobStatusController` using `withoutGlobalScopes()->findOrFail($numericId)` with **no tenant ownership re-check**), there are live, exploitable **cross-tenant IDOR** holes today. Add a hardcoded `MASTER_OTP` 2FA backdoor and an inverted designer-token expiry check, and there are three independently critical findings.

RAG verdict on cross-tenant isolation: ● **RED**. The strong path (admin token → user-derived tenancy) is sound, but it is undermined by (a) two confirmed cross-tenant IDOR write paths on sequential integer IDs, (b) a fail-open scope with no DB-level backstop, and © a 2FA bypass and forever-valid designer tokens. None of these are theoretical — each has a concrete exploit and a concrete fix below.

Dimension	Verdict	Why
Admin token tenant binding	 Green	Tenancy derived from <code>Auth::user()->tenant_id</code> (<code>InitializeTenancy.php:19</code>) — host replay can't cross tenants
Customer token tenant binding	 Amber	Blocked only by app-layer <code>EnsureCustomerBelongsToCurrentStore</code> ; no DB token scoping
Eloquent scope coverage	 Red	Two controllers bypass the scope on numeric IDs; fail-open when uninitialized
Storefront tenant resolution	 Amber	<code>X-Tenant</code> header trusted above host, no allowlist
Auth hardening	 Red	<code>MASTER_OTP</code> backdoor, never-expiring <code>*</code> customer tokens, wildcard CORS
Rate limiting	 Amber	Admin auth throttled; customer auth surface unthrottled

2. How tenant resolution & isolation actually work today

2.1 Resolution chain

Path 1 — Admin & store-admin (`routes/api.php` , `routes/store-api.php`)

- `routes/tenant.php:20` wraps `routes/api.php` in `InitializeTenancy::class` ; `routes/api.php:101` wraps the same content *again* in `InitializeTenancy` (dual loading — see §7).
- `InitializeTenancy.php:19-20` : `if (Auth::check() && Auth::user()->tenant_id) tenancy()->initialize(Auth::user()->tenant_id)` . **Host/header/subdomain are irrelevant here.**
- Admin route stack: `['auth:sanctum', 'valid.auth', 'tenant.subscription', 'tenant.approved']` (`routes/api.php:122`).

Path 2 — Storefront (`routes/frontstore.php`)

- `InitializeStorefrontTenancy.php:55-79` resolves a domain string in priority order: (1) `X-Tenant` header (`:58`), (2) `X-Store-Id / store_id` → `Store.uuid` → linked domain (`:63`), (3) `getHost()` (`:78`).
- Resolved domain → `domains` table (cached 5 min, `:93-97`), then subdomain fallback against `{sub}`. `{app.base_domain}` (`:136-156`).
- On success: `tenancy()->initialize($tenant)` and stashes `store_id / store_uuid / store` on request attributes (`:33-37`).
- A third fallback exists: `initFromBearerToken` (`:183-208`) initializes tenancy from an admin api-guard token when no header is present (designer cross-origin case). Scoped to the user's own tenant, so not cross-tenant.

2.2 Model scoping (the only real enforcement layer)

- Tenant-owned models use `BelongsToTenant` (e.g. `Order.php:24` , `OrderStatus.php:10` , `Notification.php:11`). `bootBelongsToTenant()` (`vendor/.../BelongsToTenant.php:22-34`) adds `TenantScope` and **auto-stamps** `tenant_id` **on create** from `tenant()->getTenantKey()` .

- `TenantScope::apply()` (`vendor/.../TenantScope.php:14-22`) adds the `where(tenant_id = ...)` clause **only if** `tenancy()->initialized` **is true** — otherwise it is a no-op and **nothing is filtered**.
- Models are UUID-keyed via `HasUuid` / `HasUuids` (`app/Traits/HasUuid.php:26-29` sets `getRouteKeyName()='uuid'`), so route-model-bound endpoints bind by UUID and inherit the scope. **The exception is the status controllers, which take a raw numeric {id}** (`routes/api.php:458-459` , `491-492`).

2.3 Cache isolation (application-level, NOT stanc-level)

- `CacheTenancyBootstrapper` is **off** (`config/tenancy.php:33`), so there is no automatic per-tenant cache tag.
- `CacheService` prefixes every key with `store_{id}:v{ver}:` (`app/Services/CacheService.php:44-49`) and tags `store_{id}` (`:60-63`). Store id comes from `StoreHelper::getCurrentStore()` (`CacheService.php:21`).
- **Fallback hazard:** unset store context resolves to the literal `'default'` (`CacheService.php:21` , `:25-26`), collapsing two tenants into a shared `store_default:` namespace.

2.4 Storage isolation

- `FilesystemTenancyBootstrapper` is **off**, so disk roots are *not* tenant-suffixed automatically. Isolation of files depends on application-level per-tenant paths (e.g. store-theme-branding writes `{tenant base}/branding/brand.css`). Verify those paths are relative and tenant-prefixed; an absolute path defeats isolation.

2.5 Queue isolation

- `QueueTenancyBootstrapper` is **off** (`config/tenancy.php:35`), so **queued jobs do NOT auto-restore tenant context**. Any Eloquent query on a `BelongsToTenant` model inside a job that did not call `tenancy()->initialize()` runs **unscoped across all tenants** (because `TenantScope` is a no-op when uninitialized).
- Only `ProcessInboundEmail.php:46` and `SendEmailCampaignJob.php:71` call `tenancy()->initialize()` . `SendAppNotification.php` does **not** — it relies on hand-written `tenant_id` filters.

2.6 Auth scoping

- Sanctum Bearer tokens (not SPA cookie sessions). Stateful domains list only localhost (`config/sanctum.php:18`) — confirming token mode, not cookie mode.
 - `personal_access_tokens` is a **shared** table; Sanctum's `findToken()` looks up by token-hash **globally with no tenant filter**. The custom `App\Models\PersonalAccessToken` (`AppServiceProvider.php:185`) adds only `last_used_at` throttling — **no BelongsToTenant** , **no findToken() override**. Cross-tenant token safety rests on (a) 40-char token randomness and (b) the app-layer middleware (`EnsureCustomerBelongsToCurrentStore.php:42-56` , `InitializeTenancy.php:19`).
-

3. Cross-tenant attack matrix

#	Vector	Verdict	Current defense	Remediation
1	Cross-tenant IDOR on Order statuses (PUT/DELETE/reorder by numeric id)	 vulnerable	NONE — <code>OrderStatus::withoutGlobalScopes()->findOrFail(\$id)</code> on raw numeric id, no tenant re-check (<code>OrderStatusController.php:86,110,142,169</code>)	After <code>findOrFail</code> , <code>abort(404)</code> unless <code>!\$status->is_system && (string)\$status->tenant_id === (string)tenant()->getTenantKey()</code>
2	Cross-tenant IDOR on Job statuses	 vulnerable	NONE — identical pattern (<code>JobStatusController.php:82,103,131,154</code>)	Same fix as #1
3	Universal admin 2FA bypass via MASTER_OTP default 702702	 vulnerable	NONE when env unset — <code>\$request->otp != env('MASTER_OTP', 702702)</code> accepted for any user (<code>AuthController.php:368</code>)	Remove the path; if break-glass needed, fail closed on empty env, restrict to non-prod, audit-log
4	Designer-auth token expiry inverted — expired tokens accepted forever	 vulnerable	NONE (logic bug) — <code>decodeTokenInternal</code> returns payload when expired (<code>DesignerAuthTokenService.php:90-98</code>)	<code>return null</code> when expired under <code>!\$allowExpired</code> ; throttle <code>issueAuthToken</code>
5	CORS wildcard + credentials → any origin/XSS reads API with victim token	 likely-vulnerable	WEAK — <code>paths/methods/origins/headers = ['*']</code> , <code>supports_credentials=true</code> (<code>config/cors.php:18-32</code>); tokens in non-httpOnly cookies (<code>useCustomerAuth.ts:56</code>)	Explicit origin allowlist + <code>allowed_origins_patterns</code> ; narrow <code>paths</code> to <code>['api/*','sanctum/csrf-cookie']</code> ; httpOnly cookies
6	Customer login enumeration + no rate limit → credential stuffing	 likely-vulnerable	WEAK — distinct messages (<code>CustomerAuthController.php:62-275</code>); no throttle on storefront auth routes	Generic “Invalid email or password”; add <code>storefront-login</code> limiter keyed email+IP
7	Designer asset library leaks one customer’s uploads to all	 vulnerable	PARTIAL — <code>applyContextScope</code> commented out in <code>getAssets</code> (<code>DesignerController.php:969</code>); returns all tenant assets	Re-enable owner scoping for customer-uploaded assets; model shared clipart separately
8	Storefront tenant spoof via X-Tenant / X-Store-Id (reads)	 likely-vulnerable	WEAK/by-design — header trusted ahead of host (<code>InitializeStorefrontTenancy.php:58</code>)	Require internal secret on header-based resolution; otherwise fall back to <code>getHost()</code>
9	Storefront WRITE attributed to arbitrary tenant via spoofed header	 likely-vulnerable	WEAK — write routes under same middleware; <code>BelongsToTenant</code> auto-stamps spoofed <code>tenant_id</code>	For unauthenticated writes, validate resolved tenant matches connecting host

#	Vector	Verdict	Current defense	Remediation
10	Cross-tenant customer token replay	🟡 likely-blocked	<code>EnsureCustomerBelongsToCurrentStore.php:42-46</code> (string compare on <code>tenant_id</code>) on the sole <code>auth:customer + customer.store</code> group (<code>routes/frontstore.php:362</code>)	Add meta-test: every <code>auth:customer</code> route must also carry <code>customer.store</code> ; scope token lookup by <code>tenant_id</code>
11	Cross-tenant admin token replay	🟢 likely-blocked	By design — tenancy from <code>Auth::user()->tenant_id</code> (<code>InitializeTenancy.php:19</code>)	Keep; add scope assertion test
12	Queued/console job runs unscoped	🟡 unverified	FRAGILE — <code>QueueTenancyBootstrapper</code> off (<code>config/tenancy.php:35</code>); scope no-op when uninitialized	Require jobs touching tenant models to call <code>tenancy()->initialize()</code> ; add static lint
13	X-Forwarded-Host spoof drives storefront resolution	🟡 unverified	PARTIAL — trusts proxies from <code>127.0.0.1</code> +private ranges (<code>bootstrap/app.php:58-63</code>)	Confirm edge strips client <code>X-Forwarded-Host</code>
14	Octane stale tenant carried into next request	🟡 unverified	NONE if Octane enabled — HTTP path never calls <code>tenancy()->end()</code>	Confirm Octane not in prod; if it is, reset tenancy per request
15	Domain → tenant 5-min cache serves old tenant after reassignment	🟡 unverified	WEAK — keyed by domain string, no invalidation (<code>InitializeStorefrontTenancy.php:93-97</code>)	Bust <code>storefront_domain:{domain}</code> on reassignment
16	Cache key collision via <code>'default'</code> store fallback	🟡 unverified	WEAK — <code>?? 'default'</code> (<code>CacheService.php:21</code>)	Fail closed (throw) instead of <code>'default'</code> ; ensure <code>setStoreId()</code> before <code>remember()</code>
17	Chat JWT forged with default key <code>123123</code>	🟡 unverified	NONE if default not overridden (<code>config/app.php:144</code> , <code>.env.example:105</code>)	Fail boot/mint if key equals <code>123123</code>
18	<code>valid.auth</code> no-op → single-session not enforced	🟡 unverified	WEAK — <code>EnsureValidAuth</code> body commented out (<code>EnsureValidAuth.php:11</code>)	Re-enable device binding or delete misleading middleware
19	Customer tokens never expire with <code>*</code> abilities	🟡 likely-vulnerable	WEAK — <code>expiration=null</code> (<code>config/sanctum.php:49</code>); <code>createToken('customer')</code> no expiry	Finite <code>expiration</code> ; <code>sanctum:prune-expired</code> daily; scoped abilities

4. Prioritized gaps & remediation

● CRITICAL

C1 — Cross-tenant IDOR on Order AND Job custom statuses. `update / destroy / reorder` use `Model::withoutGlobalScopes()->findOrFail($numericId)` with no tenant ownership re-check (`OrderStatusController.php:110/142/169`, `JobStatusController.php:103/131/154`). Routes bind a raw integer `{order_status} / {job_status}` (`routes/api.php:458-459`, `491-492`), so a Tenant A admin can mutate/delete Tenant B's workflow statuses by guessing sequential ids. **Fix (RECOMMENDATION):** after `findOrFail`, guard every single-row write:

```
abort_unless(
    !$status->is_system && (string) $status->tenant_id === (string) tenant()->getTenantKey(),
    404
);
```

For `reorder`, filter each id to `(tenant_id = current OR is_system)` before updating. Keep `index()`'s `withoutGlobalScopes()` **only** to show `is_system` rows (`OrderStatusController.php:22-25`) — never to write across tenants. Add `tests/Feature/Security/CrossTenantIsolationTest.php` (Deliverable 2) asserting block + DB unchanged.

C2 — Universal admin 2FA bypass via MASTER_OTP default 702702. `AuthController.php:368` accepts the hardcoded default for any user when the env is unset (it is absent from `.env / .env.example`). **Fix (RECOMMENDATION):** remove the `MASTER_OTP` path entirely. If a break-glass is genuinely required: fail closed when empty (never default to `702702`), gate with `app()->environment()` (non-prod only), and audit-log every use. Separately, stop `Log::info`-ing plaintext OTPs (`AuthController.php:285,346,367`) and replace `rand()` with `random_int()` in `resendOtp` (`:339`).

C3 — Designer-auth token expiry inverted. `DesignerAuthTokenService::decodeTokenInternal` (`:90-98`) returns the payload when the token is expired under `!$allowExpired`. Every signed designer token is valid forever, and `issueAuthToken` (unauthenticated, no throttle, `DesignerController.php:95`) re-mints from expired tokens. **Fix (RECOMMENDATION):** change the expired branch to `return null;`; require a positive `exp`; add a throttle to `issueAuthToken`; require the `customer` guard for customer-subject tokens. Unit-test: an expired payload decodes to `null`.

● HIGH

H1 — CORS fully wildcard with credentials. `config/cors.php:18-32`. **Fix (RECOMMENDATION):** set `allowed_origins` to the explicit admin+storefront origins, `allowed_origins_patterns` to the tenant-subdomain regex, narrow `paths` to `['api/*','sanctum/csrf-cookie']`, restrict `allowed_methods / allowed_headers`. Move auth tokens out of non-httpOnly cookies (server-set httpOnly, or in-memory). Add a CI guard rejecting `['*']` origins with credentials.

H2 — Customer storefront auth: no rate limit + enumeration. `routes/frontstore.php` login/register/forgot-password unthrottled; `CustomerAuthController.php:262-275` leaks existence + provider. **Fix (RECOMMENDATION):** add a `storefront-login` `RateLimiter` keyed email+IP: `[Limit::perMinute(500), Limit::perMinute(5)->by(strtolower($email).'.'.$ip)]`; IP throttles on register/forgot-password/google callback; Redis limiter store. Return one generic "Invalid email or password" for

not-found/no-password/wrong-password; drop the Google-login hint. Surface 429 in the Vue composable with a plain-language retry message (per [CLAUDE.md](#) §0 no-raw-technical-output).

H3 — Designer asset list leaks customer uploads. `DesignerController.php:969` (`applyContextScope` commented out). **Fix (RECOMMENDATION):** re-enable owner scoping for customer-uploaded assets (`saveAsset` / `uploadFile` origin). Model shared clipart as an explicit admin-curated set; return shared set + caller's own only. `deleteAsset` is already owner-scoped (`:1026-1027`) — make the list consistent.

H4 — Storefront tenant identity selectable via attacker headers (read + write).

`InitializeStorefrontTenancy.php:58` . **Fix (RECOMMENDATION):** only honor `X-Tenant` / `X-Store-Id` when the request carries a shared internal secret header (set by SSR/edge); otherwise fall back to `getHost()` . For unauthenticated writes, validate the resolved tenant matches the connecting host.

H5 — Fail-open isolation across all unscoped paths. Single-DB, all bootstrappers off

(`config/tenancy.php:31-37`); jobs without `initialize()` , raw `DB::table()` missing a predicate, `withoutGlobalScopes` , and the legacy `App\Models\Admin\*` duplicates lacking `BelongsToTenant` all leak silently. **Fix (RECOMMENDATION):** add the cross-tenant data-provider matrix test (Deliverable 2); audit every `DB::table()` on a tenant table for a `tenant_id` / `store_id` predicate (add a lint); delete or add `BelongsToTenant` to `App\Models\Admin\*` duplicates; require tenant-touching jobs to `initialize()` .

H6 — Chat JWT default key + never-expiring customer tokens. `config/app.php:144` (`123123`), `config/sanctum.php:49` (`null`). **Fix (RECOMMENDATION):** fail boot/mint if chat key equals `123123` ; set finite Sanctum `expiration` ; schedule `sanctum:prune-expired` daily; scope abilities.

🟡 MEDIUM / LOW

M1 — Defense-in-depth that is one mistake from breaking: customer cross-tenant replay depends solely on `EnsureCustomerBelongsToCurrentStore` ; `valid.auth` is a dead no-op; `X-Forwarded-Host` trust depends on the edge; the 5-min domain cache has no invalidation. **Fix (RECOMMENDATION):** meta-test asserting every `auth:customer` route also has `customer.store` ; scope token lookup by `tenant_id` on `PersonalAccessToken` ; re-enable or delete `EnsureValidAuth` ; confirm the LB strips client `X-Forwarded-Host` ; bust `storefront_domain:{domain}` on reassignment.

L1 — Admin `TenantController` `withoutGlobalScopes` (`:493,526`) is legitimate landlord context — **confirm the route group is super-admin only** and unreachable by a tenant store admin.

5. OWASP A01 Broken Access Control & IDOR mapping

OWASP A01:2021 is the #1 web risk; IDOR (insecure direct object reference) is its most common concrete form. How it maps here and the cheat-sheet practices to adopt:

OWASP A01 sub-pattern	Where it shows up here	Practice to adopt
IDOR — object id from the request, no ownership check	<code>OrderStatusController</code> / <code>JobStatusController</code> numeric-id write paths (C1)	Enforce record-level ownership on <i>every</i> mutating handler; never trust <code>withoutGlobalScopes()</code> without re-adding a tenant predicate
Bypassing access control by modifying the URL / internal id	Sequential integer ids on status routes (<code>routes/api.php:458</code>)	Use UUID route binding everywhere (the platform's own <code>HasUuid</code> invariant); the status controllers are the exception to fix
Acting as another user/tenant (privilege escalation)	<code>X-Tenant</code> / <code>X-Store-Id</code> spoof (H4); <code>MASTER_OTP</code> (C2)	Resolve tenant from the trusted host, never client-controlled body/headers; deny by default
Metadata manipulation (tokens) for elevation	Forever-valid designer tokens (C3), chat JWT <code>123123</code> (H6)	Enforce token expiry; reject default signing keys; least-privilege abilities
CORS misconfiguration allowing API access from untrusted origins	Wildcard CORS + credentials (H1)	Explicit origin allowlist; never <code>['*']</code> with credentials
Force browsing to authenticated pages / API as unauthenticated	Public storefront writes under spoofable tenant (H4/H9)	Validate tenant ↔ host agreement on writes

Cheat-sheet practices we should adopt (OWASP Authorization / IDOR / Authentication / Forgot-Password):

1. Deny by default; enforce ownership server-side on every object access (not just list endpoints).
2. Prefer a neutral **404** over 403 for resources the caller shouldn't know exist — keeps enumeration-safe behavior (already the platform convention for designs, `DesignCustomerScopeTest.php:110`).
3. Single generic auth-failure message + uniform timing (kills enumeration, H2).
4. Layer rate-limiting *and* per-account temporary lockout with backoff; MFA is the strongest control (do not ship a static-code backdoor, C2).
5. Log access-control failures and significant security events to the existing audit log (per [CLAUDE.md](#) audit-logging invariant).

6. Sanctum hardening checklist for this app

Item	Status	Evidence / Action
Pick one auth mode per consumer	🟡 PARTIAL	Token (Bearer) mode in practice; stateful domains = localhost only (<code>config/sanctum.php:18</code>). TODO: decide cookie mode for first-party SPAs
<code>allowed_origins</code> is an explicit allowlist	❌ TODO	<code>['*']</code> with credentials (<code>config/cors.php:18-32</code>) — fix per H1
<code>paths</code> narrowed to <code>api/*</code> (+ csrf-cookie)	❌ TODO	<code>['*']</code> today
Auth tokens in httpOnly cookies / in-memory	❌ TODO	Non-httpOnly cookies (<code>useCustomerAuth.ts:56</code> admits it; <code>nuxt/app/stores/auth.ts:33</code>)
Token expiration set	❌ TODO	<code>expiration=null</code> (<code>config/sanctum.php:49</code>); customer <code>createToken</code> has no expiry. Admin tokens DO expire (1d/1mo, <code>User.php:89</code>)
<code>sanctum:prune-expired</code> scheduled	❌ TODO	Not scheduled — table grows unbounded
Least-privilege token abilities	❌ TODO	All tokens use <code>['*']</code> (<code>User.php:89</code> , <code>CustomerAuthController.php:331</code>)
Tokens revoked on logout	🟢 DONE	<code>currentAccessToken()->delete()</code> (<code>AuthController.php:411</code> , <code>CustomerAuthController.php:350</code>); login wipes prior tokens (<code>AuthController.php:301</code>)
Tenant binding of token lookup	❌ TODO	<code>personal_access_tokens</code> shared; no <code>findToken()</code> override (<code>PersonalAccessToken.php</code>). Add <code>tenant_id</code> scoping for defense-in-depth
<code>SESSION_SECURE_COOKIE=true</code> in prod	❌ TODO	<code>env(...)</code> default false (<code>config/session.php</code>)
<code>SESSION_DOMAIN</code> not a shared leading-dot parent	🟢 DONE	<code>.env.example</code> sets null (per-host) — safe for per-tenant isolation
CSRF cookie issued in tenant context	❌ TODO (if cookie mode adopted)	<code>sanctum 'routes'=>false</code> + re-register <code>/sanctum/csrf-cookie</code> in <code>routes/tenant.php</code>
No raw 429 to users	❌ TODO	Surface friendly retry message in Vue composables

7. stanc/tenancy edge-case checklist

Edge case	Status here	Action
Bootstrappers enabled	✗ ALL OFF (<code>config/tenancy.php:31-37</code>)	Single-DB scope is the <i>only</i> isolation; no DB/cache/queue/filesystem backstop. Treat the global scope as a load-bearing invariant
<code>PreventAccessFromCentralDomains</code> on tenant routes	⦿ N/A on storefront path	Storefront never consults <code>central_domains</code> ; <code>localhost</code> falls through to subdomain parse → <code>abort(404)</code> (<code>InitializeStorefrontTenancy.php:141-143</code>). The documented SSR- <code>localhost</code> bug is real; mitigation is <code>x-tenant-host</code> . TODO: add a central-domain short-circuit so it fails safe
Identification from trusted host, never client data	✗ Storefront trusts <code>X-Tenant</code> first (<code>:58</code>)	Fix per H4
Queue context restored in jobs	✗ <code>QueueTenancyBootsrapper</code> off	Jobs must <code>tenancy()->initialize()</code> ; pass UUIDs not serialized tenant models; re-fetch in <code>handle()</code>
Octane state reset between requests	⦿ <code>config/octane.php</code> exists; HTTP never <code>end()</code> s	Confirm Octane not in prod; if it is, reset tenancy per request
Cache isolation via per-tenant prefix/tag	⦿ App-level only (<code>CacheService</code> store prefix)	Never use <code>Cache</code> facade directly; ensure <code>setStoreId()</code> before <code>remember()</code> ; fail closed instead of <code>'default'</code>
Storage per-tenant root	⦿ App-level paths only	Store relative tenant-prefixed paths; no absolute paths in DB
Tenant resolver cache is central + invalidated	✗ 5-min domain cache, no invalidation (<code>:93-97</code>)	Bust on domain reassignment
<code>Tenant</code> model NOT using <code>BelongsToTenant</code>	⦿ Correct	Tenant must be resolvable without tenant context
PDF service (Node, shared Postgres) self-isolates	⦿ Outside Laravel bootstrappers	Every <code>PdfServiceClient</code> call must pass + honor the tenant key for DB and S3 prefixes

8. Rate-limiting & brute-force checklist

Protected today (`app/Providers/AppServiceProvider.php`):

Limiter	Definition	Applied at
api	:122 (300/min by user id)	global api group
login (admin)	:150 (Lockout event + ValidationException)	routes/api.php:113
admin forgot-password	inline throttle:5,1	routes/api.php:117
admin verify	throttle	routes/api.php:120
verification-notification	:134	verification resend
uploads	:138	upload routes
coupon-validate	:144	coupon validation

Unprotected — TODO:

Endpoint	Location	Action
Customer /auth/login	routes/frontstore.php:345	Add storefront-login limiter (email+IP); reCAPTCHA is per-store-optional (CustomerAuthController.php:245) so not a reliable control
Customer /auth/register	routes/frontstore.php:344	IP throttle (mass-signup spam)
Customer /customer/forgot-password	routes/frontstore.php:353	IP/email throttle
/auth/google/callback	routes/frontstore.php:346	IP throttle
issueAuthToken (designer)	routes/frontstore.php:90	Throttle token issuance
proxy-image (designer)	routes/frontstore.php:44	Throttle (SSRF-guarded :1497-1505 but open amplifier)

Production config: set `config/cache.php 'limiter' => 'redis'` so counters coordinate across nodes (file/array store silently lets distributed attacks through).

9. Cross-tenant penetration test plan

File: `tests/Feature/Security/CrossTenantIsolationTest.php` (Deliverable 2). **Run:** `vendor/bin/phpunit --filter CrossTenantIsolation` (uses `pgsql printflow360_test` via `.env.testing`; `DatabaseTransactions` + `Queue::fake()`, mirroring `DesignCustomerScopeTest` / `AccessControlTest`).

What it asserts:

- Direct scope assertion** — with tenancy initialized to A, `Order::find(B.id) === null` and counts exclude B's rows (proves `BelongsToTenant` is the real enforcement layer, per the no-manual-`where` invariant).

2. **API route IDOR by id** — `actingAs` an A admin (`sanctum`), PUT/DELETE `/api/v1/order-statuses/{B.id}` and `/api/v1/job-statuses/{B.id}` → expect **403/404** AND assert B's DB row unchanged/present (currently 200 + mutates — the C1 reproduction).
3. **Token/host replay** — an A admin requesting a B-owned resource by UUID resolves to A's tenancy, never B's (proves `InitializeTenancy` user-derivation, vector #11).
4. **Storefront customer scope** — customer A cannot read customer B's design (`/designer/designs/{uuid}` → 404) or notifications (forged id → safe no-op), reusing the established neutral-404 convention.
5. **Body assertions, not just status** — assert the payload omits B's data (`assertJsonMissing` / field absence), catching leaky-200/empty bugs.

Convention: neutral **404** for resources the attacker shouldn't know exist; **403** only where existence is already public. End tenancy in `tearDown` (guard with `if (tenancy()->initialized)`) to prevent cross-test contamination.

10. Sources

Codebase (this repo): `config/tenancy.php` , `config/cors.php` , `config/sanctum.php` , `config/app.php` , `bootstrap/app.php` , `app/Http/Middleware/InitializeTenancy.php` , `app/Http/Middleware/InitializeStorefrontTenancy.php` , `app/Http/Middleware/EnsureCustomerBelongsToCurrentStore.php` , `app/Http/Middleware/EnsureValidAuth.php` , `app/Http/Controllers/AuthController.php` , `app/Http/Controllers/Api/Storefront/CustomerAuthController.php` , `app/Http/Controllers/Api/Order/OrderStatusController.php` , `app/Http/Controllers/Api/Job/JobStatusController.php` , `app/Http/Controllers/Api/Storefront/DesignerController.php` , `app/Services/Storefront/DesignerAuthTokenService.php` , `app/Services/CacheService.php` , `app/Helpers/StoreHelper.php` , `vendor/stancl/tenancy/src/Database/TenantScope.php` , `routes/api.php` , `routes/frontstore.php` , `tests/Feature/Storefront/DesignCustomerScopeTest.php` , `tests/Feature/Security/AccessControlTest.php` .

External research:

- <https://laravel.com/docs/11.x/sanctum> · <https://laravel.com/docs/12.x/sanctum>
- <https://github.com/laravel/sanctum>
- <https://tenancyforlaravel.com/docs/v3/integrations/sanctum/>
- <https://tenancyforlaravel.com/docs/v3/tenant-identification/>
- <https://tenancyforlaravel.com/docs/v3/routes/>
- <https://tenancyforlaravel.com/docs/v3/queues/>
- <https://tenancyforlaravel.com/docs/v3/tenancy-bootstrappers/>
- <https://tenancyforlaravel.com/docs/v3/single-database-tenancy/>
- <https://tenancyforlaravel.com/docs/v3/cached-lookup/>
- <https://tenancyforlaravel.com/docs/v3/testing>
- <https://github.com/stancl/tenancy/issues/653>

- <https://github.com/spatie/laravel-multitenancy/issues/246>
- <https://laravel.com/docs/11.x/routing#rate-limiting> · <https://laravel.com/docs/11.x/rate-limiting>
- https://owasp.org/www-community/controls/Blocking_Brute_Force_Attacks
- https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
- https://cheatsheetseries.owasp.org/cheatsheets/Forgot_Password_Cheat_Sheet.html
- https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/04-Authentication_Testing/03-Testing_for_Weak_Lock_Out_Mechanism
- <https://owasp.org/API-Security/editions/2023/en/0xa2-broken-authentication/>
- <https://aliengiraffe.ai/blog/authentication-is-not-isolation-the-five-tests-your-multi-tenant-system-is-probably-failing/>
- <https://dev.to/sharjeelz/the-laravel-queue-multi-tenancy-trap-that-cost-me-3-hours-3c3d>